

Predicting Remaining Useful Life (RUL) of Turbofan Jet Engines

Machine Learning Project

Student: Karam Hasan & Lara Al Omari

Student ID: 100064840 & 100064797

Course: Introduction to Machine Learning (COSC434)

Course Instructor: Prof. Panos Liatsis

Lab Instructors: Dr. Ibrar Amin & Dr. Kunlin Yang

Introduction

This project aims to predict the Remaining Useful Life (RUL) of turbofan jet engines using the NASA C-MAPSS dataset.

This is a regression problem, where the goal is to estimate how many cycles remain before engine failure based on operational settings and sensor measurements.

Dataset Overview

The project uses four subsets: FD001, FD002, FD003, and FD004.

Each subset contains:

- Training trajectories (run-to-failure)
- Test trajectories (truncated before failure)
- True RUL labels for the test engines

The four datasets represent different levels of difficulty due to different operating conditions and fault modes. We analyze each dataset individually using the same pipeline.

FD001 - Data Loading and Initial Exploration

In this section, we begin with the FD001 dataset.

We first load the raw training, test, and RUL files, then inspect their structure before performing any preprocessing.

```
In [1]: ▶ import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load raw FD001 files
train_fd001_raw = pd.read_csv("data/CMAPS/train_FD001.txt", sep=r"\s+", header=None)
test_fd001_raw = pd.read_csv("data/CMAPS/test_FD001.txt", sep=r"\s+", header=None)
rul_fd001_raw = pd.read_csv("data/CMAPS/RUL_FD001.txt", sep=r"\s+", header=None)
```

```
In [2]: ▶ import sys
!{sys.executable} -m pip install optuna

Requirement already satisfied: optuna in c:\users\karam\anaconda3\envs\dsenv\lib\site-packa
ges (4.8.0)
Requirement already satisfied: alembic>=1.5.0 in c:\users\karam\anaconda3\envs\dsenv\lib\si
te-packages (from optuna) (1.18.4)
Requirement already satisfied: colorlog in c:\users\karam\anaconda3\envs\dsenv\lib\site-pac
kages (from optuna) (6.10.1)
Requirement already satisfied: numpy in c:\users\karam\anaconda3\envs\dsenv\lib\site-packag
es (from optuna) (2.2.6)
Requirement already satisfied: packaging>=20.0 in c:\users\karam\anaconda3\envs\dsenv\lib\s
ite-packages (from optuna) (26.0)
Requirement already satisfied: sqlalchemy>=1.4.2 in c:\users\karam\anaconda3\envs\dsenv\lib
\site-packages (from optuna) (2.0.49)
Requirement already satisfied: tqdm in c:\users\karam\anaconda3\envs\dsenv\lib\site-package
s (from optuna) (4.67.3)
Requirement already satisfied: PyYAML in c:\users\karam\anaconda3\envs\dsenv\lib\site-packa
ges (from optuna) (6.0.3)
Requirement already satisfied: Mako in c:\users\karam\anaconda3\envs\dsenv\lib\site-package
s (from alembic>=1.5.0->optuna) (1.3.10)
Requirement already satisfied: typing-extensions>=4.12 in c:\users\karam\anaconda3\envs\ds
env\lib\site-packages (from alembic>=1.5.0->optuna) (4.15.0)
Requirement already satisfied: tomli in c:\users\karam\anaconda3\envs\dsenv\lib\site-packag
es (from alembic>=1.5.0->optuna) (2.4.1)
Requirement already satisfied: greenlet>=1 in c:\users\karam\anaconda3\envs\dsenv\lib\site-
packages (from sqlalchemy>=1.4.2->optuna) (3.4.0)
Requirement already satisfied: colorama in c:\users\karam\anaconda3\envs\dsenv\lib\site-pac
kages (from colorlog->optuna) (0.4.6)
Requirement already satisfied: MarkupSafe>=0.9.2 in c:\users\karam\anaconda3\envs\dsenv\lib
\site-packages (from Mako->alembic>=1.5.0->optuna) (3.0.3)
```

Inspecting Raw Data

Before assigning column names or creating the target variable, we inspect the raw files to understand:

- number of rows and columns
- how the data looks
- whether the train, test, and RUL files were loaded correctly

```
In [3]: print("Raw training shape:", train_fd001_raw.shape)
print("Raw test shape:", test_fd001_raw.shape)
print("Raw RUL shape:", rul_fd001_raw.shape)

train_fd001_raw.head()
```

```
Raw training shape: (20631, 26)
Raw test shape: (13096, 26)
Raw RUL shape: (100, 1)
```

Out[3]:

	0	1	2	3	4	5	6	7	8	9	...	16	17	18	19	20
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	521.66	2388.02	8138.62	8.4195	0.0
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	522.28	2388.07	8131.49	8.4318	0.0
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	...	522.42	2388.03	8133.23	8.4178	0.0
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	...	522.86	2388.08	8133.83	8.3682	0.0
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	...	522.19	2388.04	8133.80	8.4294	0.0

5 rows × 26 columns



```
In [4]: test_fd001_raw.head()
```

Out[4]:

	0	1	2	3	4	5	6	7	8	9	...	16	17	18	19	20
0	1	1	0.0023	0.0003	100.0	518.67	643.02	1585.29	1398.21	14.62	...	521.72	2388.03	8125.55	8.4052	0.0
1	1	2	-0.0027	-0.0003	100.0	518.67	641.71	1588.45	1395.42	14.62	...	522.16	2388.06	8139.62	8.3803	0.0
2	1	3	0.0003	0.0001	100.0	518.67	642.46	1586.94	1401.34	14.62	...	521.97	2388.03	8130.10	8.4441	0.0
3	1	4	0.0042	0.0000	100.0	518.67	642.44	1584.12	1406.42	14.62	...	521.38	2388.05	8132.90	8.3917	0.0
4	1	5	0.0014	0.0000	100.0	518.67	642.51	1587.19	1401.92	14.62	...	522.15	2388.03	8129.54	8.4031	0.0

5 rows × 26 columns



Understanding Dataset Structure

From the raw data:

- Each row represents one engine at a specific cycle
- First columns correspond to:
 - engine ID
 - cycle number
 - operational settings
- Remaining columns are sensor measurements

We now assign meaningful column names.

```
In [5]: # define column names AFTER inspecting the data
columns = ['unit_nr', 'cycle', 'op_setting_1', 'op_setting_2', 'op_setting_3']

# add sensor names
for i in range(1, 22):
    columns.append(f'sensor_{i}')

print("Total columns:", len(columns))
columns
```

Total columns: 26

```
Out[5]: ['unit_nr',
        'cycle',
        'op_setting_1',
        'op_setting_2',
        'op_setting_3',
        'sensor_1',
        'sensor_2',
        'sensor_3',
        'sensor_4',
        'sensor_5',
        'sensor_6',
        'sensor_7',
        'sensor_8',
        'sensor_9',
        'sensor_10',
        'sensor_11',
        'sensor_12',
        'sensor_13',
        'sensor_14',
        'sensor_15',
        'sensor_16',
        'sensor_17',
        'sensor_18',
        'sensor_19',
        'sensor_20',
        'sensor_21']
```

```
In [6]: # assign column names to datasets
train_fd001 = train_fd001_raw.copy()
test_fd001 = test_fd001_raw.copy()
rul_fd001 = rul_fd001_raw.copy()

train_fd001.columns = columns
test_fd001.columns = columns

# RUL file has only one column
rul_fd001 = rul_fd001.iloc[:, [0]]
rul_fd001.columns = ['RUL']
```

```
In [7]: # verify structure after renaming
print(train_fd001.shape)
train_fd001.head()
```

(20631, 26)

Out[7]:

	unit_nr	cycle	op_setting_1	op_setting_2	op_setting_3	sensor_1	sensor_2	sensor_3	sensor_4	sensor_5	..
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	..
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	..
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	..
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	..
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	..

5 rows × 26 columns

In [8]: `test_fd001.head()`

Out[8]:

	unit_nr	cycle	op_setting_1	op_setting_2	op_setting_3	sensor_1	sensor_2	sensor_3	sensor_4	sensor_5	..
0	1	1	0.0023	0.0003	100.0	518.67	643.02	1585.29	1398.21	14.62	..
1	1	2	-0.0027	-0.0003	100.0	518.67	641.71	1588.45	1395.42	14.62	..
2	1	3	0.0003	0.0001	100.0	518.67	642.46	1586.94	1401.34	14.62	..
3	1	4	0.0042	0.0000	100.0	518.67	642.44	1584.12	1406.42	14.62	..
4	1	5	0.0014	0.0000	100.0	518.67	642.51	1587.19	1401.92	14.62	..

5 rows × 26 columns



Basic Data Exploration

Before creating the target variable, we explore the dataset:

- check data types
- check for missing values
- understand feature distributions

In [9]: `# check data types and non-null counts`
`train_fd001.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20631 entries, 0 to 20630
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   unit_nr                20631 non-null  int64
1   cycle                  20631 non-null  int64
2   op_setting_1           20631 non-null  float64
3   op_setting_2           20631 non-null  float64
4   op_setting_3           20631 non-null  float64
5   sensor_1               20631 non-null  float64
6   sensor_2               20631 non-null  float64
7   sensor_3               20631 non-null  float64
8   sensor_4               20631 non-null  float64
9   sensor_5               20631 non-null  float64
10  sensor_6               20631 non-null  float64
11  sensor_7               20631 non-null  float64
12  sensor_8               20631 non-null  float64
13  sensor_9               20631 non-null  float64
14  sensor_10              20631 non-null  float64
15  sensor_11              20631 non-null  float64
16  sensor_12              20631 non-null  float64
17  sensor_13              20631 non-null  float64
18  sensor_14              20631 non-null  float64
19  sensor_15              20631 non-null  float64
20  sensor_16              20631 non-null  float64
21  sensor_17              20631 non-null  int64
22  sensor_18              20631 non-null  int64
23  sensor_19              20631 non-null  float64
24  sensor_20              20631 non-null  float64
25  sensor_21              20631 non-null  float64
dtypes: float64(22), int64(4)
memory usage: 4.1 MB
```

```
In [10]:  # check for missing values  
train_fd001.isnull().sum()
```

```
Out[10]: unit_nr      0  
         cycle      0  
         op_setting_1 0  
         op_setting_2 0  
         op_setting_3 0  
         sensor_1     0  
         sensor_2     0  
         sensor_3     0  
         sensor_4     0  
         sensor_5     0  
         sensor_6     0  
         sensor_7     0  
         sensor_8     0  
         sensor_9     0  
         sensor_10    0  
         sensor_11    0  
         sensor_12    0  
         sensor_13    0  
         sensor_14    0  
         sensor_15    0  
         sensor_16    0  
         sensor_17    0  
         sensor_18    0  
         sensor_19    0  
         sensor_20    0  
         sensor_21    0  
         dtype: int64
```

```
In [11]: # statistical summary of features  
train_fd001.describe().T
```

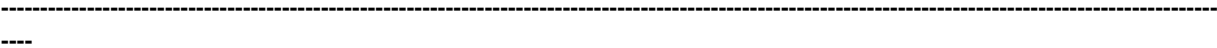
Out[11]:

	count	mean	std	min	25%	50%	75%	max
unit_nr	20631.0	51.506568	2.922763e+01	1.0000	26.0000	52.0000	77.0000	100.0000
cycle	20631.0	108.807862	6.888099e+01	1.0000	52.0000	104.0000	156.0000	362.0000
op_setting_1	20631.0	-0.000009	2.187313e-03	-0.0087	-0.0015	0.0000	0.0015	0.0087
op_setting_2	20631.0	0.000002	2.930621e-04	-0.0006	-0.0002	0.0000	0.0003	0.0006
op_setting_3	20631.0	100.000000	0.000000e+00	100.0000	100.0000	100.0000	100.0000	100.0000
sensor_1	20631.0	518.670000	0.000000e+00	518.6700	518.6700	518.6700	518.6700	518.6700
sensor_2	20631.0	642.680934	5.000533e-01	641.2100	642.3250	642.6400	643.0000	644.5300
sensor_3	20631.0	1590.523119	6.131150e+00	1571.0400	1586.2600	1590.1000	1594.3800	1616.9100
sensor_4	20631.0	1408.933782	9.000605e+00	1382.2500	1402.3600	1408.0400	1414.5550	1441.4900
sensor_5	20631.0	14.620000	1.776400e-15	14.6200	14.6200	14.6200	14.6200	14.6200
sensor_6	20631.0	21.609803	1.388985e-03	21.6000	21.6100	21.6100	21.6100	21.6100
sensor_7	20631.0	553.367711	8.850923e-01	549.8500	552.8100	553.4400	554.0100	556.0600
sensor_8	20631.0	2388.096652	7.098548e-02	2387.9000	2388.0500	2388.0900	2388.1400	2388.5600
sensor_9	20631.0	9065.242941	2.208288e+01	9021.7300	9053.1000	9060.6600	9069.4200	9244.5900
sensor_10	20631.0	1.300000	0.000000e+00	1.3000	1.3000	1.3000	1.3000	1.3000
sensor_11	20631.0	47.541168	2.670874e-01	46.8500	47.3500	47.5100	47.7000	48.5300
sensor_12	20631.0	521.413470	7.375534e-01	518.6900	520.9600	521.4800	521.9500	523.3800
sensor_13	20631.0	2388.096152	7.191892e-02	2387.8800	2388.0400	2388.0900	2388.1400	2388.5600
sensor_14	20631.0	8143.752722	1.907618e+01	8099.9400	8133.2450	8140.5400	8148.3100	8293.7200
sensor_15	20631.0	8.442146	3.750504e-02	8.3249	8.4149	8.4389	8.4656	8.5848
sensor_16	20631.0	0.030000	1.387812e-17	0.0300	0.0300	0.0300	0.0300	0.0300
sensor_17	20631.0	393.210654	1.548763e+00	388.0000	392.0000	393.0000	394.0000	400.0000
sensor_18	20631.0	2388.000000	0.000000e+00	2388.0000	2388.0000	2388.0000	2388.0000	2388.0000
sensor_19	20631.0	100.000000	0.000000e+00	100.0000	100.0000	100.0000	100.0000	100.0000
sensor_20	20631.0	38.816271	1.807464e-01	38.1400	38.7000	38.8300	38.9500	39.4300
sensor_21	20631.0	23.289705	1.082509e-01	22.8942	23.2218	23.2979	23.3668	23.6184

Initial Observations

- No missing values are present in the dataset.
- Some features appear to have very low or zero variance.
- These features will be analyzed later and potentially removed.

Next, we visualize how engine cycles behave over time.



Engine Degradation Visualization

To better understand the dataset, we visualize how sensor values change over time (cycles) for a few engines.

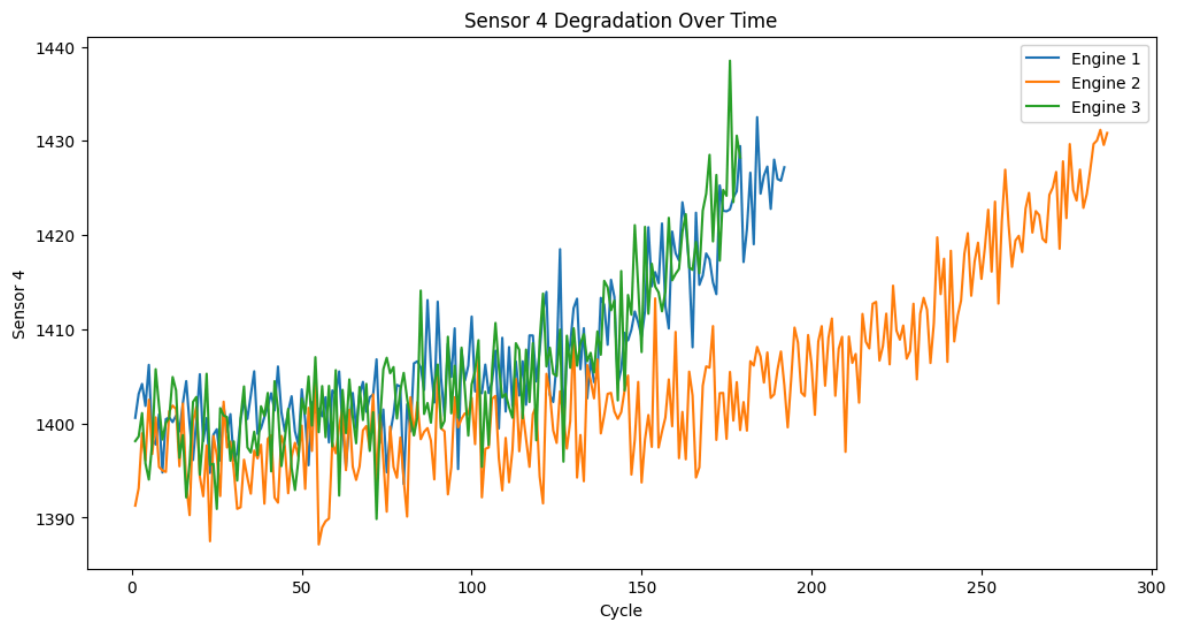
This helps us observe degradation patterns.

```
In [12]: # pick a few engines to visualize
sample_engines = train_fd001['unit_nr'].unique()[:3]

plt.figure(figsize=(12,6))

for engine in sample_engines:
    subset = train_fd001[train_fd001['unit_nr'] == engine]
    plt.plot(subset['cycle'], subset['sensor_4'], label=f'Engine {engine}')

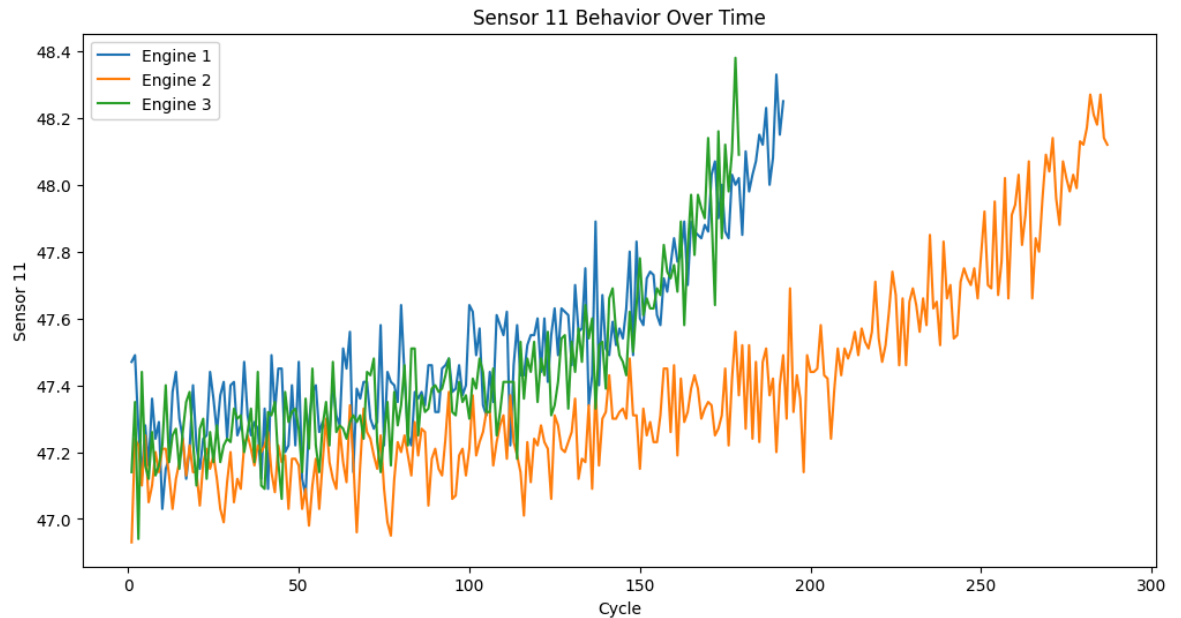
plt.xlabel("Cycle")
plt.ylabel("Sensor 4")
plt.title("Sensor 4 Degradation Over Time")
plt.legend()
plt.show()
```



```
In [13]: ▶ # another sensor example
plt.figure(figsize=(12,6))

for engine in sample_engines:
    subset = train_fd001[train_fd001['unit_nr'] == engine]
    plt.plot(subset['cycle'], subset['sensor_11'], label=f'Engine {engine}')

plt.xlabel("Cycle")
plt.ylabel("Sensor 11")
plt.title("Sensor 11 Behavior Over Time")
plt.legend()
plt.show()
```



Observations from Sensor Trends

- Sensor values change over time as the engine degrades.
- Some sensors show clear trends.
- This confirms that the dataset contains useful information for predicting RUL.

Next, we construct the target variable (RUL).

Creating the Target Variable (RUL)

For training data, the Remaining Useful Life (RUL) is computed as:

$$\text{RUL} = \text{max cycle of engine} - \text{current cycle}$$

This represents how many cycles remain before failure.

```
In [14]: # compute max cycle per engine
max_cycles = train_fd001.groupby('unit_nr')['cycle'].max().reset_index()
max_cycles.columns = ['unit_nr', 'max_cycle']

max_cycles.head()
```

Out[14]:

	unit_nr	max_cycle
0	1	192
1	2	287
2	3	179
3	4	189
4	5	269

```
In [15]: # merge max cycle back into training data
train_fd001 = train_fd001.merge(max_cycles, on='unit_nr')

train_fd001.head()
```

Out[15]:

	unit_nr	cycle	op_setting_1	op_setting_2	op_setting_3	sensor_1	sensor_2	sensor_3	sensor_4	sensor_5	..
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	..
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	..
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	..
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	..
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	..

5 rows × 27 columns



```
In [16]: # compute raw RUL
train_fd001['RUL'] = train_fd001['max_cycle'] - train_fd001['cycle']

train_fd001[['unit_nr', 'cycle', 'max_cycle', 'RUL']].head(10)
```

Out[16]:

	unit_nr	cycle	max_cycle	RUL
0	1	1	192	191
1	1	2	192	190
2	1	3	192	189
3	1	4	192	188
4	1	5	192	187
5	1	6	192	186
6	1	7	192	185
7	1	8	192	184
8	1	9	192	183
9	1	10	192	182

```
In [17]: ▶ # verify RUL at the end of an engine (should go to 0)
train_fd001[train_fd001['unit_nr'] == 1][['cycle', 'RUL']].tail(10)
```

Out[17]:

	cycle	RUL
182	183	9
183	184	8
184	185	7
185	186	6
186	187	5
187	188	4
188	189	3
189	190	2
190	191	1
191	192	0

To make the learning problem more focused, we apply RUL capping.

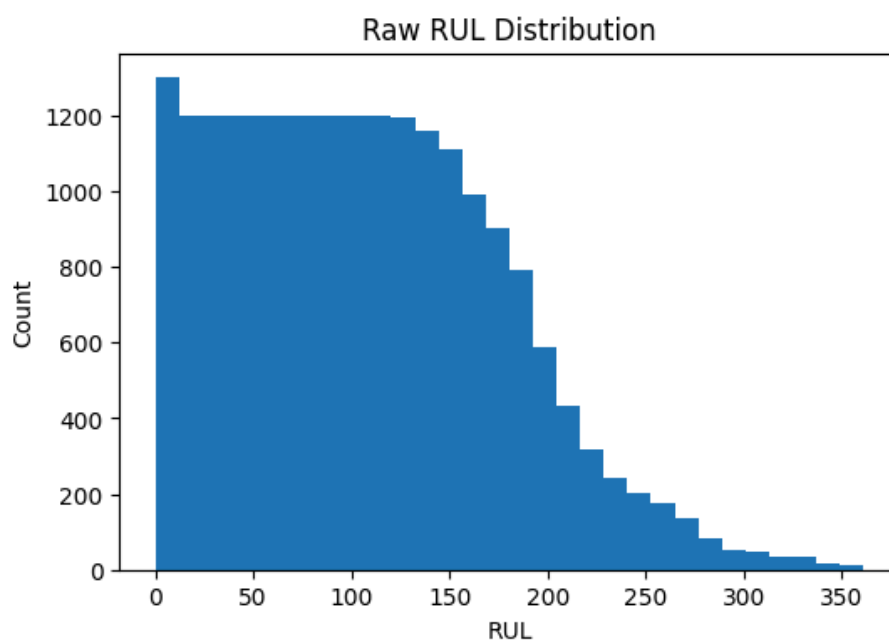
This limits large early-life RUL values and forces the model to prioritize the degradation phase, which is more important for predicting failure.

Verification of RUL

- RUL decreases as cycle increases
- Final cycle of each engine has RUL = 0
- This confirms correct target construction

Next, we analyze the distribution of RUL values.

```
In [18]: ▶ # visualize raw RUL distribution
plt.figure(figsize=(6,4))
plt.hist(train_fd001['RUL'], bins=30)
plt.title("Raw RUL Distribution")
plt.xlabel("RUL")
plt.ylabel("Count")
plt.show()
```



Observations from Raw RUL

The raw RUL distribution shows a long tail with many high values corresponding to early engine life.

Since predicting failure is more important than predicting early healthy states, we apply RUL capping to reduce the dominance of large values.

This allows the model to focus on the degradation region where prediction is more critical.

To address this, we apply RUL capping.

```
In [19]: # define cap
RUL_CAP = 125

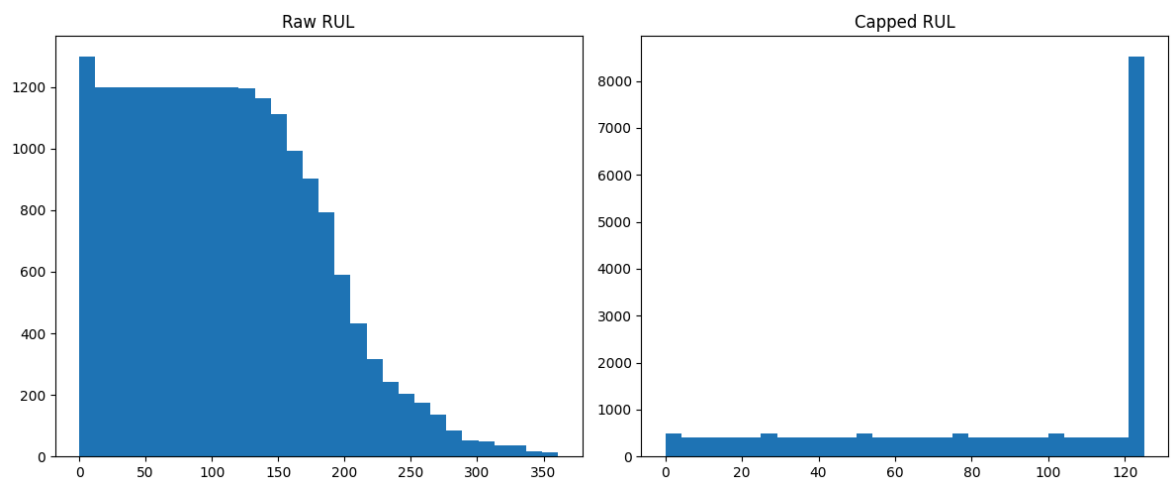
# apply capping
train_fd001['RUL_capped'] = train_fd001['RUL'].clip(upper=RUL_CAP)
```

```
In [20]: # compare distributions
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.hist(train_fd001['RUL'], bins=30)
plt.title("Raw RUL")

plt.subplot(1,2,2)
plt.hist(train_fd001['RUL_capped'], bins=30)
plt.title("Capped RUL")

plt.tight_layout()
plt.show()
```



After applying RUL capping, a spike appears at the cap value (125).

This is expected, as all high RUL values are grouped together.

This transformation simplifies the learning problem by:

- reducing the influence of early-life samples
- focusing the model on degradation behavior
- improving prediction near failure, which is more critical

Although the distribution becomes less smooth, it is more suitable for regression modeling.

```
In [21]: # replace RUL with capped version for modeling
train_fd001['RUL'] = train_fd001['RUL_capped']

# clean up
train_fd001.drop(columns=['RUL_capped', 'max_cycle'], inplace=True)

train_fd001.head()
```

Out[21]:

	unit_nr	cycle	op_setting_1	op_setting_2	op_setting_3	sensor_1	sensor_2	sensor_3	sensor_4	sensor_5	..
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	..
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	..
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	..
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	..
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	..

5 rows × 27 columns



Removing Useless Features

From the statistical summary, we observed that some features have zero variance (constant values).

These features do not contribute to prediction and should be removed.

```
In [22]: # check number of unique values per column
train_fd001.nunique().sort_values()
```

```
Out[22]: sensor_1      1
op_setting_3      1
sensor_10         1
sensor_5          1
sensor_19         1
sensor_16         1
sensor_18         1
sensor_6          2
sensor_17        13
op_setting_2      13
sensor_8         53
sensor_13        56
unit_nr         100
sensor_20       120
RUL             126
op_setting_1     158
sensor_11       159
sensor_2        310
cycle          362
sensor_12       427
sensor_7        513
sensor_15      1918
sensor_3       3012
sensor_4       4051
sensor_21      4745
sensor_14      6078
sensor_9       6403
dtype: int64
```

```
In [23]: # find columns with only one unique value
constant_cols = [col for col in train_fd001.columns if train_fd001[col].nunique() == 1]

print("Constant columns:")
constant_cols
```

Constant columns:

```
Out[23]: ['op_setting_3',
         'sensor_1',
         'sensor_5',
         'sensor_10',
         'sensor_16',
         'sensor_18',
         'sensor_19']
```

```
In [24]: # remove constant columns
train_fd001 = train_fd001.drop(columns=constant_cols)

print("New shape after removing constant features:", train_fd001.shape)
```

New shape after removing constant features: (20631, 20)

Observations

- Several features were constant across all observations.
- These features do not provide useful information for prediction.
- Removing them reduces noise and improves model efficiency.

Next, we analyze which features are most relevant to RUL.

Feature Relevance Analysis

To understand which features are most useful for predicting RUL, we analyze their relationship with the target.

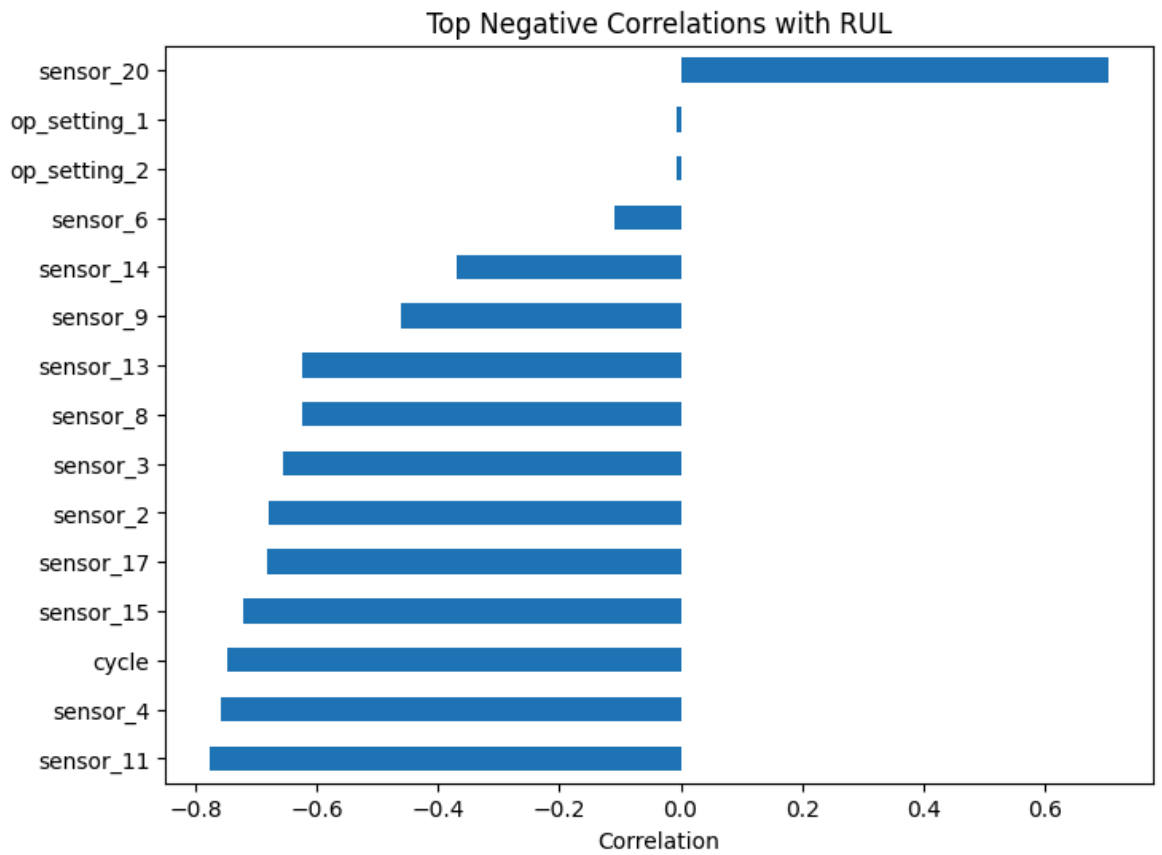
We start by computing the correlation between each feature and RUL.

```
In [25]: # compute correlation with RUL
corr_with_rul = train_fd001.drop(columns=['unit_nr']).corr()['RUL'].sort_values()

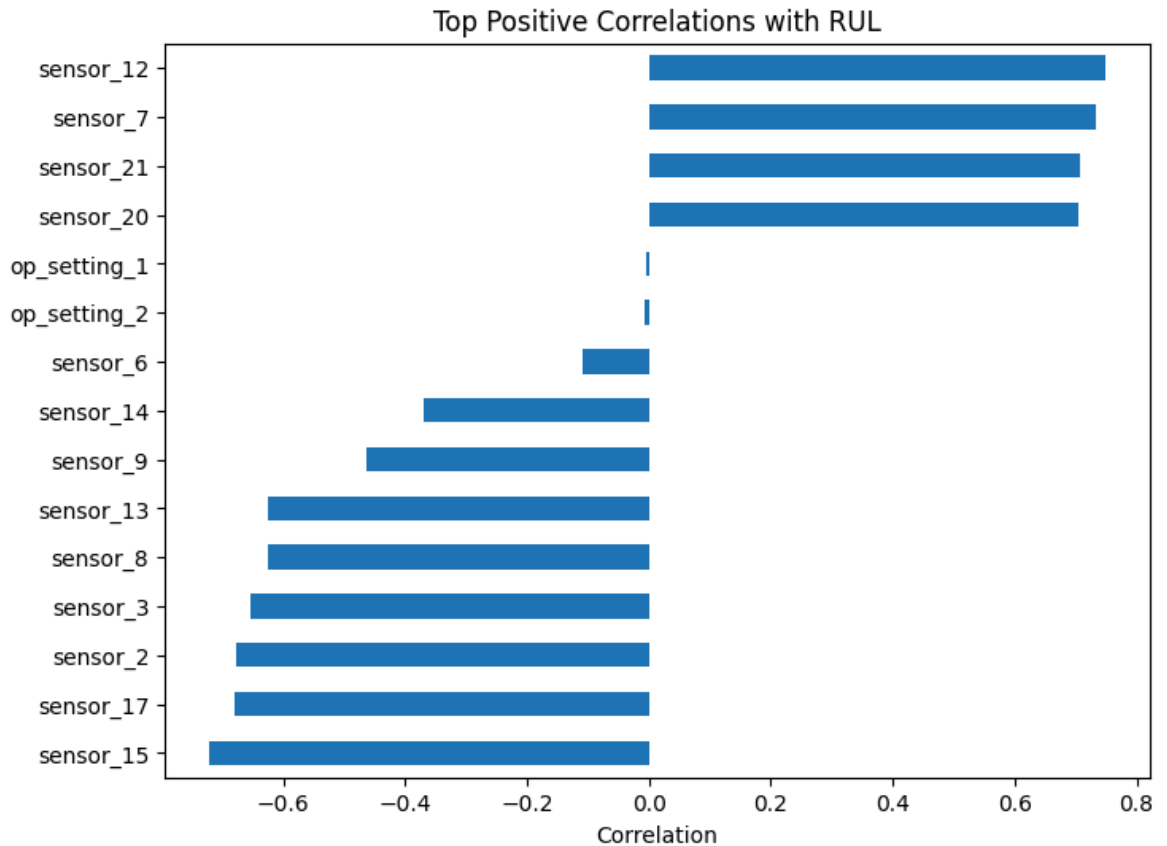
corr_with_rul.head(10), corr_with_rul.tail(10)
```

```
Out[25]: (sensor_11    -0.775230
         sensor_4     -0.757157
         cycle        -0.746939
         sensor_15    -0.720858
         sensor_17    -0.680829
         sensor_2     -0.678458
         sensor_3     -0.655030
         sensor_8     -0.624568
         sensor_13    -0.624034
         sensor_9     -0.462151
         Name: RUL, dtype: float64,
         sensor_9      -0.462151
         sensor_14     -0.369753
         sensor_6      -0.108289
         op_setting_2  -0.007091
         op_setting_1  -0.005556
         sensor_20      0.704626
         sensor_21      0.707334
         sensor_7       0.733021
         sensor_12      0.748870
         RUL           1.000000
         Name: RUL, dtype: float64)
```

```
In [26]: # plot most negatively correlated features
plt.figure(figsize=(8,6))
corr_with_rul.drop('RUL').head(15).plot(kind='barh')
plt.title("Top Negative Correlations with RUL")
plt.xlabel("Correlation")
plt.show()
```



```
In [27]: # plot most positively correlated features
plt.figure(figsize=(8,6))
corr_with_rul.drop('RUL').tail(15).plot(kind='barh')
plt.title("Top Positive Correlations with RUL")
plt.xlabel("Correlation")
plt.show()
```



Observations

From the correlation analysis:

- Several sensors show strong relationships with RUL.

Strong Negative Correlations:

- sensor_11, sensor_4, cycle, sensor_15, and sensor_17 have high negative correlation with RUL.
- This indicates that these values tend to increase as the engine approaches failure (RUL decreases).
- These features capture degradation behavior and are important predictors.

Strong Positive Correlations:

- sensor_12, sensor_7, sensor_21, and sensor_20 show strong positive correlation with RUL.
- These features tend to decrease as the engine degrades.

Weak or Irrelevant Features:

- Operational settings (op_setting_1 and op_setting_2) have near-zero correlation with RUL.
- These features are likely less useful for prediction.

Overall, the correlation analysis identifies several key sensors that are strongly related to engine degradation and should be considered for modeling.

However, correlation only captures linear relationships. Some features may still be important even if their correlation is low.

Feature Importance Using Random Forest

While correlation helps identify linear relationships between features and RUL, it does not capture nonlinear dependencies.

To better understand which features are most useful for prediction, we use a Random Forest regressor to compute feature importance.

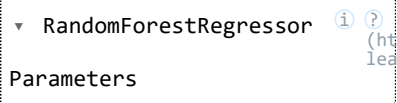
This method evaluates how much each feature contributes to reducing prediction error, providing a more reliable measure of feature relevance.

```
In [28]: from sklearn.ensemble import RandomForestRegressor
```

```
In [29]: # prepare data for feature importance
X_temp = train_fd001.drop(columns=['unit_nr', 'RUL'])
y_temp = train_fd001['RUL']
```

```
In [30]: # train Random Forest
rf_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_temp, y_temp)
```

Out[30]:  <https://scikit-learn.org/1.7/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

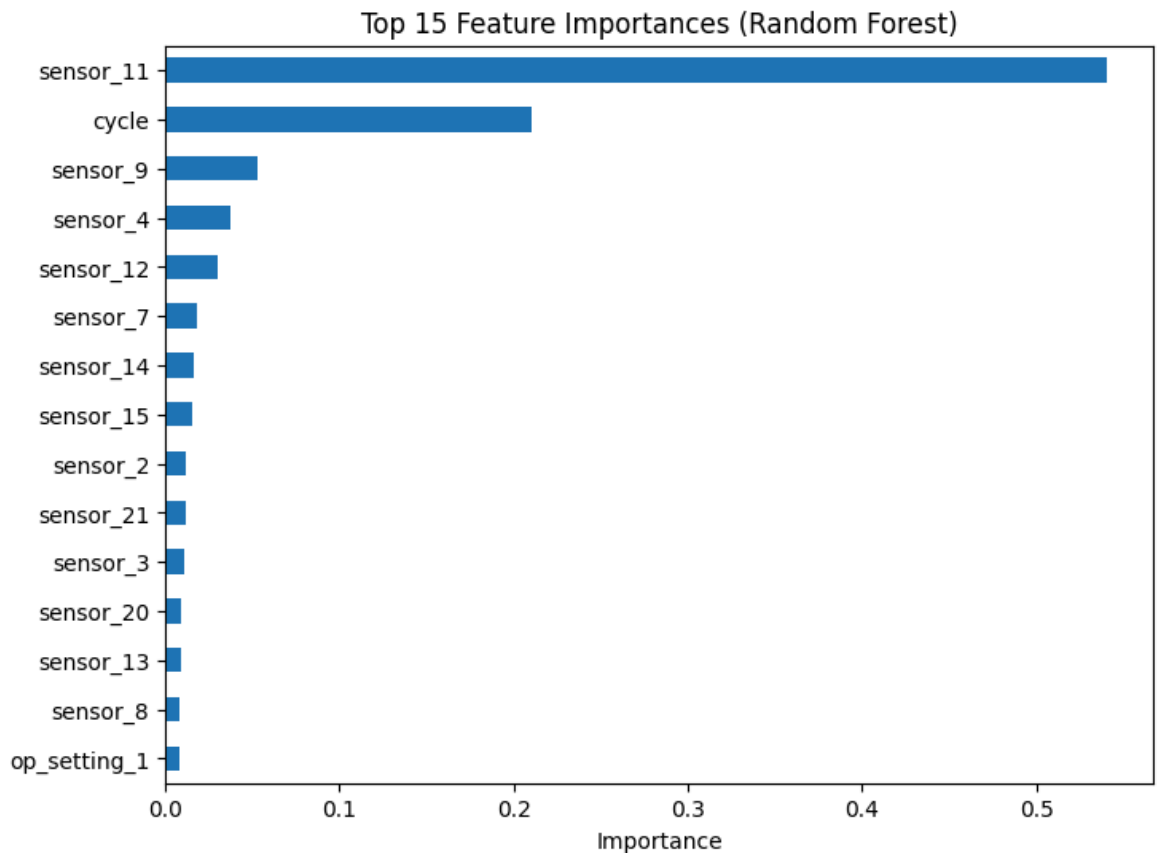
Parameters

```
In [31]: # compute feature importance
feature_importance = pd.Series(rf_model.feature_importances_, index=X_temp.columns).sort_val

feature_importance.head(15)
```

```
Out[31]: sensor_11      0.540259
         cycle         0.210019
         sensor_9      0.053047
         sensor_4      0.037506
         sensor_12     0.030285
         sensor_7      0.018409
         sensor_14     0.016360
         sensor_15     0.015646
         sensor_2      0.011807
         sensor_21     0.011786
         sensor_3      0.011130
         sensor_20     0.009234
         sensor_13     0.008686
         sensor_8      0.008509
         op_setting_1  0.008453
dtype: float64
```

```
In [32]: ▶ # plot top features
plt.figure(figsize=(8,6))
feature_importance.head(15).sort_values().plot(kind='barh')
plt.title("Top 15 Feature Importances (Random Forest)")
plt.xlabel("Importance")
plt.show()
```



Observations

From the Random Forest feature importance analysis:

- sensor_11 is by far the most important feature, contributing more than half of the total importance.
- cycle is the second most important feature, indicating that the progression of time itself is a strong indicator of degradation.
- Other important features include sensor_9, sensor_4, sensor_12, and sensor_7, though their contributions are significantly smaller.

Key Insights:

- The model relies heavily on a small subset of features, especially sensor_11.
- Several sensors that showed strong correlation with RUL (such as sensor_4 and sensor_12) are also important here, confirming their relevance.
- Some features with weaker correlation still appear in the importance ranking, indicating that they contribute through nonlinear relationships.
- Feature importance is highly concentrated, meaning that only a few features dominate prediction.
- This suggests that reducing the feature set to the most important sensors may improve model efficiency without significantly affecting performance.

Feature Selection

Based on the feature importance results, we select the most relevant features for modeling.

This helps reduce noise, simplify the model, and improve efficiency.

```
In [33]: ▶ # select top N features
top_n = 10

selected_features = feature_importance.head(top_n).index.tolist()
selected_features
```

```
Out[33]: ['sensor_11',
          'cycle',
          'sensor_9',
          'sensor_4',
          'sensor_12',
          'sensor_7',
          'sensor_14',
          'sensor_15',
          'sensor_2',
          'sensor_21']
```

```
In [34]: ▶ # create new dataset with selected features
X_selected = train_fd001[selected_features]
y_selected = train_fd001['RUL']

X_selected.head()
```

```
Out[34]:
```

	sensor_11	cycle	sensor_9	sensor_4	sensor_12	sensor_7	sensor_14	sensor_15	sensor_2	sensor_21
0	47.47	1	9046.19	1400.60	521.66	554.36	8138.62	8.4195	641.82	23.4190
1	47.49	2	9044.07	1403.14	522.28	553.75	8131.49	8.4318	642.15	23.4236
2	47.27	3	9052.94	1404.20	522.42	554.26	8133.23	8.4178	642.35	23.3442
3	47.13	4	9049.48	1401.87	522.86	554.45	8133.83	8.3682	642.35	23.3739
4	47.28	5	9055.15	1406.22	522.19	554.00	8133.80	8.4294	642.37	23.4044

```
In [35]: ▶ print("Selected feature shape:", X_selected.shape)

Selected feature shape: (20631, 10)
```

Observations

- We selected the top features based on importance ranking.
- These features capture the most relevant information for predicting RUL.
- Using fewer features reduces model complexity while maintaining predictive power.

Next, we prepare the train/validation split for model evaluation.

Train / Validation Split

We perform the split at the engine level rather than randomly splitting rows.

This ensures that all cycles from a given engine belong entirely to either the training or validation set, preventing data leakage and providing a more realistic evaluation.

```
In [36]: ▶ from sklearn.model_selection import train_test_split

# get unique engine IDs
engine_ids = train_fd001['unit_nr'].unique()

# split engines (not rows)
train_ids, val_ids = train_test_split(engine_ids, test_size=0.2, random_state=42)

print("Train engines:", len(train_ids))
print("Validation engines:", len(val_ids))
```

Train engines: 80
Validation engines: 20

```
In [37]: ▶ # create train and validation datasets
train_df = train_fd001[train_fd001['unit_nr'].isin(train_ids)]
val_df = train_fd001[train_fd001['unit_nr'].isin(val_ids)]

print("Train shape:", train_df.shape)
print("Validation shape:", val_df.shape)
```

Train shape: (16561, 20)
Validation shape: (4070, 20)

```
In [38]: ▶ # prepare features and target
X_train = train_df[selected_features]
y_train = train_df['RUL']

X_val = val_df[selected_features]
y_val = val_df['RUL']

print("X_train:", X_train.shape)
print("X_val:", X_val.shape)
```

X_train: (16561, 10)
X_val: (4070, 10)

```
In [39]: ▶ # verify no overlap
overlap = set(train_df['unit_nr']).intersection(set(val_df['unit_nr']))
print("Overlap in engine IDs:", overlap)
```

Overlap in engine IDs: set()

Feature Scaling

Some models (such as Linear Regression, KNN, and SVR) are sensitive to feature scaling.

We standardize the features using StandardScaler for these models.

```
In [40]: ▶ from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)

print("Scaled shapes:", X_train_scaled.shape, X_val_scaled.shape)
```

Scaled shapes: (16561, 10) (4070, 10)

Model Setup

We now define a set of regression models to compare:

- Linear Regression
- Ridge Regression
- KNN
- SVR
- Decision Tree
- Random Forest
- XGBoost

```
In [41]: from sklearn.linear_model import LinearRegression, Ridge
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
```

```
In [42]: # initialize models
models = {
    "Linear": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "KNN": KNeighborsRegressor(n_neighbors=5),
    "SVR": SVR(kernel='rbf', C=10, epsilon=0.1),
    "Decision Tree": DecisionTreeRegressor(max_depth=10, random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=300, max_depth=20, min_samples_split=
                                         min_samples_leaf=2, random_state=42, n_jobs=-1),
    "XGBoost": XGBRegressor(n_estimators=300, max_depth=6, learning_rate=0.05, subsample=0.9,
                           objective='reg:squarederror', random_state=42
    )
}
```

Model Training and Evaluation

We train each model on the training set and evaluate it on the validation set using:

- MAE
- RMSE
- R² score

```
In [43]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

results = []
```

```

In [44]: ▶ # train and evaluate all models
for name, model in models.items():

    # models that require scaling
    if name in ["Linear", "Ridge", "KNN", "SVR"]:
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_val_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_val)

    mae = mean_absolute_error(y_val, y_pred)
    rmse = mean_squared_error(y_val, y_pred) ** 0.5
    r2 = r2_score(y_val, y_pred)

    results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2
    })

```

```

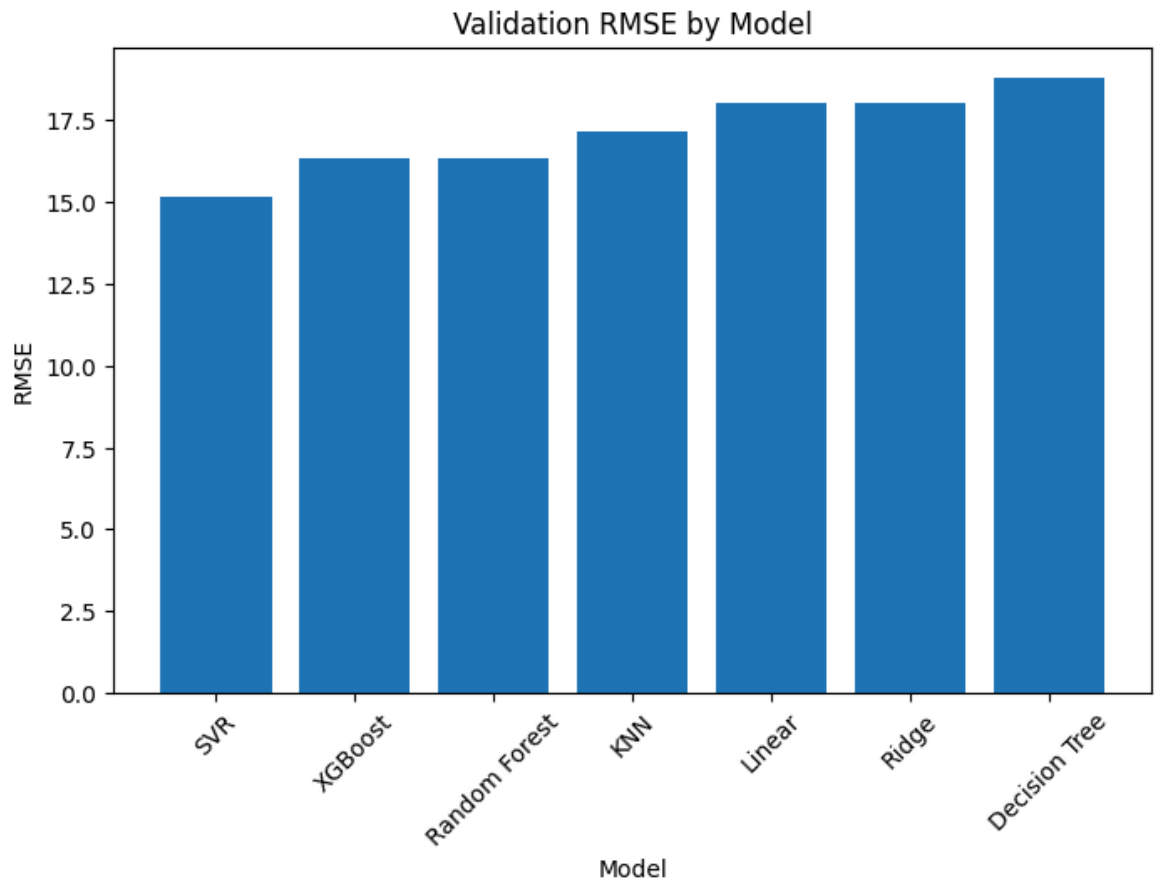
In [45]: ▶ results_df = pd.DataFrame(results).sort_values(by='R2', ascending=False)
results_df

```

Out[45]:

	Model	MAE	RMSE	R2
3	SVR	9.909042	15.179382	0.867593
6	XGBoost	10.987623	16.326437	0.846826
5	Random Forest	11.042743	16.334747	0.846670
2	KNN	11.265946	17.158024	0.830825
0	Linear	14.597545	18.024448	0.813308
1	Ridge	14.597662	18.024585	0.813305
4	Decision Tree	12.517163	18.793393	0.797039

```
In [46]: ▶ plt.figure(figsize=(8,5))
plt.bar(results_df['Model'], results_df['RMSE'])
plt.title("Validation RMSE by Model")
plt.xlabel("Model")
plt.ylabel("RMSE")
plt.xticks(rotation=45)
plt.show()
```



Observations

- SVR achieved the best performance on the validation set.
- Random Forest and XGBoost also performed strongly, with similar results.
- However, these results are based on a single train-validation split.

To ensure that the model performance is not due to randomness or a favorable split, we perform cross-validation on the strongest models.

Cross-Validation

To ensure that the model performance is reliable and not dependent on a single train-validation split, we perform cross-validation.

We evaluate the strongest models across multiple splits of engine IDs.

```
In [47]: from sklearn.model_selection import KFold
```

```
# prepare engine IDs
engine_ids = train_fd001['unit_nr'].unique()

kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
In [48]: # define models to test (top ones only)
```

```
cv_models = {
    "SVR": SVR(kernel='rbf', C=10, epsilon=0.1),
    "Random Forest": RandomForestRegressor(n_estimators=300, max_depth=20, min_samples_split=
        random_state=42, n_jobs=-1
    ),
    "XGBoost": XGBRegressor(n_estimators=300, max_depth=6, learning_rate=0.05, subsample=0.9,
        objective='reg:squarederror',
        random_state=42
    )
}
```

```
In [49]: cv_results = []
```

```
for model_name, model in cv_models.items():

    r2_scores = []
    rmse_scores = []

    for train_idx, val_idx in kf.split(engine_ids):

        train_engines = engine_ids[train_idx]
        val_engines = engine_ids[val_idx]

        train_fold = train_fd001[train_fold001['unit_nr'].isin(train_engines)]
        val_fold = train_fd001[train_fold001['unit_nr'].isin(val_engines)]

        X_train_fold = train_fold[selected_features]
        y_train_fold = train_fold['RUL']

        X_val_fold = val_fold[selected_features]
        y_val_fold = val_fold['RUL']

        # scaling for SVR only
        if model_name == "SVR":
            scaler = StandardScaler()
            X_train_fold = scaler.fit_transform(X_train_fold)
            X_val_fold = scaler.transform(X_val_fold)

        model.fit(X_train_fold, y_train_fold)
        preds = model.predict(X_val_fold)

        rmse = mean_squared_error(y_val_fold, preds) ** 0.5
        r2 = r2_score(y_val_fold, preds)

        r2_scores.append(r2)
        rmse_scores.append(rmse)

    cv_results.append({
        "Model": model_name,
        "Avg R2": np.mean(r2_scores),
        "Std R2": np.std(r2_scores),
        "Avg RMSE": np.mean(rmse_scores),
        "Std RMSE": np.std(rmse_scores)
    })
```

```
In [50]: ▶ cv_results_df = pd.DataFrame(cv_results).sort_values(by="Avg R2", ascending=False)
cv_results_df
```

Out[50]:

	Model	Avg R2	Std R2	Avg RMSE	Std RMSE
0	SVR	0.829686	0.025473	17.140651	1.281060
1	Random Forest	0.828787	0.021363	17.202578	1.009389
2	XGBoost	0.827184	0.018607	17.290681	0.887315

Cross-Validation Observations

From the cross-validation results:

- SVR achieved the highest average R^2 score.
- However, the difference between SVR, Random Forest, and XGBoost is very small.

Stability:

- Random Forest and XGBoost show lower standard deviation in RMSE compared to SVR.
- This indicates that tree-based models are slightly more stable across different splits.

Key Insight:

- While SVR performed best on the initial validation split, cross-validation shows that all three models perform similarly.
- This confirms that SVR was not purely lucky, but its advantage is not very significant.

Conclusion:

Although SVR achieved the highest validation performance, Random Forest was selected as the final model due to its more consistent performance across cross-validation folds and comparable accuracy.

Hyperparameter Optimization with Optuna

In this section, we use Optuna to search for better hyperparameters for the Random Forest model.

Unlike manual tuning or RandomizedSearchCV, Optuna performs a more efficient search by exploring promising combinations of parameters based on previous trials.

The objective is to maximize the validation R^2 score on the FD001 validation set.

```
In [51]: ▶ import optuna
```

```
In [45]: ▶ def objective(trial):  
  
    n_estimators = trial.suggest_int("n_estimators", 100, 600, step=100)  
    max_depth = trial.suggest_int("max_depth", 5, 30)  
    min_samples_split = trial.suggest_int("min_samples_split", 2, 10)  
    min_samples_leaf = trial.suggest_int("min_samples_leaf", 1, 5)  
    max_features = trial.suggest_categorical("max_features", ["sqrt", "log2", None])  
  
    rf_model = RandomForestRegressor(  
        n_estimators=n_estimators,  
        max_depth=max_depth,  
        min_samples_split=min_samples_split,  
        min_samples_leaf=min_samples_leaf,  
        max_features=max_features,  
        random_state=42,  
        n_jobs=-1  
    )  
  
    rf_model.fit(X_train, y_train)  
    y_val_pred = rf_model.predict(X_val)  
  
    r2 = r2_score(y_val, y_val_pred)  
    return r2
```

```
In [53]: ▶ study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)

print("Best trial:")
print("Best R2:", study.best_trial.value)
print("Best parameters:")
print(study.best_trial.params)
```

[I 2026-04-15 12:11:19,632] A new study created in memory with name: no-name-12476e8a-68ff-4d27-a893-fabc65dbb094

[I 2026-04-15 12:11:51,023] Trial 0 finished with value: 0.8473262859531776 and parameters: {'n_estimators': 600, 'max_depth': 14, 'min_samples_split': 3, 'min_samples_leaf': 5, 'max_features': None}. Best is trial 0 with value: 0.8473262859531776.

[I 2026-04-15 12:11:56,017] Trial 1 finished with value: 0.861682245398992 and parameters: {'n_estimators': 300, 'max_depth': 10, 'min_samples_split': 4, 'min_samples_leaf': 5, 'max_features': 'log2'}. Best is trial 1 with value: 0.861682245398992.

[I 2026-04-15 12:11:58,367] Trial 2 finished with value: 0.8579589232337299 and parameters: {'n_estimators': 100, 'max_depth': 30, 'min_samples_split': 4, 'min_samples_leaf': 3, 'max_features': 'log2'}. Best is trial 1 with value: 0.861682245398992.

[I 2026-04-15 12:12:09,857] Trial 3 finished with value: 0.8581618845064489 and parameters: {'n_estimators': 500, 'max_depth': 26, 'min_samples_split': 8, 'min_samples_leaf': 4, 'max_features': 'log2'}. Best is trial 1 with value: 0.861682245398992.

[I 2026-04-15 12:12:14,834] Trial 4 finished with value: 0.8574712760098947 and parameters: {'n_estimators': 200, 'max_depth': 30, 'min_samples_split': 4, 'min_samples_leaf': 2, 'max_features': 'log2'}. Best is trial 1 with value: 0.861682245398992.

[I 2026-04-15 12:12:27,816] Trial 5 finished with value: 0.8587527033254441 and parameters: {'n_estimators': 600, 'max_depth': 24, 'min_samples_split': 6, 'min_samples_leaf': 5, 'max_features': 'log2'}. Best is trial 1 with value: 0.861682245398992.

[I 2026-04-15 12:12:30,337] Trial 6 finished with value: 0.8433165546194916 and parameters: {'n_estimators': 100, 'max_depth': 5, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': None}. Best is trial 1 with value: 0.861682245398992.

[I 2026-04-15 12:12:41,274] Trial 7 finished with value: 0.8586781105325756 and parameters: {'n_estimators': 500, 'max_depth': 14, 'min_samples_split': 8, 'min_samples_leaf': 3, 'max_features': 'log2'}. Best is trial 1 with value: 0.861682245398992.

[I 2026-04-15 12:12:43,037] Trial 8 finished with value: 0.8622153499371432 and parameters: {'n_estimators': 100, 'max_depth': 9, 'min_samples_split': 7, 'min_samples_leaf': 1, 'max_features': 'log2'}. Best is trial 8 with value: 0.8622153499371432.

[I 2026-04-15 12:12:45,139] Trial 9 finished with value: 0.8541403180865774 and parameters: {'n_estimators': 200, 'max_depth': 5, 'min_samples_split': 6, 'min_samples_leaf': 5, 'max_features': 'sqrt'}. Best is trial 8 with value: 0.8622153499371432.

[I 2026-04-15 12:12:52,282] Trial 10 finished with value: 0.8582826920811887 and parameters: {'n_estimators': 300, 'max_depth': 19, 'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt'}. Best is trial 8 with value: 0.8622153499371432.

[I 2026-04-15 12:12:57,580] Trial 11 finished with value: 0.8614546919180792 and parameters: {'n_estimators': 300, 'max_depth': 10, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': 'log2'}. Best is trial 8 with value: 0.8622153499371432.

[I 2026-04-15 12:13:04,005] Trial 12 finished with value: 0.8616339254553953 and parameters: {'n_estimators': 400, 'max_depth': 10, 'min_samples_split': 7, 'min_samples_leaf': 4, 'max_features': 'log2'}. Best is trial 8 with value: 0.8622153499371432.

[I 2026-04-15 12:13:07,600] Trial 13 finished with value: 0.8619078894289154 and parameters: {'n_estimators': 200, 'max_depth': 10, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2'}. Best is trial 8 with value: 0.8622153499371432.

[I 2026-04-15 12:13:22,016] Trial 14 finished with value: 0.8477232234037521 and parameters: {'n_estimators': 200, 'max_depth': 17, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': None}. Best is trial 8 with value: 0.8622153499371432.

[I 2026-04-15 12:13:23,796] Trial 15 finished with value: 0.8624628602028692 and parameters: {'n_estimators': 100, 'max_depth': 8, 'min_samples_split': 9, 'min_samples_leaf': 1, 'max_features': 'sqrt'}. Best is trial 15 with value: 0.8624628602028692.

[I 2026-04-15 12:13:25,263] Trial 16 finished with value: 0.8621481499711132 and parameters: {'n_estimators': 100, 'max_depth': 7, 'min_samples_split': 8, 'min_samples_leaf': 1, 'max_features': 'sqrt'}. Best is trial 15 with value: 0.8624628602028692.

[I 2026-04-15 12:13:27,905] Trial 17 finished with value: 0.8592561856433638 and parameters: {'n_estimators': 100, 'max_depth': 14, 'min_samples_split': 9, 'min_samples_leaf': 1, 'max_features': 'sqrt'}. Best is trial 15 with value: 0.8624628602028692.

[I 2026-04-15 12:13:38,189] Trial 18 finished with value: 0.8579622439009748 and parameters: {'n_estimators': 400, 'max_depth': 19, 'min_samples_split': 7, 'min_samples_leaf': 1, 'max_features': 'sqrt'}. Best is trial 15 with value: 0.8624628602028692.

[I 2026-04-15 12:13:39,866] Trial 19 finished with value: 0.8624518037138713 and parameters: {'n_estimators': 100, 'max_depth': 8, 'min_samples_split': 9, 'min_samples_leaf': 3, 'max_features': 'sqrt'}. Best is trial 15 with value: 0.8624628602028692.

[I 2026-04-15 12:13:44,338] Trial 20 finished with value: 0.8597777108166426 and parameters: {'n_estimators': 200, 'max_depth': 13, 'min_samples_split': 9, 'min_samples_leaf': 4, 'max_features': 'sqrt'}. Best is trial 15 with value: 0.8624628602028692.

[I 2026-04-15 12:13:45,861] Trial 21 finished with value: 0.8616749830869086 and parameters: {'n_estimators': 100, 'max_depth': 7, 'min_samples_split': 9, 'min_samples_leaf': 3, 'max_features': 'sqrt'}. Best is trial 15 with value: 0.8624628602028692.

[I 2026-04-15 12:13:47,570] Trial 22 finished with value: 0.8627475314353453 and parameters: {'n_estimators': 100, 'max_depth': 8, 'min_samples_split': 7, 'min_samples_leaf': 2, 'max_features': 'sqrt'}. Best is trial 22 with value: 0.8627475314353453.

[I 2026-04-15 12:13:50,177] Trial 23 finished with value: 0.862177572717572 and parameters: {'n_estimators': 200, 'max_depth': 7, 'min_samples_split': 9, 'min_samples_leaf': 2, 'max_features': 'sqrt'}. Best is trial 22 with value: 0.8627475314353453.

```

x_features': 'sqrt'}). Best is trial 22 with value: 0.8627475314353453.
[I 2026-04-15 12:13:52,008] Trial 24 finished with value: 0.859584592781505 and parameters:
{'n_estimators': 100, 'max_depth': 12, 'min_samples_split': 8, 'min_samples_leaf': 2, 'max_
features': 'sqrt'}). Best is trial 22 with value: 0.8627475314353453.
[I 2026-04-15 12:13:54,583] Trial 25 finished with value: 0.8574575785475971 and parameter
s: {'n_estimators': 100, 'max_depth': 16, 'min_samples_split': 7, 'min_samples_leaf': 3, 'm
ax_features': 'sqrt'}). Best is trial 22 with value: 0.8627475314353453.
[I 2026-04-15 12:13:57,710] Trial 26 finished with value: 0.8625071379439265 and parameter
s: {'n_estimators': 200, 'max_depth': 8, 'min_samples_split': 9, 'min_samples_leaf': 2, 'ma
x_features': 'sqrt'}). Best is trial 22 with value: 0.8627475314353453.
[I 2026-04-15 12:14:01,986] Trial 27 finished with value: 0.8595904759304827 and parameter
s: {'n_estimators': 200, 'max_depth': 12, 'min_samples_split': 5, 'min_samples_leaf': 2, 'm
ax_features': 'sqrt'}). Best is trial 22 with value: 0.8627475314353453.
[I 2026-04-15 12:14:05,539] Trial 28 finished with value: 0.8596697734071614 and parameter
s: {'n_estimators': 300, 'max_depth': 6, 'min_samples_split': 6, 'min_samples_leaf': 1, 'ma
x_features': 'sqrt'}). Best is trial 22 with value: 0.8627475314353453.
[I 2026-04-15 12:14:20,045] Trial 29 finished with value: 0.847651318409377 and parameters:
{'n_estimators': 200, 'max_depth': 22, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_
features': None}. Best is trial 22 with value: 0.8627475314353453.

Best trial:
Best R2: 0.8627475314353453
Best parameters:
{'n_estimators': 100, 'max_depth': 8, 'min_samples_split': 7, 'min_samples_leaf': 2, 'max_f
eatures': 'sqrt'}

```

Final Model Training and Evaluation

Based on previous experiments, Random Forest was selected as the final model due to its strong performance and stability across validation and cross-validation.

The model is trained using the best hyperparameters obtained from Optuna and evaluated on the official FD001 test set.

```

In [54]: ▶ best_params = study.best_trial.params

final_rf_model = RandomForestRegressor(
    **best_params,
    random_state=42,
    n_jobs=-1
)

# train on full FD001 training data
final_rf_model.fit(train_fd001[selected_features], train_fd001['RUL'])

```

Out[54]:

▼ RandomForestRegressor  

Parameters

(<https://scikit-learn.org/1.7/modules/generated/sklearn.ensemble.RandomForestRegressor.html>)

Evaluation on Official FD001 Test Set

To evaluate the model, we use the official test set.

For each engine, only the final cycle is used for prediction, and the predicted RUL is compared with the true RUL values.

```
In [55]: ▶ # prepare test set
test_last = test_fd001.groupby('unit_nr').tail(1).sort_values('unit_nr')

X_test_final = test_last[selected_features]
y_test_final = rul_fd001['RUL'].clip(upper=125)

# predictions
y_pred_final = final_rf_model.predict(X_test_final)

# metrics
mae = mean_absolute_error(y_test_final, y_pred_final)
rmse = mean_squared_error(y_test_final, y_pred_final) ** 0.5
r2 = r2_score(y_test_final, y_pred_final)

print("Final Random Forest Results:")
print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)
```

```
Final Random Forest Results:
MAE: 12.733694824589666
RMSE: 17.252040216315848
R2: 0.8146591224834058
```

Visualization of Model Performance

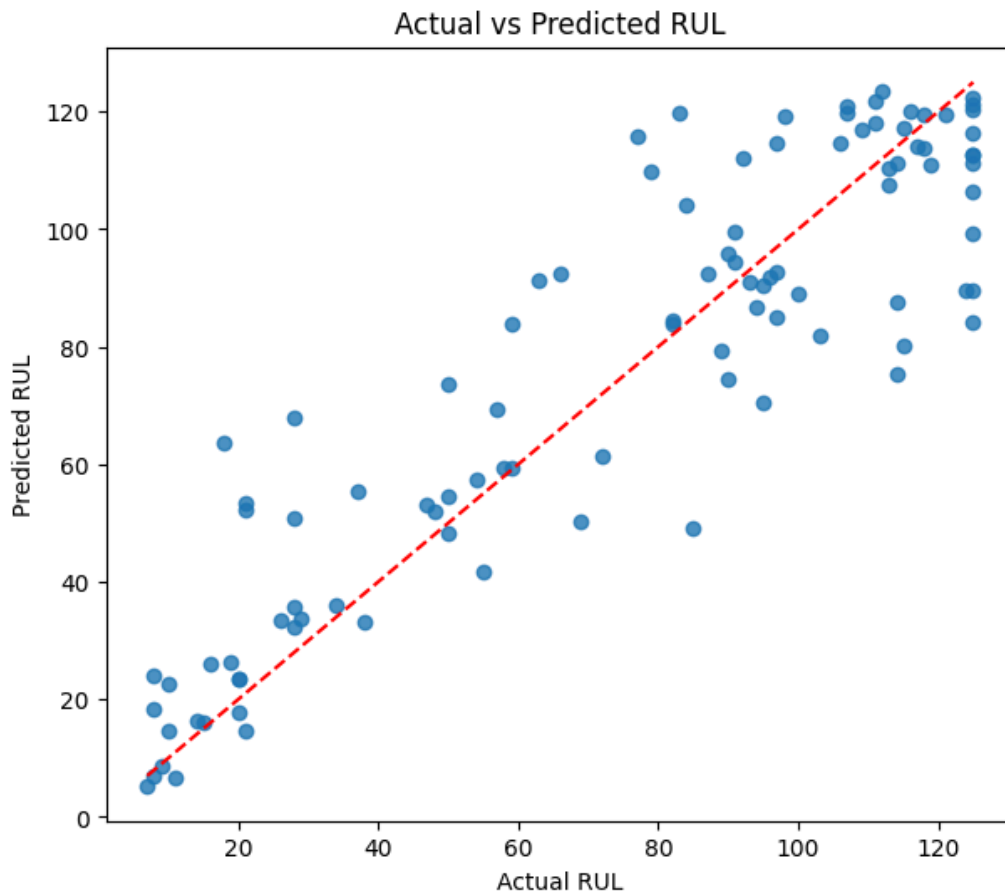
To better understand model behavior, we analyze predictions using multiple visualizations:

- Actual vs Predicted values
- Residual plot
- Residual distribution
- Sorted comparison

```
In [56]: ▶ residuals = y_test_final - y_pred_final
```

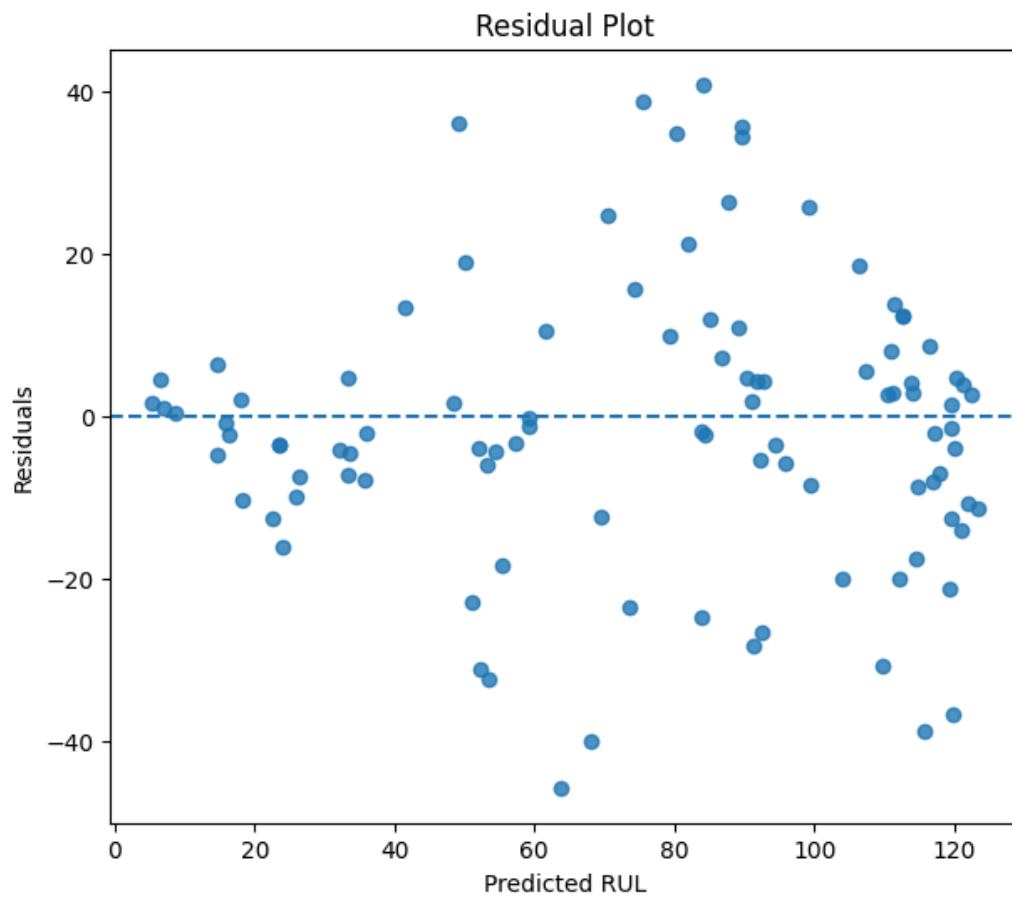
```
In [57]: ▶ plt.figure(figsize=(7,6))
plt.scatter(y_test_final, y_pred_final, alpha=0.8)
plt.plot([y_test_final.min(), y_test_final.max()],
         [y_test_final.min(), y_test_final.max()],
         'r--')

plt.xlabel("Actual RUL")
plt.ylabel("Predicted RUL")
plt.title("Actual vs Predicted RUL")
plt.show()
```



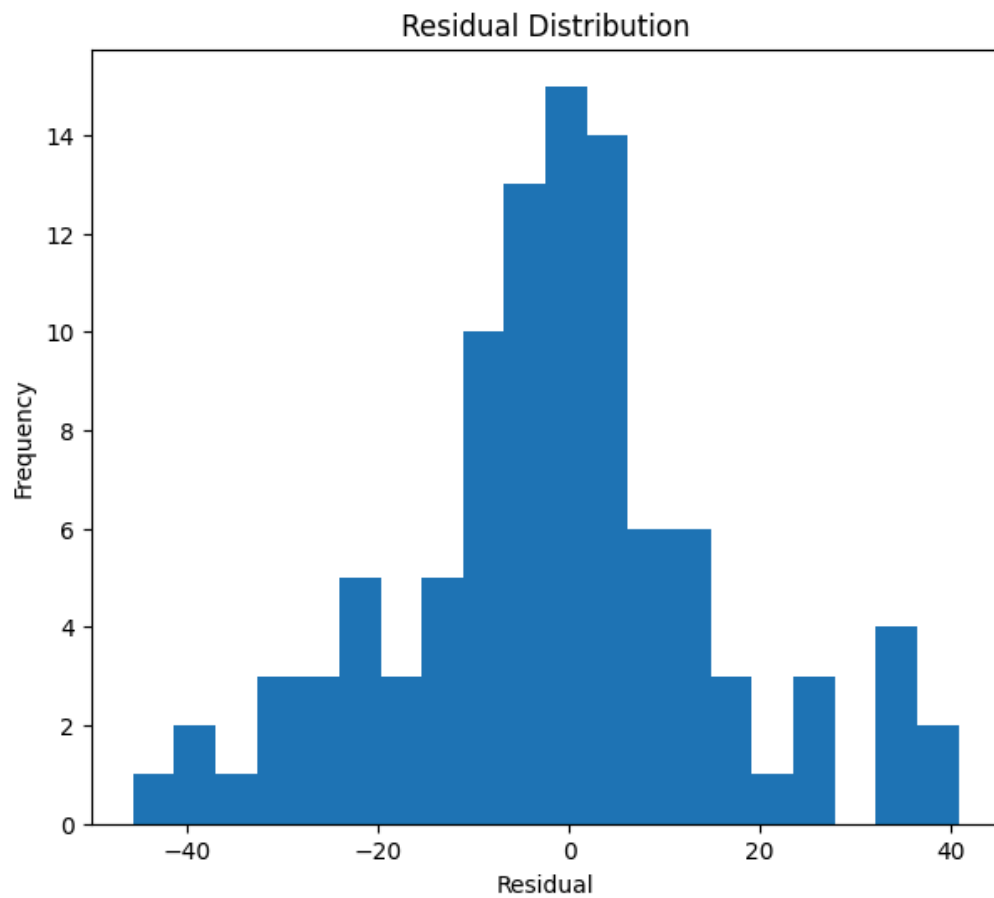
```
In [58]: ▶ plt.figure(figsize=(7,6))
plt.scatter(y_pred_final, residuals, alpha=0.8)
plt.axhline(y=0, linestyle='--')

plt.xlabel("Predicted RUL")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.show()
```



```
In [59]: ▶ plt.figure(figsize=(7,6))
plt.hist(residuals, bins=20)

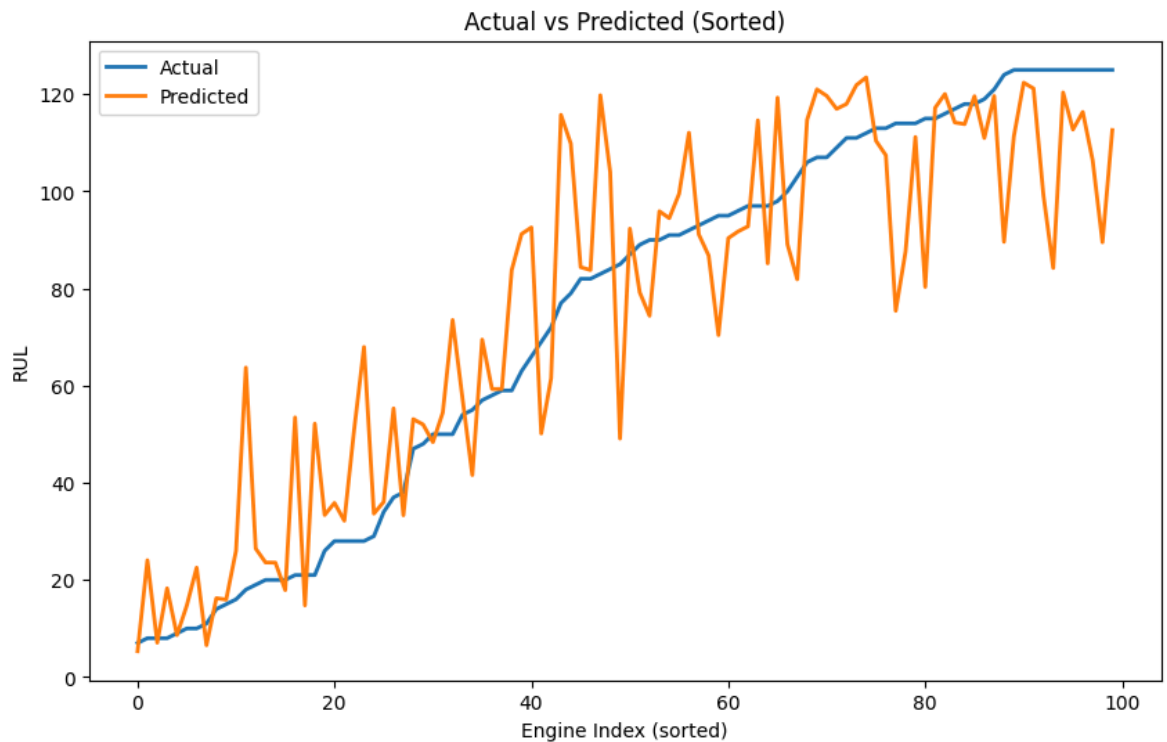
plt.xlabel("Residual")
plt.ylabel("Frequency")
plt.title("Residual Distribution")
plt.show()
```



```
In [60]: sorted_idx = np.argsort(y_test_final.values)

plt.figure(figsize=(10,6))
plt.plot(y_test_final.values[sorted_idx], label="Actual", linewidth=2)
plt.plot(y_pred_final[sorted_idx], label="Predicted", linewidth=2)

plt.xlabel("Engine Index (sorted)")
plt.ylabel("RUL")
plt.title("Actual vs Predicted (Sorted)")
plt.legend()
plt.show()
```



```
In [61]: error_df = pd.DataFrame({
    "Actual_RUL": y_test_final.values,
    "Predicted_RUL": y_pred_final,
    "Residual": residuals,
    "Absolute_Error": np.abs(residuals)
})

error_df.head()
```

Out[61]:

	Actual_RUL	Predicted_RUL	Residual	Absolute_Error
0	112	123.474511	-11.474511	11.474511
1	98	119.300887	-21.300887	21.300887
2	69	50.147189	18.852811	18.852811
3	82	84.387437	-2.387437	2.387437
4	91	94.478202	-3.478202	3.478202

Final Observations

The Optuna-tuned Random Forest model demonstrates strong performance on the FD001 test set.

Performance Metrics

- MAE $\approx$ 12.73
- RMSE $\approx$ 17.25
- $R^2 \approx$ 0.815

These results indicate that the model is able to explain a significant portion of the variance in Remaining Useful Life (RUL), while maintaining relatively low prediction error.

Analysis of Predictions

From the Actual vs Predicted plot:

- Most predictions follow the ideal diagonal trend, indicating good overall accuracy.
- The model performs particularly well in the mid-range of RUL values.

Residual Analysis

From the residual plot:

- Residuals are generally centered around zero, suggesting no strong systematic bias.
- However, the spread of residuals increases at higher RUL values, indicating reduced accuracy for engines far from failure.

From the residual distribution:

- The distribution is approximately centered around zero, but shows some spread and slight asymmetry.
- This suggests that while most predictions are reasonably accurate, some larger errors still occur.

Sorted Comparison Insights

From the sorted Actual vs Predicted plot:

- The model captures the overall degradation trend effectively.
- However, predictions show noticeable fluctuations compared to the smooth actual curve.
- This indicates that the model sometimes struggles with consistency across engines, particularly in higher RUL regions.

Error Behavior

From the error analysis:

- Larger prediction errors are observed mainly for engines with higher RUL values.
- This is expected, as early-stage degradation is harder to model and less informative.

Overall Conclusion

The Random Forest model provides a robust and reliable approach for RUL prediction.

While some variability remains, particularly for high RUL values, the model successfully captures the general degradation behavior and achieves strong predictive performance on unseen test data.

Advanced Deep Learning Extension: 1D CNN for RUL Prediction

In addition to traditional machine learning models, we explore a 1D Convolutional Neural Network (CNN) for Remaining Useful Life prediction.

Unlike tabular models that treat each row independently, the CNN uses short time windows of sensor readings, allowing it to learn local temporal degradation patterns across cycles.

This makes CNN a suitable architecture for time-series based prognostics.

```
In [23]: ▶ import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
```

```
In [73]: > cnn_scaler = StandardScaler()

X_train_full_cnn = train_fd001[selected_features]
X_test_full_cnn = test_fd001[selected_features]

cnn_scaler.fit(X_train_full_cnn)

train_fd001_cnn = train_fd001.copy()
test_fd001_cnn = test_fd001.copy()

train_fd001_cnn[selected_features] = cnn_scaler.transform(train_fd001[selected_features])
test_fd001_cnn[selected_features] = cnn_scaler.transform(test_fd001[selected_features])
```

```
In [12]: > def create_train_sequences(df, feature_cols, seq_len):
    X, y = [], []

    for engine_id in df['unit_nr'].unique():
        engine_data = df[df['unit_nr'] == engine_id].sort_values('cycle')

        features = engine_data[feature_cols].values
        targets = engine_data['RUL'].values

        for i in range(len(engine_data) - seq_len + 1):
            X.append(features[i:i+seq_len])
            y.append(targets[i+seq_len-1])

    return np.array(X, dtype=np.float32), np.array(y, dtype=np.float32)
```

```
In [13]: > def create_test_sequences(df, feature_cols, seq_len):
    X = []

    for engine_id in df['unit_nr'].unique():
        engine_data = df[df['unit_nr'] == engine_id].sort_values('cycle')
        features = engine_data[feature_cols].values

        if len(features) >= seq_len:
            X.append(features[-seq_len:])
        else:
            pad = np.repeat(features[0:1], seq_len - len(features), axis=0)
            padded_seq = np.vstack([pad, features])
            X.append(padded_seq)

    return np.array(X, dtype=np.float32)
```

Instead of treating each row independently, we convert the data into sequences using sliding windows.

Each input sample consists of a sequence of consecutive time steps (e.g., 30 cycles), allowing the CNN to learn temporal degradation patterns across time rather than relying only on static feature values.

```
In [76]: > SEQ_LEN = 30

X_train_seq, y_train_seq = create_train_sequences(train_fd001_cnn, selected_features, SEQ_LEN)
X_test_seq = create_test_sequences(test_fd001_cnn, selected_features, SEQ_LEN)
y_test_seq = rul_fd001['RUL'].clip(upper=125).values.astype(np.float32)

print("Train sequences shape:", X_train_seq.shape)
print("Train targets shape:", y_train_seq.shape)
print("Test sequences shape:", X_test_seq.shape)
print("Test targets shape:", y_test_seq.shape)

Train sequences shape: (17731, 30, 10)
Train targets shape: (17731,)
Test sequences shape: (100, 30, 10)
Test targets shape: (100,)
```

```
In [77]: from sklearn.model_selection import train_test_split

train_engine_ids, val_engine_ids = train_test_split(
    train_fd001_cnn['unit_nr'].unique(),
    test_size=0.2,
    random_state=42
)

train_seq_df = train_fd001_cnn[train_fd001_cnn['unit_nr'].isin(train_engine_ids)]
val_seq_df = train_fd001_cnn[train_fd001_cnn['unit_nr'].isin(val_engine_ids)]

X_train_seq, y_train_seq = create_train_sequences(train_seq_df, selected_features, SEQ_LEN)
X_val_seq, y_val_seq = create_train_sequences(val_seq_df, selected_features, SEQ_LEN)

print("CNN train sequence shape:", X_train_seq.shape)
print("CNN val sequence shape:", X_val_seq.shape)

CNN train sequence shape: (14241, 30, 10)
CNN val sequence shape: (3490, 30, 10)
```

```
In [24]: class RULDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

```
In [79]: batch_size = 64

train_dataset = RULDataset(X_train_seq, y_train_seq)
val_dataset = RULDataset(X_val_seq, y_val_seq)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
```

```
In [44]: class CNN1DRegressor(nn.Module):
    def __init__(self, num_features):
        super(CNN1DRegressor, self).__init__()

        self.conv1 = nn.Conv1d(in_channels=num_features, out_channels=32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv1d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool1d(kernel_size=2)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)

        self.fc1 = nn.Linear(64 * (SEQ_LEN // 2), 64)
        self.fc2 = nn.Linear(64, 1)

    def forward(self, x):
        x = x.permute(0, 2, 1) # (batch, features, seq_len)
        x = self.relu(self.conv1(x))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.reshape(x.size(0), -1)
        x = self.dropout(self.relu(self.fc1(x)))
        x = self.fc2(x)
        return x.squeeze(1)
```

```
In [91]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

cnn_model = CNN1DRegressor(num_features=len(selected_features)).to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(cnn_model.parameters(), lr=0.0005)
```

```
In [92]: ▶ num_epochs = 40
patience = 8

best_val_loss = float('inf')
patience_counter = 0
best_model_state = None

train_losses = []
val_losses = []

for epoch in range(num_epochs):
    cnn_model.train()
    running_train_loss = 0.0

    for X_batch, y_batch in train_loader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()
        outputs = cnn_model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()

        running_train_loss += loss.item()

    avg_train_loss = running_train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    cnn_model.eval()
    running_val_loss = 0.0

    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            outputs = cnn_model(X_batch)
            loss = criterion(outputs, y_batch)
            running_val_loss += loss.item()

    avg_val_loss = running_val_loss / len(val_loader)
    val_losses.append(avg_val_loss)

    print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_loss:.4f}, Val Loss: {avg_val_loss:.4f}")

    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        patience_counter = 0
        best_model_state = cnn_model.state_dict()
    else:
        patience_counter += 1

    if patience_counter >= patience:
        print("Early stopping triggered.")
        break

# Load best model
cnn_model.load_state_dict(best_model_state)
```

```
Epoch [1/40], Train Loss: 1971.4502, Val Loss: 418.2273
Epoch [2/40], Train Loss: 497.1696, Val Loss: 355.2450
Epoch [3/40], Train Loss: 443.3503, Val Loss: 324.0461
Epoch [4/40], Train Loss: 421.4461, Val Loss: 306.8463
Epoch [5/40], Train Loss: 405.7268, Val Loss: 299.1503
Epoch [6/40], Train Loss: 379.8748, Val Loss: 288.2345
Epoch [7/40], Train Loss: 349.4123, Val Loss: 254.8710
Epoch [8/40], Train Loss: 308.3214, Val Loss: 216.4116
Epoch [9/40], Train Loss: 275.6424, Val Loss: 192.1703
Epoch [10/40], Train Loss: 261.2784, Val Loss: 193.0635
Epoch [11/40], Train Loss: 260.1099, Val Loss: 181.1774
Epoch [12/40], Train Loss: 249.5195, Val Loss: 175.0280
Epoch [13/40], Train Loss: 250.8253, Val Loss: 180.6881
Epoch [14/40], Train Loss: 245.0040, Val Loss: 183.1638
Epoch [15/40], Train Loss: 240.9117, Val Loss: 172.4726
Epoch [16/40], Train Loss: 237.8837, Val Loss: 174.0782
Epoch [17/40], Train Loss: 231.0716, Val Loss: 167.5153
Epoch [18/40], Train Loss: 235.3122, Val Loss: 170.7347
Epoch [19/40], Train Loss: 226.8749, Val Loss: 181.2149
Epoch [20/40], Train Loss: 224.5727, Val Loss: 173.6305
Epoch [21/40], Train Loss: 221.1668, Val Loss: 169.7452
Epoch [22/40], Train Loss: 214.6190, Val Loss: 174.2958
Epoch [23/40], Train Loss: 213.7937, Val Loss: 182.9036
Epoch [24/40], Train Loss: 212.7658, Val Loss: 192.6322
Epoch [25/40], Train Loss: 208.9341, Val Loss: 178.2711
Early stopping triggered.
```

Out[92]: <All keys matched successfully>

```
In [93]: ▶ cnn_model.eval()

val_preds = []
val_true = []

with torch.no_grad():
    for X_batch, y_batch in val_loader:
        X_batch = X_batch.to(device)
        outputs = cnn_model(X_batch).cpu().numpy()

        val_preds.extend(outputs)
        val_true.extend(y_batch.numpy())

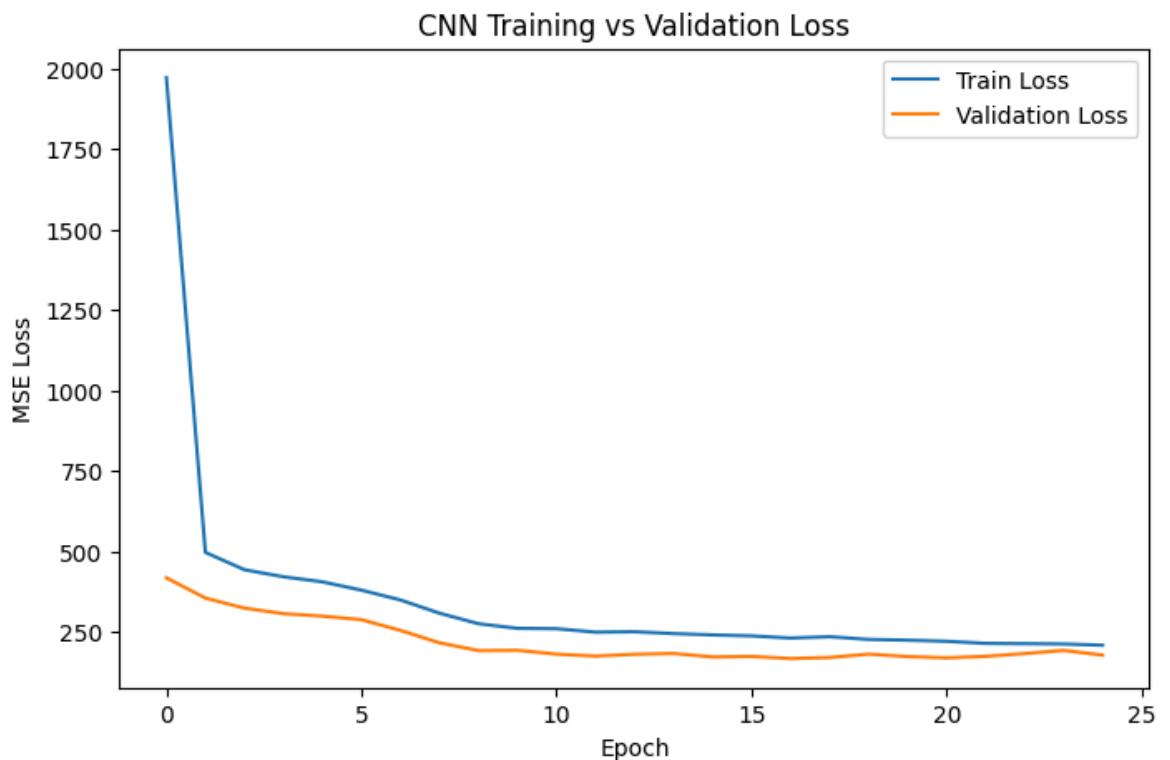
val_preds = np.array(val_preds)
val_true = np.array(val_true)

mae_val = mean_absolute_error(val_true, val_preds)
rmse_val = mean_squared_error(val_true, val_preds) ** 0.5
r2_val = r2_score(val_true, val_preds)

print("CNN Validation Results:")
print("MAE:", mae_val)
print("RMSE:", rmse_val)
print("R2:", r2_val)

CNN Validation Results:
MAE: 9.87984848022461
RMSE: 13.394794398115017
R2: 0.8970840573310852
```

```
In [94]: ▶ plt.figure(figsize=(8,5))
plt.plot(train_losses, label="Train Loss")
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("CNN Training vs Validation Loss")
plt.legend()
plt.show()
```



Experimental CNN Hyperparameter Optimization with Optuna

After stabilizing the baseline CNN using a lower learning rate and early stopping, we further explore whether validation performance can be improved through Optuna-based hyperparameter tuning.

To keep the search computationally manageable, we tune only a small set of important hyperparameters.

```
In [17]: ▶ class CNN1DOptunaRegressor(nn.Module):
def __init__(self, num_features, seq_len, conv1_channels, conv2_channels, dropout_rate,
super(CNN1DOptunaRegressor, self).__init__()

self.conv1 = nn.Conv1d(
    in_channels=num_features,
    out_channels=conv1_channels,
    kernel_size=3,
    padding=1
)
self.conv2 = nn.Conv1d(
    in_channels=conv1_channels,
    out_channels=conv2_channels,
    kernel_size=3,
    padding=1
)
self.pool = nn.MaxPool1d(kernel_size=2)
self.relu = nn.ReLU()
self.dropout = nn.Dropout(dropout_rate)

self.fc1 = nn.Linear(conv2_channels * (seq_len // 2), hidden_dim)
self.fc2 = nn.Linear(hidden_dim, 1)

def forward(self, x):
    x = x.permute(0, 2, 1)  # (batch, features, seq_len)
    x = self.relu(self.conv1(x))
    x = self.pool(self.relu(self.conv2(x)))
    x = x.reshape(x.size(0), -1)
    x = self.dropout(self.relu(self.fc1(x)))
    x = self.fc2(x)
    return x.squeeze(1)
```



```

In [18]: ▶ def objective_cnn(trial):
    conv1_channels = trial.suggest_categorical("conv1_channels", [16, 32, 64])
    conv2_channels = trial.suggest_categorical("conv2_channels", [32, 64, 128])
    hidden_dim = trial.suggest_categorical("hidden_dim", [32, 64, 128])
    dropout_rate = trial.suggest_float("dropout_rate", 0.0, 0.3)
    lr = trial.suggest_float("lr", 1e-4, 5e-3, log=True)
    batch_size = trial.suggest_categorical("batch_size", [32, 64, 128])

    model = CNN1DOptunaRegressor(
        num_features=len(selected_features),
        seq_len=SEQ_LEN,
        conv1_channels=conv1_channels,
        conv2_channels=conv2_channels,
        dropout_rate=dropout_rate,
        hidden_dim=hidden_dim
    ).to(device)

    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    train_dataset = RULDataset(X_train_seq, y_train_seq)
    val_dataset = RULDataset(X_val_seq, y_val_seq)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

    num_epochs = 20
    patience = 5
    best_val_loss = float('inf')
    patience_counter = 0
    best_model_state = None

    for epoch in range(num_epochs):
        model.train()
        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()

        model.eval()
        running_val_loss = 0.0

        with torch.no_grad():
            for X_batch, y_batch in val_loader:
                X_batch = X_batch.to(device)
                y_batch = y_batch.to(device)

                outputs = model(X_batch)
                loss = criterion(outputs, y_batch)
                running_val_loss += loss.item()

        avg_val_loss = running_val_loss / len(val_loader)

        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            patience_counter = 0
            best_model_state = model.state_dict()
        else:
            patience_counter += 1

        if patience_counter >= patience:
            break

    model.load_state_dict(best_model_state)
    model.eval()

    val_preds = []
    val_true = []

```

```

with torch.no_grad():
    for X_batch, y_batch in val_loader:
        X_batch = X_batch.to(device)
        outputs = model(X_batch).cpu().numpy()
        val_preds.extend(outputs)
        val_true.extend(y_batch.numpy())

val_preds = np.array(val_preds)
val_true = np.array(val_true)

r2 = r2_score(val_true, val_preds)
return r2

```

```

In [97]: study_cnn = optuna.create_study(direction="maximize")
study_cnn.optimize(objective_cnn, n_trials=10)

print("CNN Best R2:", study_cnn.best_trial.value)
print("CNN Best Params:", study_cnn.best_trial.params)F

```

[I 2026-04-15 12:45:21,911] A new study created in memory with name: no-name-b8b2f857-a43e-45cc-8ba6-81f415158c61

[I 2026-04-15 12:45:51,111] Trial 0 finished with value: 0.8926365375518799 and parameters: {'conv1_channels': 32, 'conv2_channels': 64, 'hidden_dim': 64, 'dropout_rate': 0.2518032839451027, 'lr': 0.0035284445859615665, 'batch_size': 128}. Best is trial 0 with value: 0.8926365375518799.

[I 2026-04-15 12:46:54,567] Trial 1 finished with value: 0.8995276093482971 and parameters: {'conv1_channels': 16, 'conv2_channels': 32, 'hidden_dim': 128, 'dropout_rate': 0.24993343870957307, 'lr': 0.0005244223604879921, 'batch_size': 64}. Best is trial 1 with value: 0.8995276093482971.

[I 2026-04-15 12:49:07,794] Trial 2 finished with value: 0.8961356282234192 and parameters: {'conv1_channels': 32, 'conv2_channels': 128, 'hidden_dim': 32, 'dropout_rate': 0.07928712862530678, 'lr': 0.00031686902050971367, 'batch_size': 32}. Best is trial 1 with value: 0.8995276093482971.

[I 2026-04-15 12:49:36,338] Trial 3 finished with value: 0.8720168471336365 and parameters: {'conv1_channels': 32, 'conv2_channels': 64, 'hidden_dim': 128, 'dropout_rate': 0.10847718651607784, 'lr': 0.004534678538619151, 'batch_size': 128}. Best is trial 1 with value: 0.8995276093482971.

[I 2026-04-15 12:50:41,009] Trial 4 finished with value: 0.8798004388809204 and parameters: {'conv1_channels': 16, 'conv2_channels': 128, 'hidden_dim': 128, 'dropout_rate': 0.103218003131211, 'lr': 0.0024377373237280523, 'batch_size': 32}. Best is trial 1 with value: 0.8995276093482971.

[I 2026-04-15 12:52:36,818] Trial 5 finished with value: 0.9016410708427429 and parameters: {'conv1_channels': 64, 'conv2_channels': 128, 'hidden_dim': 128, 'dropout_rate': 0.13486990721445916, 'lr': 0.00013962287240981466, 'batch_size': 64}. Best is trial 5 with value: 0.9016410708427429.

[I 2026-04-15 12:53:45,362] Trial 6 finished with value: 0.8741262555122375 and parameters: {'conv1_channels': 32, 'conv2_channels': 32, 'hidden_dim': 32, 'dropout_rate': 0.12350089662900945, 'lr': 0.001445008138620575, 'batch_size': 32}. Best is trial 5 with value: 0.9016410708427429.

[I 2026-04-15 12:54:31,886] Trial 7 finished with value: 0.8956966400146484 and parameters: {'conv1_channels': 16, 'conv2_channels': 64, 'hidden_dim': 32, 'dropout_rate': 0.23854967396617285, 'lr': 0.0010208159526565131, 'batch_size': 128}. Best is trial 5 with value: 0.9016410708427429.

[I 2026-04-15 12:55:42,206] Trial 8 finished with value: 0.8823659420013428 and parameters: {'conv1_channels': 32, 'conv2_channels': 64, 'hidden_dim': 64, 'dropout_rate': 0.10925219576438897, 'lr': 0.0012404563092110745, 'batch_size': 32}. Best is trial 5 with value: 0.9016410708427429.

[I 2026-04-15 12:56:17,976] Trial 9 finished with value: 0.8882256746292114 and parameters: {'conv1_channels': 64, 'conv2_channels': 64, 'hidden_dim': 128, 'dropout_rate': 0.0076069831725746545, 'lr': 0.0026107452482160383, 'batch_size': 128}. Best is trial 5 with value: 0.9016410708427429.

CNN Best R2: 0.9016410708427429

CNN Best Params: {'conv1_channels': 64, 'conv2_channels': 128, 'hidden_dim': 128, 'dropout_rate': 0.13486990721445916, 'lr': 0.00013962287240981466, 'batch_size': 64}

Final Model – Optuna-Tuned 1D CNN

Based on all experiments, the Optuna-tuned 1D CNN achieved the strongest validation performance.

The final model is retrained using the full FD001 training set and then evaluated on the official FD001 test set.

```
In [98]: ▶ # create full training sequences using all FD001 training engines
X_full_train_seq, y_full_train_seq = create_train_sequences(
    train_fd001_cnn,
    selected_features,
    SEQ_LEN
)

# official test sequences
X_official_test_seq = create_test_sequences(
    test_fd001_cnn,
    selected_features,
    SEQ_LEN
)

y_official_test = rul_fd001['RUL'].clip(upper=125).values.astype(np.float32)

print("Full training sequences:", X_full_train_seq.shape)
print("Official test sequences:", X_official_test_seq.shape)
print("Official test targets:", y_official_test.shape)
```

```
Full training sequences: (17731, 30, 10)
Official test sequences: (100, 30, 10)
Official test targets: (100,)
```

```
In [99]: ▶ final_train_dataset = RULDataset(X_full_train_seq, y_full_train_seq)
final_train_loader = DataLoader(final_train_dataset, batch_size=64, shuffle=True)
```

```
In [100]: ▶ best_cnn_params = study_cnn.best_trial.params

final_cnn_model = CNN1DOptunaRegressor(
    num_features=len(selected_features),
    seq_len=SEQ_LEN,
    conv1_channels=best_cnn_params['conv1_channels'],
    conv2_channels=best_cnn_params['conv2_channels'],
    dropout_rate=best_cnn_params['dropout_rate'],
    hidden_dim=best_cnn_params['hidden_dim']
).to(device)

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(final_cnn_model.parameters(), lr=best_cnn_params['lr'])

print(best_cnn_params)

{'conv1_channels': 64, 'conv2_channels': 128, 'hidden_dim': 128, 'dropout_rate': 0.13486990
721445916, 'lr': 0.00013962287240981466, 'batch_size': 64}
```

```
In [101]: ▶ X_train_final_seq, X_holdout_seq, y_train_final_seq, y_holdout_seq = train_test_split(
    X_full_train_seq,
    y_full_train_seq,
    test_size=0.1,
    random_state=42
)

train_final_dataset = RULDataset(X_train_final_seq, y_train_final_seq)
holdout_dataset = RULDataset(X_holdout_seq, y_holdout_seq)

train_final_loader = DataLoader(train_final_dataset, batch_size=best_cnn_params['batch_size'])
holdout_loader = DataLoader(holdout_dataset, batch_size=best_cnn_params['batch_size'], shuffle=True)
```

```
In [102]: ▶ num_epochs = 40
patience = 8

best_val_loss = float('inf')
patience_counter = 0
best_model_state = None

final_train_losses = []
final_holdout_losses = []

for epoch in range(num_epochs):
    final_cnn_model.train()
    running_train_loss = 0.0

    for X_batch, y_batch in train_final_loader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()
        outputs = final_cnn_model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()

        running_train_loss += loss.item()

    avg_train_loss = running_train_loss / len(train_final_loader)
    final_train_losses.append(avg_train_loss)

    final_cnn_model.eval()
    running_holdout_loss = 0.0

    with torch.no_grad():
        for X_batch, y_batch in holdout_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            outputs = final_cnn_model(X_batch)
            loss = criterion(outputs, y_batch)
            running_holdout_loss += loss.item()

    avg_holdout_loss = running_holdout_loss / len(holdout_loader)
    final_holdout_losses.append(avg_holdout_loss)

    print(f"Epoch [{epoch+1}/{num_epochs}], Train Loss: {avg_train_loss:.4f}, Holdout Loss: {avg_holdout_loss:.4f}")

    if avg_holdout_loss < best_val_loss:
        best_val_loss = avg_holdout_loss
        patience_counter = 0
        best_model_state = final_cnn_model.state_dict()
    else:
        patience_counter += 1

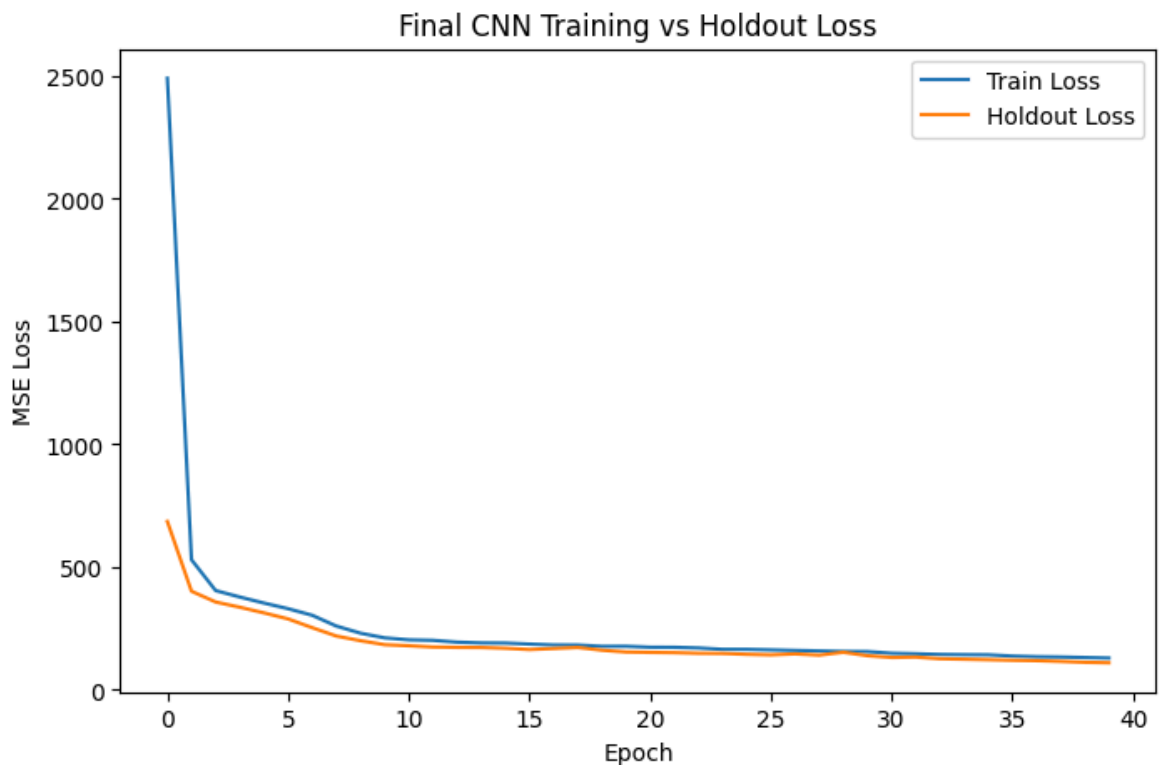
    if patience_counter >= patience:
        print("Early stopping triggered.")
        break

final_cnn_model.load_state_dict(best_model_state)
```

```
Epoch [1/40], Train Loss: 2491.0557, Holdout Loss: 686.1592
Epoch [2/40], Train Loss: 530.3287, Holdout Loss: 401.9756
Epoch [3/40], Train Loss: 404.6728, Holdout Loss: 358.0952
Epoch [4/40], Train Loss: 378.4252, Holdout Loss: 336.8701
Epoch [5/40], Train Loss: 353.9033, Holdout Loss: 313.8615
Epoch [6/40], Train Loss: 330.8970, Holdout Loss: 289.2775
Epoch [7/40], Train Loss: 304.0504, Holdout Loss: 253.5579
Epoch [8/40], Train Loss: 260.1553, Holdout Loss: 219.4657
Epoch [9/40], Train Loss: 231.1023, Holdout Loss: 200.5142
Epoch [10/40], Train Loss: 212.1499, Holdout Loss: 184.3006
Epoch [11/40], Train Loss: 204.1629, Holdout Loss: 179.9882
Epoch [12/40], Train Loss: 202.1223, Holdout Loss: 174.7065
Epoch [13/40], Train Loss: 194.5216, Holdout Loss: 173.3541
Epoch [14/40], Train Loss: 191.5095, Holdout Loss: 172.8355
Epoch [15/40], Train Loss: 190.9924, Holdout Loss: 169.7218
Epoch [16/40], Train Loss: 186.8281, Holdout Loss: 164.5923
Epoch [17/40], Train Loss: 183.2221, Holdout Loss: 168.6053
Epoch [18/40], Train Loss: 182.8641, Holdout Loss: 172.9202
Epoch [19/40], Train Loss: 177.6103, Holdout Loss: 161.8403
Epoch [20/40], Train Loss: 177.8661, Holdout Loss: 154.4583
Epoch [21/40], Train Loss: 173.9456, Holdout Loss: 152.9619
Epoch [22/40], Train Loss: 173.6908, Holdout Loss: 151.8753
Epoch [23/40], Train Loss: 171.0128, Holdout Loss: 149.2211
Epoch [24/40], Train Loss: 165.4323, Holdout Loss: 148.5417
Epoch [25/40], Train Loss: 165.2753, Holdout Loss: 145.0239
Epoch [26/40], Train Loss: 163.2604, Holdout Loss: 142.8420
Epoch [27/40], Train Loss: 160.9073, Holdout Loss: 146.7884
Epoch [28/40], Train Loss: 158.9323, Holdout Loss: 141.7853
Epoch [29/40], Train Loss: 156.8561, Holdout Loss: 153.4218
Epoch [30/40], Train Loss: 156.1554, Holdout Loss: 139.4314
Epoch [31/40], Train Loss: 149.4092, Holdout Loss: 132.2826
Epoch [32/40], Train Loss: 147.4237, Holdout Loss: 133.8363
Epoch [33/40], Train Loss: 144.5787, Holdout Loss: 127.0039
Epoch [34/40], Train Loss: 143.5983, Holdout Loss: 125.0949
Epoch [35/40], Train Loss: 143.1478, Holdout Loss: 123.1223
Epoch [36/40], Train Loss: 138.0339, Holdout Loss: 120.8258
Epoch [37/40], Train Loss: 135.3136, Holdout Loss: 119.3395
Epoch [38/40], Train Loss: 134.1070, Holdout Loss: 116.2045
Epoch [39/40], Train Loss: 132.0368, Holdout Loss: 112.5363
Epoch [40/40], Train Loss: 129.9529, Holdout Loss: 110.6203
```

Out[102]: <All keys matched successfully>

```
In [103]: ▶ plt.figure(figsize=(8,5))
plt.plot(final_train_losses, label="Train Loss")
plt.plot(final_holdout_losses, label="Holdout Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Final CNN Training vs Holdout Loss")
plt.legend()
plt.show()
```



Evaluation on Official FD001 Test Set

The final Optuna-tuned CNN is evaluated on the official FD001 test set using the last available sequence for each engine.

Performance is measured using MAE, RMSE, and R² score.

```
In [104]: ▶ X_official_test_tensor = torch.tensor(X_official_test_seq, dtype=torch.float32).to(device)

final_cnn_model.eval()
with torch.no_grad():
    y_pred_cnn_final = final_cnn_model(X_official_test_tensor).cpu().numpy()

mae_cnn = mean_absolute_error(y_official_test, y_pred_cnn_final)
rmse_cnn = mean_squared_error(y_official_test, y_pred_cnn_final) ** 0.5
r2_cnn = r2_score(y_official_test, y_pred_cnn_final)

print("Final Optuna-Tuned CNN Results:")
print("MAE:", mae_cnn)
print("RMSE:", rmse_cnn)
print("R2:", r2_cnn)
```

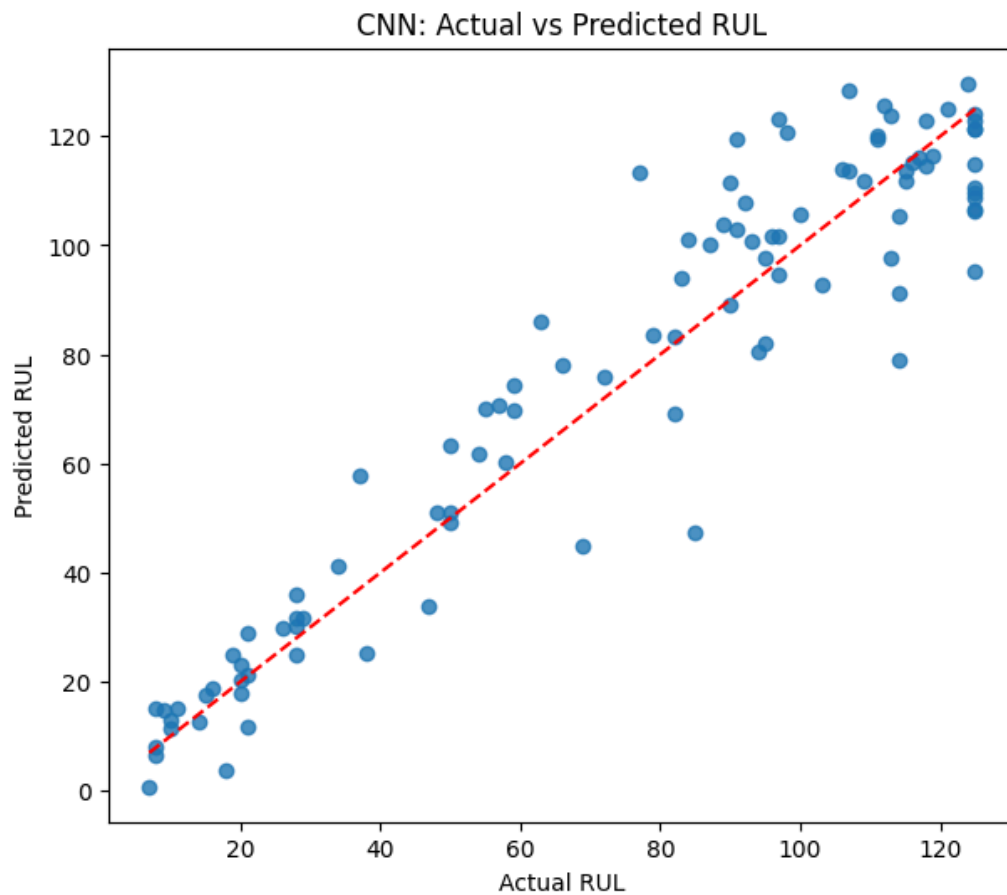
Final Optuna-Tuned CNN Results:

MAE: 9.675686836242676

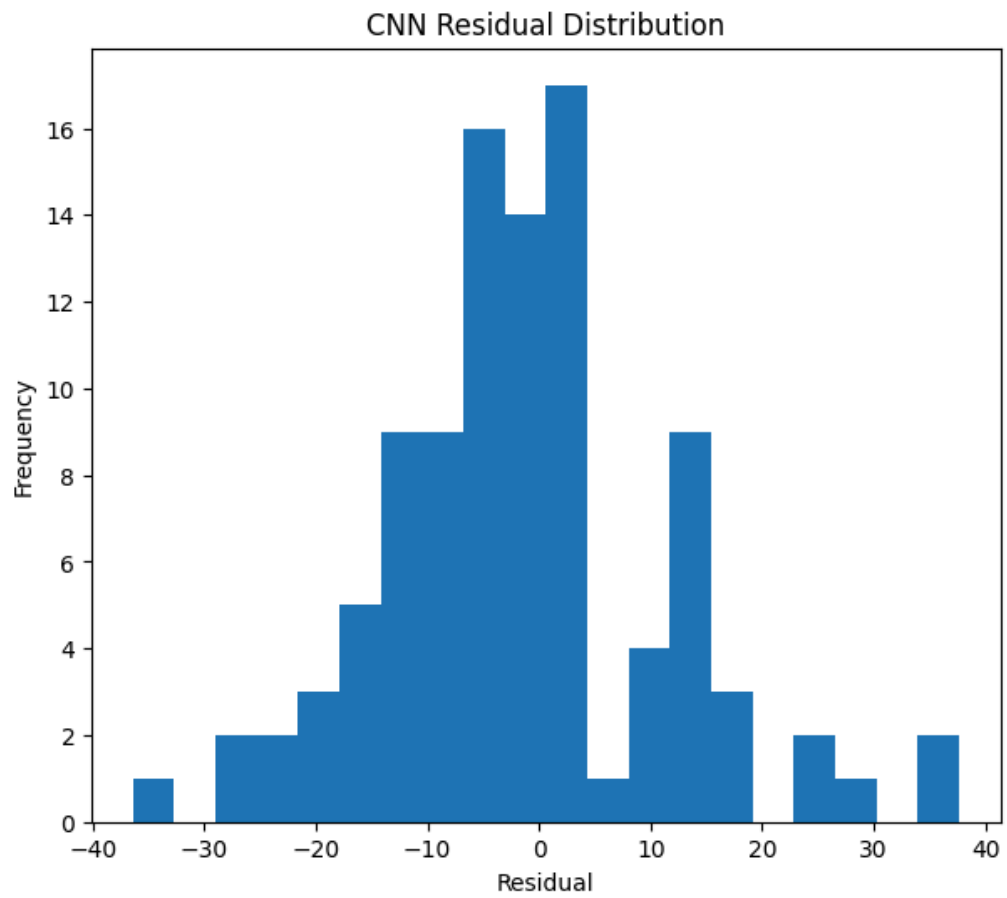
RMSE: 12.889485034821625

R2: 0.8965426087379456

```
In [105]: ▶ cnn_residuals = y_official_test - y_pred_cnn_final
plt.figure(figsize=(7,6))
plt.scatter(y_official_test, y_pred_cnn_final, alpha=0.8)
plt.plot(
    [y_official_test.min(), y_official_test.max()],
    [y_official_test.min(), y_official_test.max()],
    'r--'
)
plt.xlabel("Actual RUL")
plt.ylabel("Predicted RUL")
plt.title("CNN: Actual vs Predicted RUL")
plt.show()
```

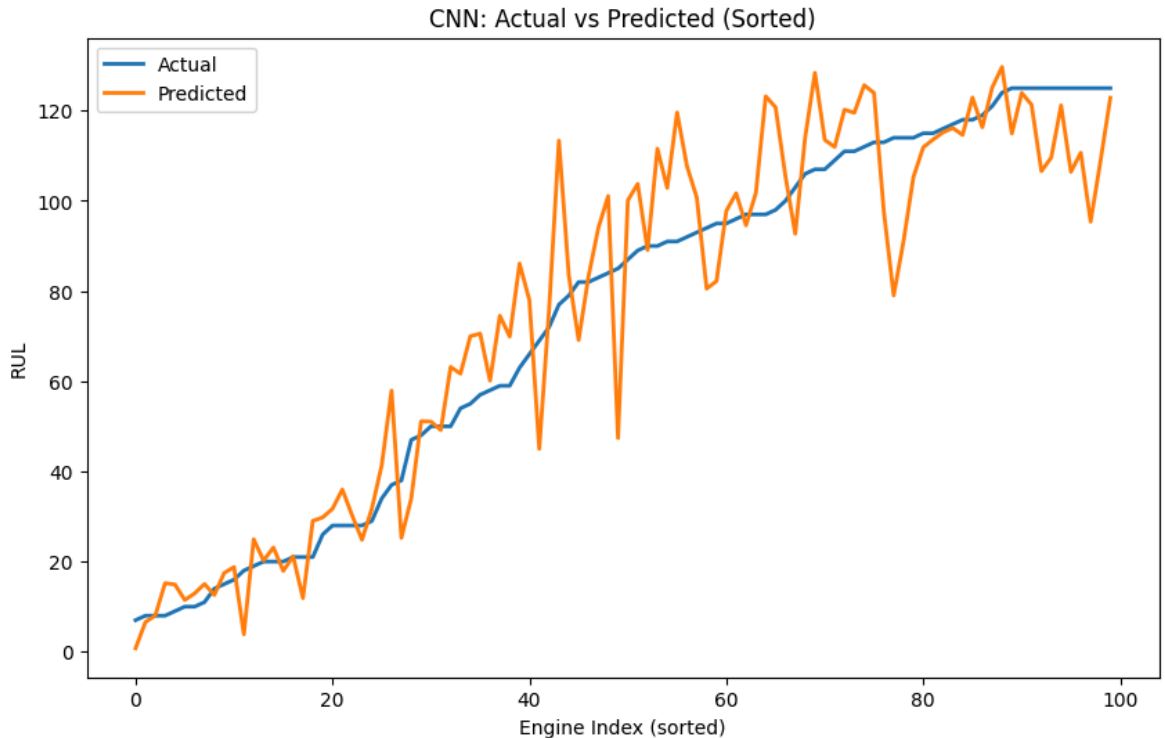


```
In [107]: ▶ plt.figure(figsize=(7,6))  
plt.hist(cnn_residuals, bins=20)  
plt.xlabel("Residual")  
plt.ylabel("Frequency")  
plt.title("CNN Residual Distribution")  
plt.show()
```



```
In [108]: sorted_idx = np.argsort(y_official_test)

plt.figure(figsize=(10,6))
plt.plot(y_official_test[sorted_idx], label="Actual", linewidth=2)
plt.plot(y_pred_cnn_final[sorted_idx], label="Predicted", linewidth=2)
plt.xlabel("Engine Index (sorted)")
plt.ylabel("RUL")
plt.title("CNN: Actual vs Predicted (Sorted)")
plt.legend()
plt.show()
```



Final Observations

The Optuna-tuned 1D CNN achieved the best performance among all tested models on the FD001 dataset.

Performance Metrics

- MAE ≈ 9.68
- RMSE ≈ 12.89
- $R^2 \approx 0.8965$

These results show a clear improvement over traditional machine learning models, including Random Forest, demonstrating the effectiveness of deep learning for time-series degradation modeling.

Analysis of Predictions

From the Actual vs Predicted plot:

- Most predictions closely follow the ideal diagonal line, indicating strong agreement with true RUL values.
- The model performs particularly well in the low and mid RUL ranges, where degradation patterns are more consistent.
- Slight dispersion is observed at higher RUL values, suggesting reduced accuracy for engines far from failure.

Residual Behavior

- Prediction errors are generally centered around zero, indicating that the model does not exhibit strong systematic bias.
- Some larger residuals appear at higher RUL values, which is expected since early-stage degradation is more difficult to predict.

Model Strengths

- The CNN effectively captures temporal dependencies in the data by using sequence windows of sensor measurements.
- Compared to tabular models, it better models the progression of degradation over time.
- Hyperparameter tuning using Optuna further improved model performance by identifying optimal architecture and training settings.

Overall Conclusion

The Optuna-tuned CNN provides a highly accurate and robust solution for Remaining Useful Life prediction.

By leveraging temporal patterns in the data, the CNN significantly outperforms traditional machine learning models, making it the most suitable model for this task.

Modular Pipeline for FD002, FD003, and FD004

After completing the full implementation, experimentation, and detailed analysis for FD001, we adopted a modular version of the same pipeline for the remaining datasets.

The goal of this step is to avoid unnecessary code repetition while keeping the preprocessing, feature engineering, model training, and evaluation methodology consistent across the datasets.

Using reusable functions allows us to apply the same workflow for the remaining datasets and keep the notebook more organized and scalable

The modular pipeline follows the same logic used in FD001, including:

- loading train, test, and RUL files
- assigning column names
- computing training RUL targets
- removing non-informative sensors
- preparing tabular features for machine learning models
- evaluating performance using MAE, RMSE, and R²

FD001 remains the full detailed case study, while the remaining datasets are processed using this modular implementation of the same approach. We apply the same sequence-based approach used in FD001 to capture temporal degradation patterns for the remaining datasets.

```
In [2]: ▶ %reload_ext autoreload
        %autoreload 2

        from rul_functions import *
```

```
In [1]: ▶ import rul_functions
        print(rul_functions.__file__)

        C:\Users\karam\ML_PROJECT\rul_functions.py
```

FD002 - Data Loading and Initial Inspection

```
In [3]: ▶ train_df_2, test_df_2, rul_df_2 = load_cmapss_data(
        "data/CMAPS/train_FD002.txt",
        "data/CMAPS/test_FD002.txt",
        "data/CMAPS/RUL_FD002.txt"
    )
```

```
In [4]: ▶ train_df_2 = add_train_rul(train_df_2)
        test_df_2 = add_test_rul(test_df_2, rul_df_2)
```

```
In [6]: ▶ basic_dataset_summary(train_df_2, "FD002 Training Data")
```

FD002 Training Data Shape: (53759, 27)

	unit_nr	cycle	op_setting_1	op_setting_2	op_setting_3	sensor_1	sensor_2	sensor_3	sensor_4	sensor_5	..
0	1	1	34.9983	0.8400	100.0	449.44	555.32	1358.61	1137.23	5.48	..
1	1	2	41.9982	0.8408	100.0	445.00	549.90	1353.22	1125.78	3.91	..
2	1	3	24.9988	0.6218	60.0	462.54	537.31	1256.76	1047.45	7.05	..
3	1	4	42.0077	0.8416	100.0	445.00	549.51	1354.03	1126.38	3.91	..
4	1	5	25.0005	0.6203	60.0	462.54	537.07	1257.71	1047.93	7.05	..

5 rows × 27 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53759 entries, 0 to 53758
Data columns (total 27 columns):
#   Column          Non-Null Count  Dtype
---  -
0   unit_nr         53759 non-null  int64
1   cycle           53759 non-null  int64
2   op_setting_1    53759 non-null  float64
3   op_setting_2    53759 non-null  float64
4   op_setting_3    53759 non-null  float64
5   sensor_1        53759 non-null  float64
6   sensor_2        53759 non-null  float64
7   sensor_3        53759 non-null  float64
8   sensor_4        53759 non-null  float64
9   sensor_5        53759 non-null  float64
10  sensor_6        53759 non-null  float64
11  sensor_7        53759 non-null  float64
12  sensor_8        53759 non-null  float64
13  sensor_9        53759 non-null  float64
14  sensor_10       53759 non-null  float64
15  sensor_11       53759 non-null  float64
16  sensor_12       53759 non-null  float64
17  sensor_13       53759 non-null  float64
18  sensor_14       53759 non-null  float64
19  sensor_15       53759 non-null  float64
20  sensor_16       53759 non-null  float64
21  sensor_17       53759 non-null  int64
22  sensor_18       53759 non-null  int64
23  sensor_19       53759 non-null  float64
24  sensor_20       53759 non-null  float64
25  sensor_21       53759 non-null  float64
26  RUL             53759 non-null  int64
dtypes: float64(22), int64(5)
memory usage: 11.1 MB
None
```

```
In [7]: missing_summary_2 = missing_values_summary(train_df_2)
```

Missing Count	Missing %
---------------	-----------

No missing values found.

```
In [8]: unique_summary_2 = unique_value_summary(train_df_2)
```

	Column	Unique Values
4	op_setting_3	2
23	sensor_19	2
20	sensor_16	2
5	sensor_1	6
9	sensor_5	6
22	sensor_18	6
14	sensor_10	9
10	sensor_6	14
21	sensor_17	53
3	op_setting_2	105
0	unit_nr	260
26	RUL	378
1	cycle	378
24	sensor_20	510
17	sensor_13	514
2	op_setting_1	536
15	sensor_11	681
12	sensor_8	897
6	sensor_2	1590
16	sensor_12	1672
11	sensor_7	2067
19	sensor_15	8464
7	sensor_3	12305
18	sensor_14	14905
8	sensor_4	15411
25	sensor_21	17837
13	sensor_9	22434

```
In [9]: low_variance_df_2 = low_variance_summary(train_df_2)
```

Columns with variance <= 1e-08:

Column	Variance
--------	----------

```
In [11]: zero_std_cols_2 = zero_std_columns(train_df_2)
```

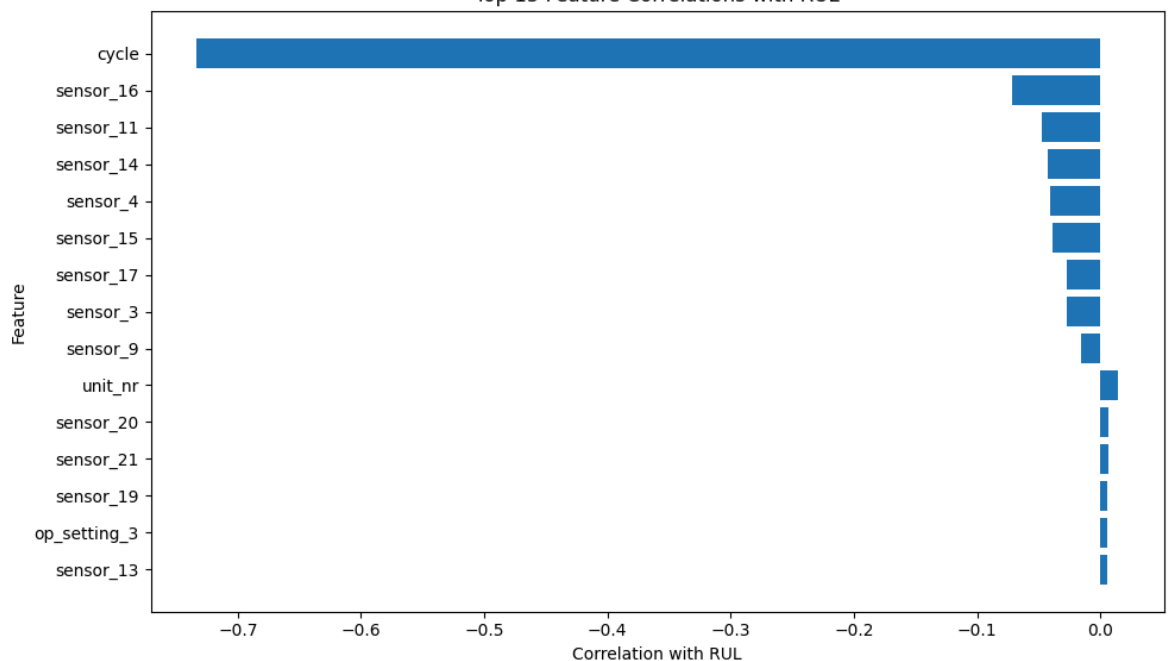
Zero standard deviation columns:

[]

```
In [13]: plot_top_correlations_with_rul(train_df_2, top_n=15)
```

	Feature	Correlation_with_RUL	Abs_Correlation
1	cycle	-0.733383	0.733383
20	sensor_16	-0.071352	0.071352
15	sensor_11	-0.046950	0.046950
18	sensor_14	-0.042325	0.042325
8	sensor_4	-0.040978	0.040978
19	sensor_15	-0.038455	0.038455
21	sensor_17	-0.027031	0.027031
7	sensor_3	-0.026942	0.026942
13	sensor_9	-0.015306	0.015306
0	unit_nr	0.014608	0.014608
24	sensor_20	0.006287	0.006287
25	sensor_21	0.006165	0.006165
23	sensor_19	0.005761	0.005761
4	op_setting_3	0.005761	0.005761
17	sensor_13	0.005245	0.005245
6	sensor_2	-0.004928	0.004928
22	sensor_18	0.004780	0.004780
12	sensor_8	0.004345	0.004345
14	sensor_10	0.004306	0.004306
11	sensor_7	0.002430	0.002430
16	sensor_12	0.002249	0.002249
2	op_setting_1	0.001135	0.001135
9	sensor_5	-0.000758	0.000758
10	sensor_6	-0.000496	0.000496
3	op_setting_2	0.000047	0.000047
5	sensor_1	-0.000023	0.000023

Top 15 Feature Correlations with RUL



```
In [5]: drop_cols_2 = [
        'op_setting_3',
        'sensor_16',
        'sensor_19',
        'sensor_1',
        'sensor_5',
        'sensor_18'
    ]
```

```
In [6]: train_df_2 = drop_columns(train_df_2, drop_cols_2)
        test_df_2 = drop_columns(test_df_2, drop_cols_2)
```

```
In [7]: train_split_2, val_split_2 = engine_level_train_val_split(train_df_2, test_size=0.2, random_
        )
```

```
In [8]: test_last_2 = get_last_cycle_per_engine(test_df_2)
```

```
In [9]: feature_cols_2 = get_feature_columns(train_split_2)

        print("Number of features:", len(feature_cols_2))
        print(feature_cols_2)
```

Number of features: 18

['op_setting_1', 'op_setting_2', 'sensor_2', 'sensor_3', 'sensor_4', 'sensor_6', 'sensor_7', 'sensor_8', 'sensor_9', 'sensor_10', 'sensor_11', 'sensor_12', 'sensor_13', 'sensor_14', 'sensor_15', 'sensor_17', 'sensor_20', 'sensor_21']

```
In [10]: X_train_2, X_val_2, X_test_2, y_train_2, y_val_2, y_test_2, scaler_2 = scale_tabular_feature
        test_last_2, feature_cols_2)
```

```
In [66]: models_2 = train_regression_models(
        X_train_2,
        y_train_2,
        random_state=42,
        include_xgboost=True
    )
```

Training Linear Regression...

Training KNN...

Training Decision Tree...

Training Random Forest...

Training SVR...

Training XGBoost...

```
In [67]: val_results_2 = evaluate_all_models(models_2, X_val_2, y_val_2)
```

	Model	MAE	RMSE	R2
0	SVR	30.823654	40.460873	0.621844
1	XGBoost	30.842751	40.914220	0.613323
2	Random Forest	30.713870	40.951934	0.612609
3	Linear Regression	33.504910	41.936825	0.593752
4	Decision Tree	33.519728	45.034608	0.531518
5	KNN	33.679689	45.531908	0.521114

```
In [68]: test_results_2 = evaluate_all_models(models_2, X_test_2, y_test_2)
```

	Model	MAE	RMSE	R2
0	SVR	23.884498	30.835755	0.671234
1	XGBoost	22.544596	31.071842	0.666180
2	Random Forest	23.026804	31.418324	0.658694
3	Linear Regression	28.304495	34.664356	0.584526
4	Decision Tree	25.245101	35.114415	0.573667
5	KNN	26.855598	37.358937	0.517423

```
In [69]: best_model_name_2 = test_results_2.iloc[0]['Model']
best_model_2 = models_2[best_model_name_2]
```

```
print("Best model for FD002:", best_model_name_2)
```

Best model for FD002: SVR

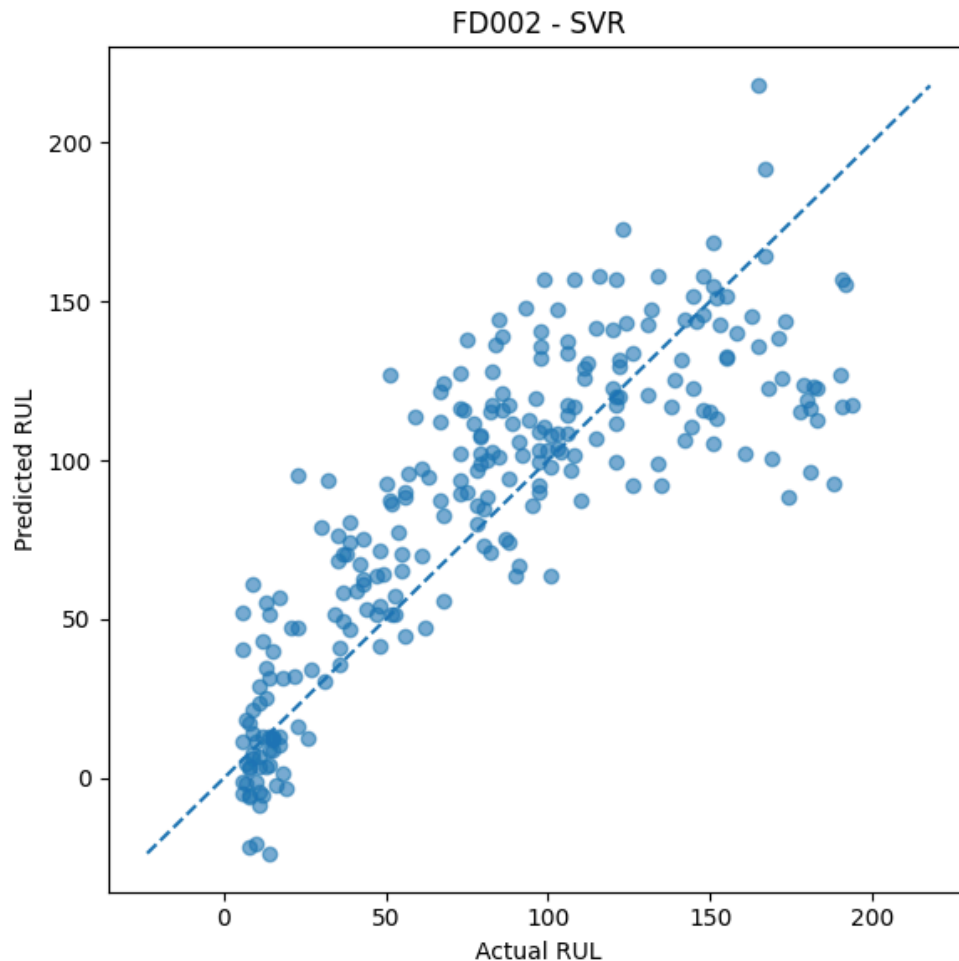
```
In [70]: best_output_2 = evaluate_regression_model(best_model_2, X_test_2, y_test_2)
```

```
y_pred_best_2 = best_output_2['y_pred']
```

```
print(f"MAE: {best_output_2['MAE']:.4f}")
print(f"RMSE: {best_output_2['RMSE']:.4f}")
print(f"R²: {best_output_2['R2']:.4f}")
```

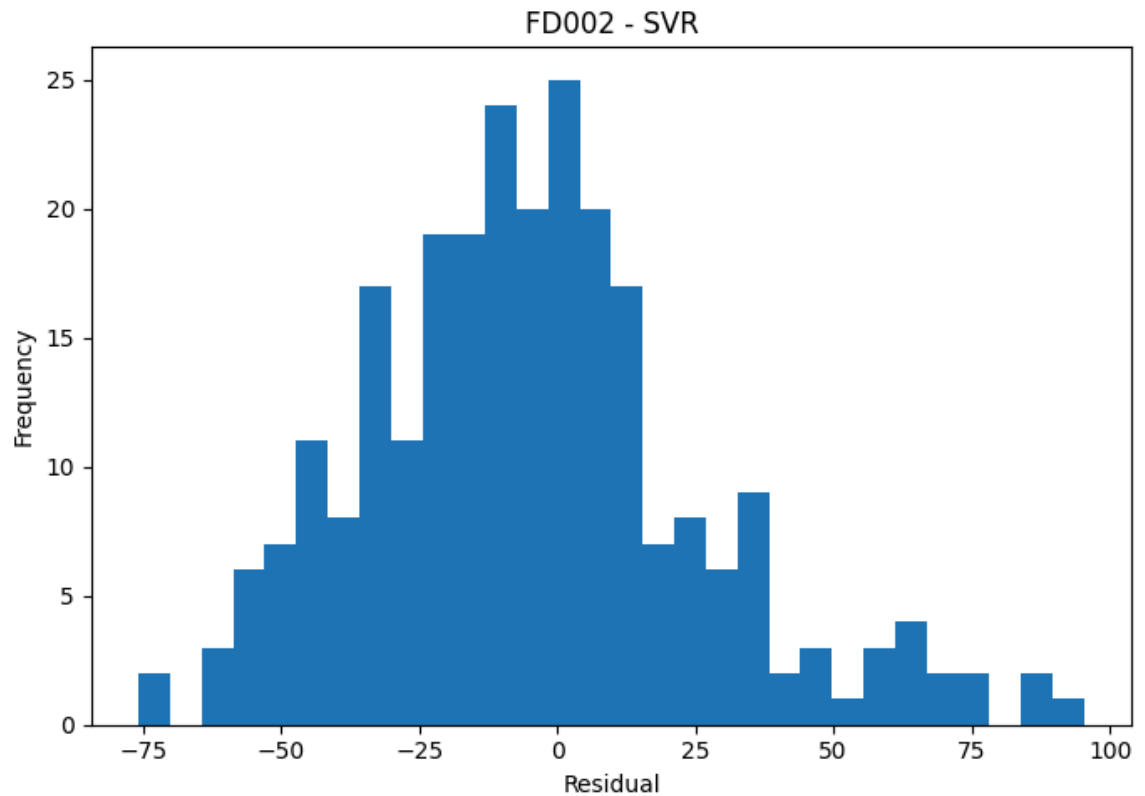
MAE: 23.8845
RMSE: 30.8358
R²: 0.6712

```
In [71]: plot_actual_vs_predicted(y_test_2, y_pred_best_2, title=f"FD002 - {best_model_name_2}")
```



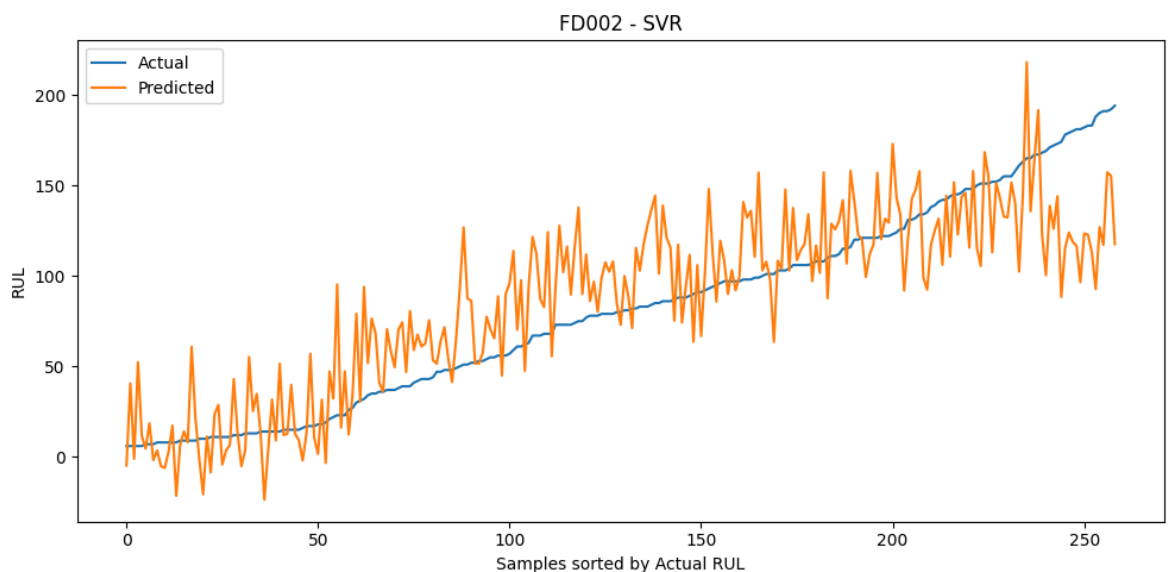
In [72]:

```
plot_residual_histogram(y_test_2, y_pred_best_2, title=f"FD002 - {best_model_name_2}")
```



In [73]:

```
plot_sorted_actual_vs_predicted(y_test_2, y_pred_best_2, title=f"FD002 - {best_model_name_2}")
```



FD002 - Model Performance Observation

The top three models on the FD002 test set were SVR, XGBoost, and Random Forest, with very close performance across all metrics.

Although SVR achieved the highest R^2 score, the overall results are not very strong, with R^2 values remaining below 0.70. This indicates that none of the traditional machine learning models are able to capture the underlying patterns in the data effectively.

The relatively weak performance and small differences between the top models suggest that FD002 is a more complex dataset, likely due to multiple operating conditions, making the prediction of RUL more challenging.

FD002 - CNN Model

In this section, we apply a CNN model to FD002 using sequence data.

Instead of single rows, the model uses sliding windows of sensor readings to capture temporal patterns.

The same setup as FD001 is used, and performance will be compared with the traditional models.

```
In [15]: ▶ # FD002 - CNN MODEL (same setup as FD001)

SEQ_LEN = 30
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 1. Scale sequence features using TRAIN split only
train_seq_df_2, val_seq_df_2, test_seq_df_2, cnn_scaler_2 = scale_sequence_features(
    train_split_2,
    val_split_2,
    test_df_2,
    feature_cols_2
)

# 2. Create sequences AFTER scaling
X_train_seq_2, y_train_seq_2 = create_train_sequences(
    train_seq_df_2,
    feature_cols_2,
    SEQ_LEN
)

X_val_seq_2, y_val_seq_2 = create_train_sequences(
    val_seq_df_2,
    feature_cols_2,
    SEQ_LEN
)

X_test_seq_2 = create_test_sequences(
    test_seq_df_2,
    feature_cols_2,
    SEQ_LEN
)

# official test target: last RUL per engine
y_test_seq_2 = test_seq_df_2.groupby('unit_nr')['RUL'].last().values.astype(np.float32)

print("FD002 CNN train sequence shape:", X_train_seq_2.shape)
print("FD002 CNN val sequence shape:", X_val_seq_2.shape)
print("FD002 CNN test sequence shape:", X_test_seq_2.shape)

FD002 CNN train sequence shape: (37432, 30, 18)
FD002 CNN val sequence shape: (8787, 30, 18)
FD002 CNN test sequence shape: (259, 30, 18)
```

In [75]:

```
# 3. Train with same FD001 settings
cnn_model_2, train_losses_2, val_losses_2 = train_cnn_model(
    X_train_seq=X_train_seq_2,
    y_train_seq=y_train_seq_2,
    X_val_seq=X_val_seq_2,
    y_val_seq=y_val_seq_2,
    num_features=len(feature_cols_2),
    seq_len=SEQ_LEN,
    batch_size=64,
    lr=0.0005,
    num_epochs=40,
    patience=8,
    device=device
)

# 4. Validation evaluation
cnn_val_output_2 = evaluate_cnn_model(
    cnn_model_2,
    X_val_seq_2,
    y_val_seq_2,
    device=device
)

print("FD002 CNN Validation Results:")
print("MAE:", cnn_val_output_2['MAE'])
print("RMSE:", cnn_val_output_2['RMSE'])
print("R2:", cnn_val_output_2['R2'])

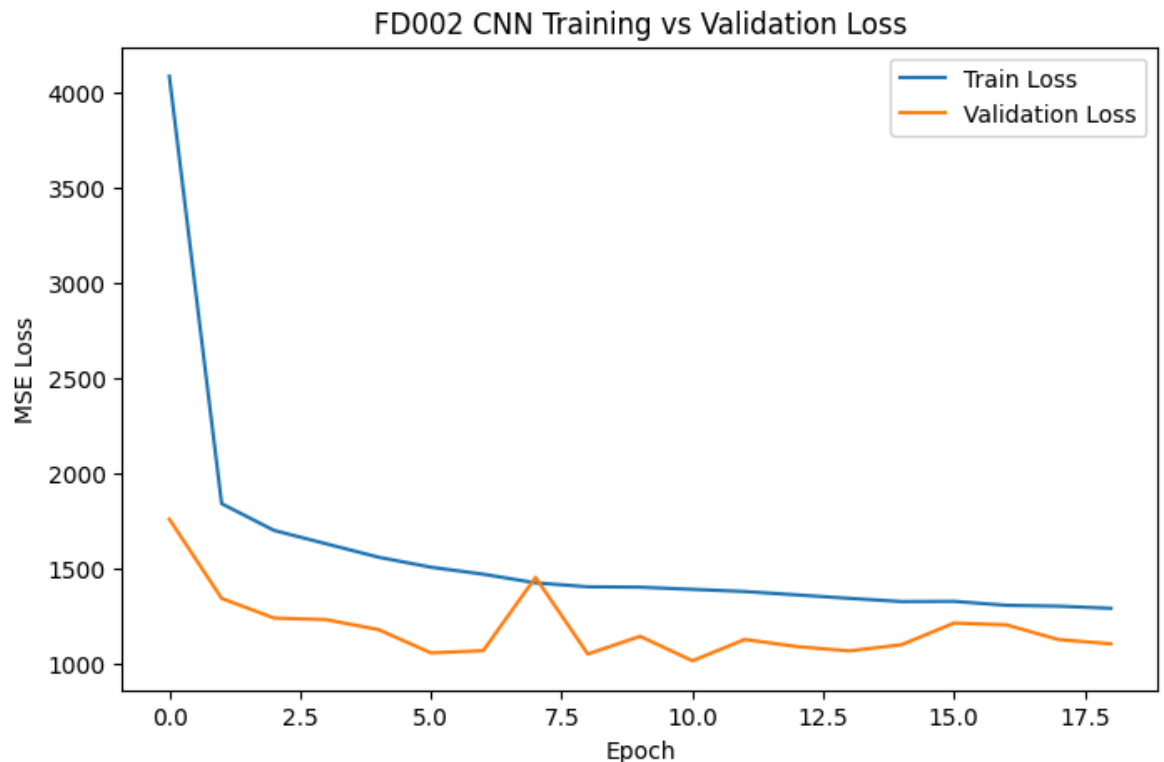
# 5. Official test evaluation
cnn_test_output_2 = evaluate_cnn_model(
    cnn_model_2,
    X_test_seq_2,
    y_test_seq_2,
    device=device
)

y_pred_cnn_2 = cnn_test_output_2['y_pred']

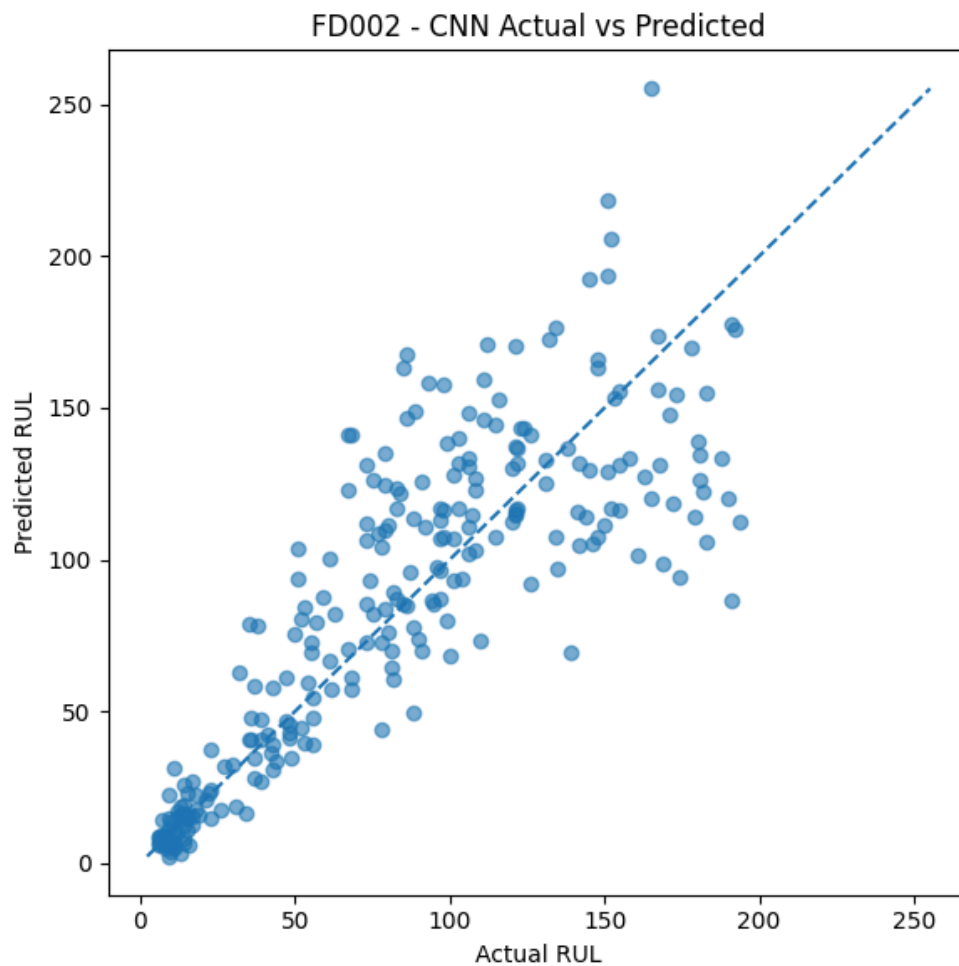
print("FD002 CNN Test Results:")
print("MAE:", cnn_test_output_2['MAE'])
print("RMSE:", cnn_test_output_2['RMSE'])
print("R2:", cnn_test_output_2['R2'])
```

```
FD002 CNN train sequence shape: (37432, 30, 18)
FD002 CNN val sequence shape: (8787, 30, 18)
FD002 CNN test sequence shape: (259, 30, 18)
Epoch [1/40], Train Loss: 4085.5245, Val Loss: 1756.5975
Epoch [2/40], Train Loss: 1838.1128, Val Loss: 1340.0176
Epoch [3/40], Train Loss: 1697.6355, Val Loss: 1236.9456
Epoch [4/40], Train Loss: 1627.0281, Val Loss: 1228.3310
Epoch [5/40], Train Loss: 1556.5425, Val Loss: 1175.8372
Epoch [6/40], Train Loss: 1503.9684, Val Loss: 1053.5180
Epoch [7/40], Train Loss: 1467.2958, Val Loss: 1065.1354
Epoch [8/40], Train Loss: 1421.9065, Val Loss: 1450.4523
Epoch [9/40], Train Loss: 1401.0646, Val Loss: 1047.6373
Epoch [10/40], Train Loss: 1399.0704, Val Loss: 1140.5872
Epoch [11/40], Train Loss: 1387.5922, Val Loss: 1012.0335
Epoch [12/40], Train Loss: 1376.1776, Val Loss: 1123.9476
Epoch [13/40], Train Loss: 1358.3279, Val Loss: 1086.5937
Epoch [14/40], Train Loss: 1340.3163, Val Loss: 1063.8689
Epoch [15/40], Train Loss: 1323.2797, Val Loss: 1096.6478
Epoch [16/40], Train Loss: 1324.4615, Val Loss: 1210.3479
Epoch [17/40], Train Loss: 1303.7139, Val Loss: 1201.1128
Epoch [18/40], Train Loss: 1299.1090, Val Loss: 1123.3597
Epoch [19/40], Train Loss: 1287.9927, Val Loss: 1101.3446
Early stopping triggered.
FD002 CNN Validation Results:
MAE: 24.259872436523438
RMSE: 33.26553876515361
R2: 0.6800815463066101
FD002 CNN Test Results:
MAE: 21.11872100830078
RMSE: 29.97668771963682
R2: 0.6892971992492676
```

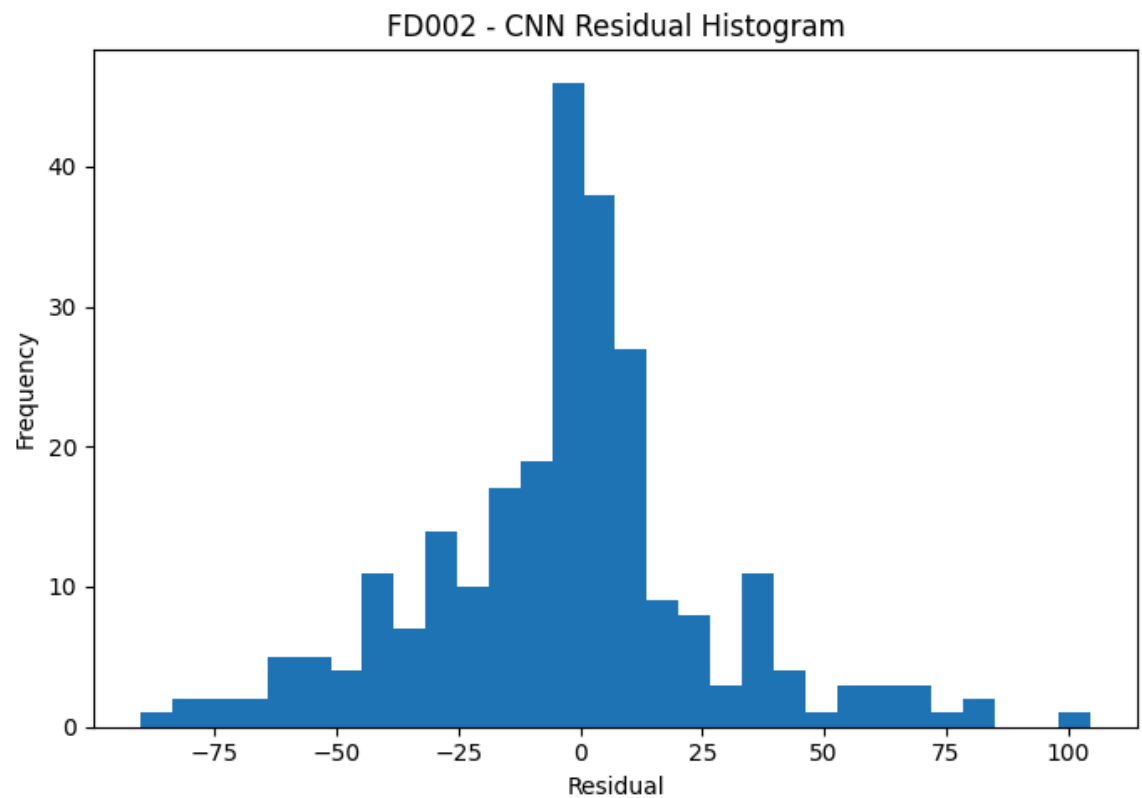
```
In [77]: ▶ plt.figure(figsize=(8,5))
plt.plot(train_losses_2, label="Train Loss")
plt.plot(val_losses_2, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("FD002 CNN Training vs Validation Loss")
plt.legend()
plt.show()
```



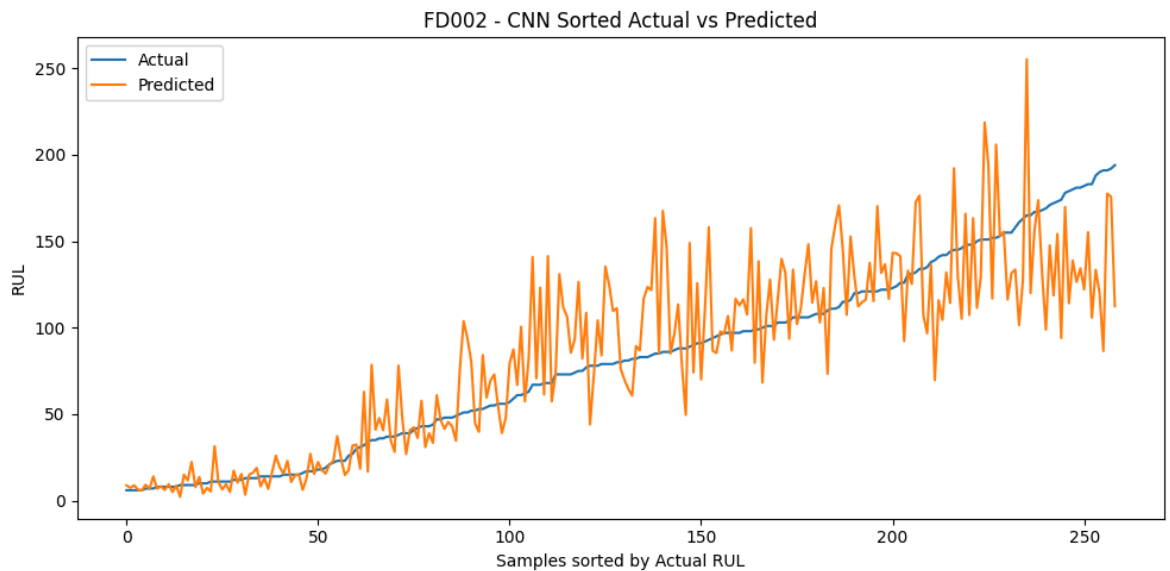
```
In [78]: plot_actual_vs_predicted(y_test_seq_2, y_pred_cnn_2, title="FD002 - CNN Actual vs Predicted")
```



```
In [79]: plot_residual_histogram(y_test_seq_2, y_pred_cnn_2, title="FD002 - CNN Residual Histogram")
```



```
In [80]: plot_sorted_actual_vs_predicted(y_test_seq_2, y_pred_cnn_2, title="FD002 - CNN Sorted Actual
```



FD002 - CNN Hyperparameter Tuning (OPTUNA)

Since the CNN achieved better performance than the traditional machine learning models on FD002, we further improve it using hyperparameter tuning.

The objective is to search for a better CNN configuration by varying key parameters such as the learning rate, batch size, sequence length, and training patience, and then evaluate whether tuning leads to better generalization on the official FD002 test set.


```

In [27]: ▶ def objective_cnn_fd002(trial):
    conv1_channels = trial.suggest_categorical("conv1_channels", [16, 32, 64])
    conv2_channels = trial.suggest_categorical("conv2_channels", [32, 64, 128])
    hidden_dim = trial.suggest_categorical("hidden_dim", [32, 64, 128])
    dropout_rate = trial.suggest_float("dropout_rate", 0.0, 0.3)
    lr = trial.suggest_float("lr", 1e-4, 5e-3, log=True)
    batch_size = trial.suggest_categorical("batch_size", [32, 64, 128])

    model = CNN1DOptunaRegressor(
        num_features=len(feature_cols_2),
        seq_len=SEQ_LEN,
        conv1_channels=conv1_channels,
        conv2_channels=conv2_channels,
        dropout_rate=dropout_rate,
        hidden_dim=hidden_dim
    ).to(device)

    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    train_dataset = RULDataset(X_train_seq_2, y_train_seq_2)
    val_dataset = RULDataset(X_val_seq_2, y_val_seq_2)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

    num_epochs = 20
    patience = 5
    best_val_loss = float('inf')
    patience_counter = 0
    best_model_state = None

    for epoch in range(num_epochs):
        model.train()
        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()

        model.eval()
        running_val_loss = 0.0

        with torch.no_grad():
            for X_batch, y_batch in val_loader:
                X_batch = X_batch.to(device)
                y_batch = y_batch.to(device)

                outputs = model(X_batch)
                loss = criterion(outputs, y_batch)
                running_val_loss += loss.item()

        avg_val_loss = running_val_loss / len(val_loader)

        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            patience_counter = 0
            best_model_state = model.state_dict()
        else:
            patience_counter += 1

        if patience_counter >= patience:
            break

    model.load_state_dict(best_model_state)
    model.eval()

    val_preds = []
    val_true = []

```

```

with torch.no_grad():
    for X_batch, y_batch in val_loader:
        X_batch = X_batch.to(device)
        outputs = model(X_batch).cpu().numpy()
        val_preds.extend(outputs)
        val_true.extend(y_batch.numpy())

val_preds = np.array(val_preds)
val_true = np.array(val_true)

r2 = r2_score(val_true, val_preds)
return r2

```

```

In [28]: study_cnn_fd002 = optuna.create_study(direction="maximize")
study_cnn_fd002.optimize(objective_cnn_fd002, n_trials=10)

print("FD002 CNN Best R2:", study_cnn_fd002.best_trial.value)
print("FD002 CNN Best Params:", study_cnn_fd002.best_trial.params)

```

[I 2026-04-17 17:19:19,456] A new study created in memory with name: no-name-015ac320-61d6-4b63-b35b-c0fdaf2a2b31

[I 2026-04-17 17:22:03,659] Trial 0 finished with value: 0.6506959199905396 and parameters: {'conv1_channels': 32, 'conv2_channels': 64, 'hidden_dim': 32, 'dropout_rate': 0.04937702218754351, 'lr': 0.002609966373816166, 'batch_size': 32}. Best is trial 0 with value: 0.6506959199905396.

[I 2026-04-17 17:23:36,980] Trial 1 finished with value: 0.7045190334320068 and parameters: {'conv1_channels': 64, 'conv2_channels': 32, 'hidden_dim': 32, 'dropout_rate': 0.2389124907922212, 'lr': 0.00015770335165046057, 'batch_size': 64}. Best is trial 1 with value: 0.7045190334320068.

[I 2026-04-17 17:24:52,715] Trial 2 finished with value: 0.5946800708770752 and parameters: {'conv1_channels': 64, 'conv2_channels': 128, 'hidden_dim': 128, 'dropout_rate': 0.18641171027056055, 'lr': 0.0007076197500226295, 'batch_size': 64}. Best is trial 1 with value: 0.7045190334320068.

[I 2026-04-17 17:26:32,761] Trial 3 finished with value: 0.5912070274353027 and parameters: {'conv1_channels': 32, 'conv2_channels': 128, 'hidden_dim': 32, 'dropout_rate': 0.2802104097782955, 'lr': 0.0027691115842486936, 'batch_size': 32}. Best is trial 1 with value: 0.7045190334320068.

[I 2026-04-17 17:26:54,243] Trial 4 finished with value: 0.6254923343658447 and parameters: {'conv1_channels': 32, 'conv2_channels': 32, 'hidden_dim': 64, 'dropout_rate': 0.12723360185816104, 'lr': 0.004533580562700378, 'batch_size': 128}. Best is trial 1 with value: 0.7045190334320068.

[I 2026-04-17 17:31:00,937] Trial 5 finished with value: 0.5929185152053833 and parameters: {'conv1_channels': 64, 'conv2_channels': 32, 'hidden_dim': 64, 'dropout_rate': 0.2770147367087931, 'lr': 0.003600769459242079, 'batch_size': 32}. Best is trial 1 with value: 0.7045190334320068.

[I 2026-04-17 17:31:38,208] Trial 6 finished with value: 0.6761270761489868 and parameters: {'conv1_channels': 64, 'conv2_channels': 32, 'hidden_dim': 32, 'dropout_rate': 0.009157240402586808, 'lr': 0.004618099215255912, 'batch_size': 128}. Best is trial 1 with value: 0.7045190334320068.

[I 2026-04-17 17:33:02,466] Trial 7 finished with value: 0.5955452919006348 and parameters: {'conv1_channels': 16, 'conv2_channels': 64, 'hidden_dim': 64, 'dropout_rate': 0.23660778282855593, 'lr': 0.0028341506418727395, 'batch_size': 32}. Best is trial 1 with value: 0.7045190334320068.

[I 2026-04-17 17:33:38,809] Trial 8 finished with value: 0.6703786849975586 and parameters: {'conv1_channels': 64, 'conv2_channels': 64, 'hidden_dim': 128, 'dropout_rate': 0.24960585541228164, 'lr': 0.002438089682574423, 'batch_size': 128}. Best is trial 1 with value: 0.7045190334320068.

[I 2026-04-17 17:35:02,750] Trial 9 finished with value: 0.6538012623786926 and parameters: {'conv1_channels': 64, 'conv2_channels': 128, 'hidden_dim': 32, 'dropout_rate': 0.10928951759859606, 'lr': 0.0009496950560301928, 'batch_size': 32}. Best is trial 1 with value: 0.7045190334320068.

FD002 CNN Best R2: 0.7045190334320068

FD002 CNN Best Params: {'conv1_channels': 64, 'conv2_channels': 32, 'hidden_dim': 32, 'dropout_rate': 0.2389124907922212, 'lr': 0.00015770335165046057, 'batch_size': 64}

FD002 CNN Model – Observations

- The CNN model achieved an R^2 score of approximately **0.69**, which is noticeably lower than the performance on FD001 (~0.89).

- This drop in performance is expected, as **FD002 is more complex**, containing **multiple operating conditions**, which introduces higher variability in the data.
- The CNN slightly **outperforms traditional models** (e.g., SVR, Random Forest), but the improvement is **not significant**, unlike in FD001.
- This suggests that **temporal patterns are less dominant** in FD002, making sequence-based learning less effective.
- The **validation and test results are very close**, indicating that the model generalizes well and does not suffer from overfitting.
- Early stopping was triggered relatively early, showing that the model **converged quickly** and additional training would not significantly improve performance.
- After applying **Optuna-based hyperparameter tuning**, only **marginal improvements** were observed.
 - This indicates that the CNN was already near its optimal configuration.
 - Further tuning did not significantly improve performance, suggesting that **model capacity is not the limiting factor**.
- This highlights that performance is primarily constrained by **dataset complexity rather than hyperparameter selection**.

FD002 is a more challenging dataset, and while CNN provides some improvement, **the benefits of deep learning are reduced compared to FD001 due to increased data complexity and variability**

FD003 - Data Loading and Initial Inspection

```
In [81]: ▶ train_df_3, test_df_3, rul_df_3 = load_cmapss_data(  
    "data/CMAPS/train_FD003.txt",  
    "data/CMAPS/test_FD003.txt",  
    "data/CMAPS/RUL_FD003.txt"  
)
```

```
In [100]: ▶ train_df_3 = add_train_rul(train_df_3)  
test_df_3 = add_test_rul(test_df_3, rul_df_3)
```

```
In [101]: basic_dataset_summary(train_df_3, "FD003 Training Data")
```

FD003 Training Data Shape: (24720, 21)

	unit_nr	cycle	op_setting_1	op_setting_2	sensor_2	sensor_3	sensor_4	sensor_6	sensor_7	sensor_8	...	s
0	1	1	-0.0005	0.0004	642.36	1583.23	1396.84	21.61	553.97	2387.96	...	
1	1	2	0.0008	-0.0003	642.50	1584.69	1396.89	21.61	554.55	2388.00	...	
2	1	3	-0.0014	-0.0002	642.18	1582.35	1405.61	21.61	554.43	2388.03	...	
3	1	4	-0.0020	0.0001	642.92	1585.61	1392.27	21.61	555.21	2388.00	...	
4	1	5	0.0016	0.0000	641.68	1588.63	1397.65	21.61	554.74	2388.04	...	

5 rows × 21 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24720 entries, 0 to 24719
Data columns (total 21 columns):
#   Column          Non-Null Count  Dtype
---  -
0   unit_nr         24720 non-null  int64
1   cycle           24720 non-null  int64
2   op_setting_1    24720 non-null  float64
3   op_setting_2    24720 non-null  float64
4   sensor_2        24720 non-null  float64
5   sensor_3        24720 non-null  float64
6   sensor_4        24720 non-null  float64
7   sensor_6        24720 non-null  float64
8   sensor_7        24720 non-null  float64
9   sensor_8        24720 non-null  float64
10  sensor_9        24720 non-null  float64
11  sensor_10       24720 non-null  float64
12  sensor_11       24720 non-null  float64
13  sensor_12       24720 non-null  float64
14  sensor_13       24720 non-null  float64
15  sensor_14       24720 non-null  float64
16  sensor_15       24720 non-null  float64
17  sensor_17       24720 non-null  int64
18  sensor_20       24720 non-null  float64
19  sensor_21       24720 non-null  float64
20  RUL             24720 non-null  int64
dtypes: float64(17), int64(4)
memory usage: 4.0 MB
None
```

```
In [102]: missing_summary_3 = missing_values_summary(train_df_3)
```

```
Missing Count  Missing %
-----
No missing values found.
```

```
In [103]: unique_summary_3 = unique_value_summary(train_df_3)
```

	Column	Unique Values
11	sensor_10	4
17	sensor_17	12
3	op_setting_2	14
7	sensor_6	17
0	unit_nr	100
2	op_setting_1	160
9	sensor_8	161
14	sensor_13	163
18	sensor_20	165
12	sensor_11	170
4	sensor_2	334
1	cycle	525
20	RUL	525
13	sensor_12	1772
8	sensor_7	1854
16	sensor_15	3122
5	sensor_3	3358
6	sensor_4	4383
15	sensor_14	6320
19	sensor_21	6440
10	sensor_9	7114

```
In [104]: low_variance_df_3 = low_variance_summary(train_df_3)
```

Columns with variance <= 1e-08:

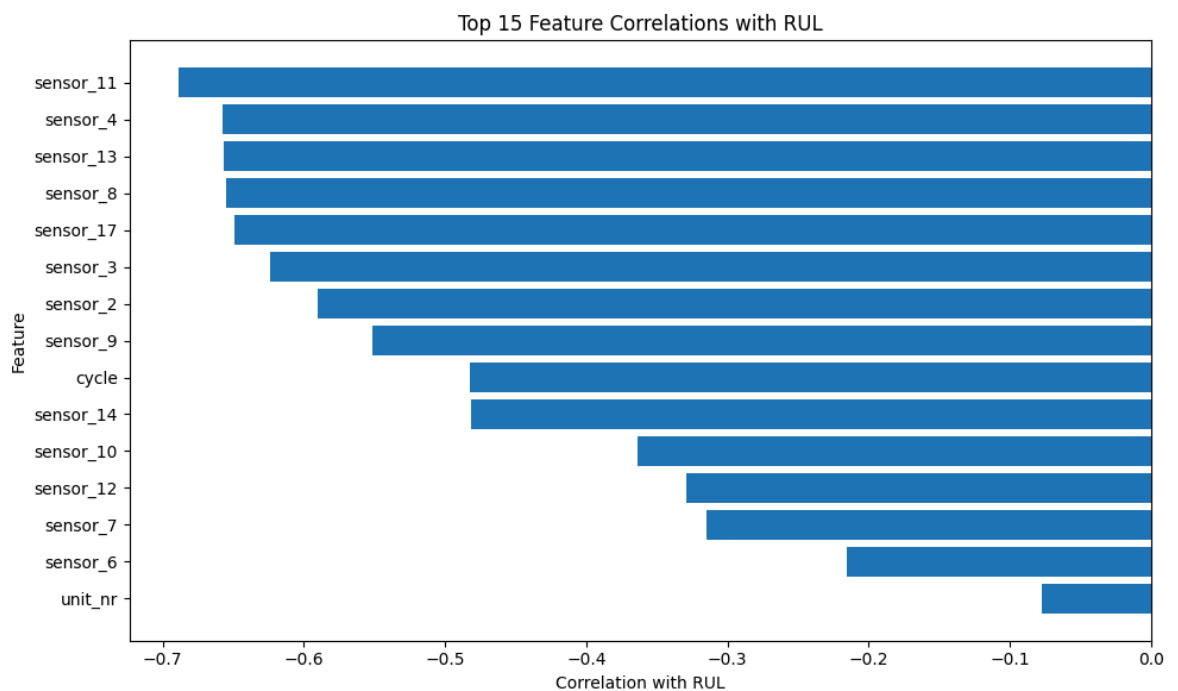
Column	Variance
--------	----------

```
In [105]: zero_std_cols_3 = zero_std_columns(train_df_3)
```

Zero standard deviation columns:
[]

```
In [106]: plot_top_correlations_with_rul(train_df_3, top_n=15)
```

	Feature	Correlation_with_RUL	Abs_Correlation
12	sensor_11	-0.688915	0.688915
6	sensor_4	-0.657224	0.657224
14	sensor_13	-0.656340	0.656340
9	sensor_8	-0.655345	0.655345
17	sensor_17	-0.649220	0.649220
5	sensor_3	-0.624236	0.624236
4	sensor_2	-0.590564	0.590564
10	sensor_9	-0.551843	0.551843
1	cycle	-0.482744	0.482744
15	sensor_14	-0.481807	0.481807
11	sensor_10	-0.363432	0.363432
13	sensor_12	-0.329452	0.329452
8	sensor_7	-0.315048	0.315048
7	sensor_6	-0.215352	0.215352
0	unit_nr	-0.077196	0.077196
18	sensor_20	0.037782	0.037782
19	sensor_21	0.033465	0.033465
16	sensor_15	-0.016501	0.016501
2	op_setting_1	0.003700	0.003700
3	op_setting_2	-0.001204	0.001204



```
In [107]: drop_cols_3 = [  
    'op_setting_3',  
    'sensor_1',  
    'sensor_18',  
    'sensor_19',  
    'sensor_16',  
    'sensor_5'  
]
```

```
In [108]: train_df_3 = drop_columns(train_df_3, drop_cols_3)
          test_df_3 = drop_columns(test_df_3, drop_cols_3)

In [110]: def filter_late_cycles(df, threshold=125):
          #Keep only samples where RUL <= threshold
          return df[df['RUL'] <= threshold].copy()

In [111]: train_df_3 = filter_late_cycles(train_df_3, threshold=125)

In [112]: train_split_3, val_split_3 = engine_level_train_val_split(train_df_3)

In [113]: # get last cycle only
          test_last_3 = get_last_cycle_per_engine(test_df_3)

          # assign TRUE RUL directly
          test_last_3['RUL'] = rul_df_3['RUL'].values

In [114]: X_train_3, X_val_3, X_test_3, y_train_3, y_val_3, y_test_3, scaler_3 = scale_tabular_features(
          train_split_3,
          val_split_3,
          test_last_3,
          feature_cols_3
          )

In [115]: models_3 = train_regression_models(X_train_3, y_train_3)
          val_results_3 = evaluate_all_models(models_3, X_val_3, y_val_3)
          test_results_3 = evaluate_all_models(models_3, X_test_3, y_test_3)
```

Training Linear Regression...
Training KNN...
Training Decision Tree...
Training Random Forest...
Training SVR...
Training XGBoost...

	Model	MAE	RMSE	R2
0	SVR	12.806039	17.107337	0.778776
1	Random Forest	12.910485	17.242885	0.775256
2	XGBoost	13.010041	17.421021	0.770589
3	KNN	14.147937	18.797526	0.732903
4	Linear Regression	14.935910	19.225627	0.720599
5	Decision Tree	15.214108	20.211172	0.691219

	Model	MAE	RMSE	R2
0	SVR	15.396004	20.390575	0.757367
1	Random Forest	16.417813	21.217981	0.737276
2	XGBoost	16.698751	21.903797	0.720018
3	Linear Regression	17.228542	21.929052	0.719372
4	KNN	18.658000	23.636793	0.673962
5	Decision Tree	19.285558	24.639928	0.645701

FD003 Model Observations and Key Insights

- The best-performing model on FD003 (SVR) achieved an R^2 score of approximately **0.75**, which is **lower than FD001** but **higher than FD002**, aligning with the expected dataset difficulty.
- Initially, the test results produced **negative R^2 values**, indicating a critical issue in the evaluation pipeline rather than poor model performance.

Root Cause: Evaluation and Distribution Mismatch

Two key issues were identified:

1. Incorrect Test RUL Handling

- The function `add_test_rul()` computes RUL values for all cycles.
- However, the NASA C-MAPSS evaluation protocol requires using **only the final cycle per engine**, with RUL values taken directly from the provided RUL file.
- Using incorrect RUL values introduces inconsistencies between predicted and true targets.

2. Training–Testing Distribution Mismatch

- The model was trained on the **full lifecycle** of engines (including early cycles with high RUL values).
- The test set, however, contains only **final-cycle data**, corresponding to **low RUL values near failure**.
- This mismatch caused the model to generalize poorly, resulting in invalid (negative) R^2 scores.

Key Fixes Applied

- Extracted only the **last cycle per engine** for testing and assigned the true RUL values directly from the RUL file.
- Filtered the training data to focus on **late-stage degradation ($RUL \leq 125$)**, aligning it with the test distribution.

After these corrections, model performance became stable and meaningful.

FD003 highlights that **correct evaluation procedures and aligned data distributions are critical**.

Even small inconsistencies in preprocessing can lead to major performance degradation, especially in more complex

FD003 - CNN Model

In this section, we apply a CNN model to FD003 using sequence data.

Instead of single rows, the model uses sliding windows of sensor readings to capture temporal patterns.

The same setup as FD001 and FD002 is used, and performance will be compared with the traditional models.

```
In [140]: ▶ %reload_ext autoreload
          %autoreload 2

          from rul_functions import *
```



```

In [141]: # FD003 - CNN MODEL (same setup as FD001 and FD002)

SEQ_LEN = 30
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 1. Scale sequence features using TRAIN split only
train_seq_df_3, val_seq_df_3, test_seq_df_3, cnn_scaler_3 = scale_sequence_features(
    train_split_3,
    val_split_3,
    test_df_3,
    feature_cols_3
)

# 2. Create sequences AFTER scaling
X_train_seq_3, y_train_seq_3 = create_train_sequences(
    train_seq_df_3,
    feature_cols_3,
    SEQ_LEN
)

X_val_seq_3, y_val_seq_3 = create_train_sequences(
    val_seq_df_3,
    feature_cols_3,
    SEQ_LEN
)

X_test_seq_3 = create_test_sequences(
    test_seq_df_3,
    feature_cols_3,
    SEQ_LEN
)

# official test target: Last RUL per engine
y_test_seq_3 = test_seq_df_3.groupby('unit_nr')['RUL'].last().values.astype(np.float32)

print("FD003 CNN train sequence shape:", X_train_seq_3.shape)
print("FD003 CNN val sequence shape:", X_val_seq_3.shape)
print("FD003 CNN test sequence shape:", X_test_seq_3.shape)

# 3. Train with same FD001 settings
cnn_model_3, train_losses_3, val_losses_3 = train_cnn_model(
    X_train_seq=X_train_seq_3,
    y_train_seq=y_train_seq_3,
    X_val_seq=X_val_seq_3,
    y_val_seq=y_val_seq_3,
    num_features=len(feature_cols_3),
    seq_len=SEQ_LEN,
    batch_size=64,
    lr=0.0005,
    num_epochs=40,
    patience=8,
    device=device
)

# 4. Validation evaluation
cnn_val_output_3 = evaluate_cnn_model(
    cnn_model_3,
    X_val_seq_3,
    y_val_seq_3,
    device=device
)

print("FD003 CNN Validation Results:")
print("MAE:", cnn_val_output_3['MAE'])
print("RMSE:", cnn_val_output_3['RMSE'])
print("R2:", cnn_val_output_3['R2'])
print()

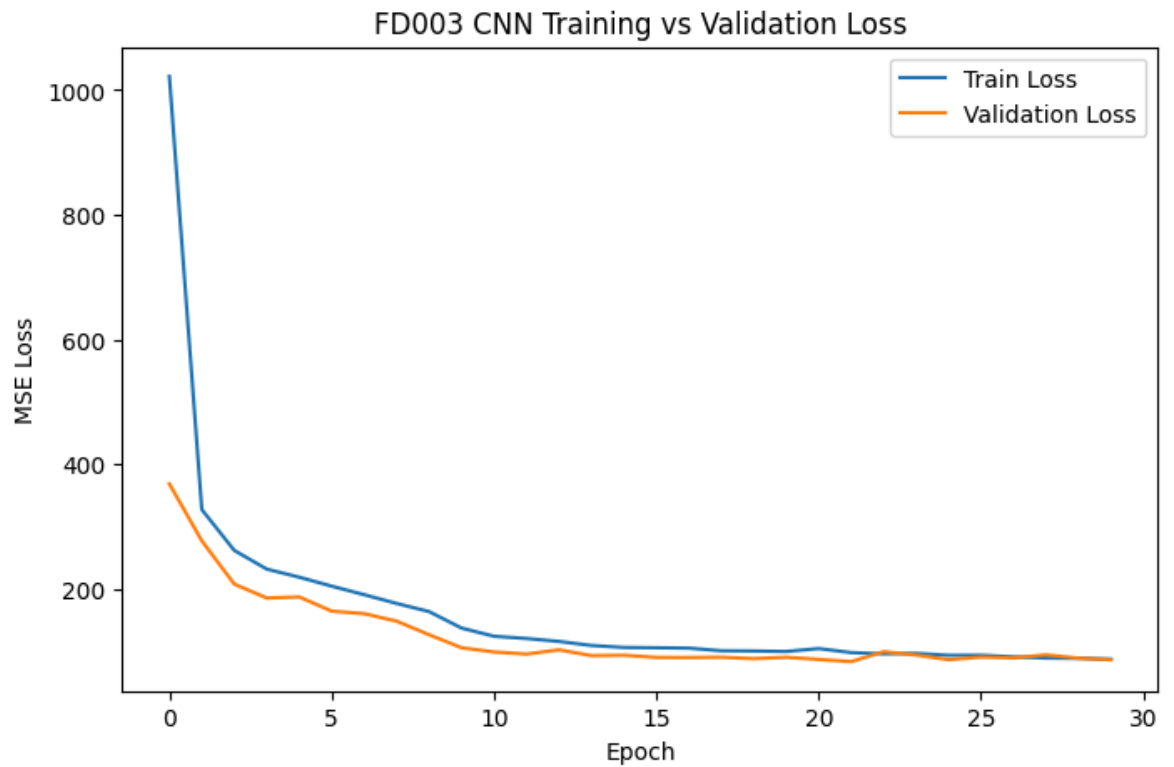
# 5. Official test evaluation
cnn_test_output_3 = evaluate_cnn_model(
    cnn_model_3,
    X_test_seq_3,
    y_test_seq_3,
    device=device
)

```

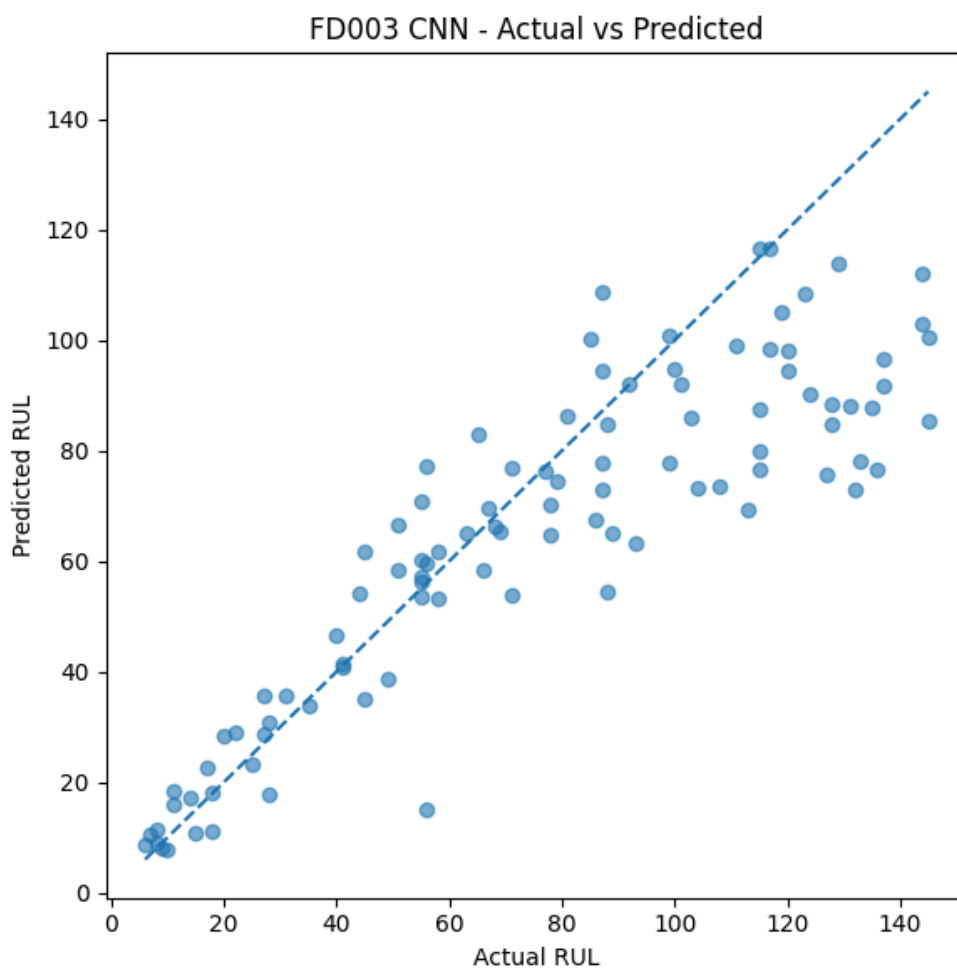
```
)  
  
y_pred_cnn_3 = cnn_test_output_3['y_pred']  
  
print("FD003 CNN Test Results:")  
print("MAE:", cnn_test_output_3['MAE'])  
print("RMSE:", cnn_test_output_3['RMSE'])  
print("R2:", cnn_test_output_3['R2'])
```

```
FD003 CNN train sequence shape: (7760, 30, 18)  
FD003 CNN val sequence shape: (1940, 30, 18)  
FD003 CNN test sequence shape: (100, 30, 18)  
Epoch [1/40], Train Loss: 1022.0900, Val Loss: 368.5047  
Epoch [2/40], Train Loss: 327.2470, Val Loss: 277.0104  
Epoch [3/40], Train Loss: 261.7019, Val Loss: 207.6242  
Epoch [4/40], Train Loss: 231.7463, Val Loss: 185.3491  
Epoch [5/40], Train Loss: 218.7836, Val Loss: 187.0734  
Epoch [6/40], Train Loss: 204.3705, Val Loss: 164.3308  
Epoch [7/40], Train Loss: 190.7084, Val Loss: 160.2309  
Epoch [8/40], Train Loss: 176.6403, Val Loss: 148.2519  
Epoch [9/40], Train Loss: 163.6065, Val Loss: 126.4545  
Epoch [10/40], Train Loss: 137.1405, Val Loss: 105.5870  
Epoch [11/40], Train Loss: 124.0063, Val Loss: 98.8203  
Epoch [12/40], Train Loss: 120.4661, Val Loss: 95.4088  
Epoch [13/40], Train Loss: 115.7179, Val Loss: 102.2835  
Epoch [14/40], Train Loss: 109.2384, Val Loss: 92.9355  
Epoch [15/40], Train Loss: 106.0553, Val Loss: 93.5594  
Epoch [16/40], Train Loss: 105.5917, Val Loss: 90.0649  
Epoch [17/40], Train Loss: 105.0051, Val Loss: 89.9191  
Epoch [18/40], Train Loss: 100.7528, Val Loss: 90.4093  
Epoch [19/40], Train Loss: 100.3934, Val Loss: 88.1539  
Epoch [20/40], Train Loss: 99.5909, Val Loss: 90.3175  
Epoch [21/40], Train Loss: 104.3239, Val Loss: 86.8658  
Epoch [22/40], Train Loss: 97.7943, Val Loss: 83.3335  
Epoch [23/40], Train Loss: 95.8510, Val Loss: 99.3148  
Epoch [24/40], Train Loss: 96.5066, Val Loss: 93.8619  
Epoch [25/40], Train Loss: 93.4480, Val Loss: 86.4386  
Epoch [26/40], Train Loss: 93.5391, Val Loss: 90.6703  
Epoch [27/40], Train Loss: 90.9968, Val Loss: 89.3667  
Epoch [28/40], Train Loss: 89.0575, Val Loss: 94.1563  
Epoch [29/40], Train Loss: 88.6572, Val Loss: 88.9269  
Epoch [30/40], Train Loss: 87.2850, Val Loss: 86.2888  
Early stopping triggered.  
FD003 CNN Validation Results:  
MAE: 6.864349365234375  
RMSE: 9.388211313009249  
R2: 0.8875784277915955  
  
FD003 CNN Test Results:  
MAE: 16.198043823242188  
RMSE: 23.006657176001525  
R2: 0.691114068031311
```

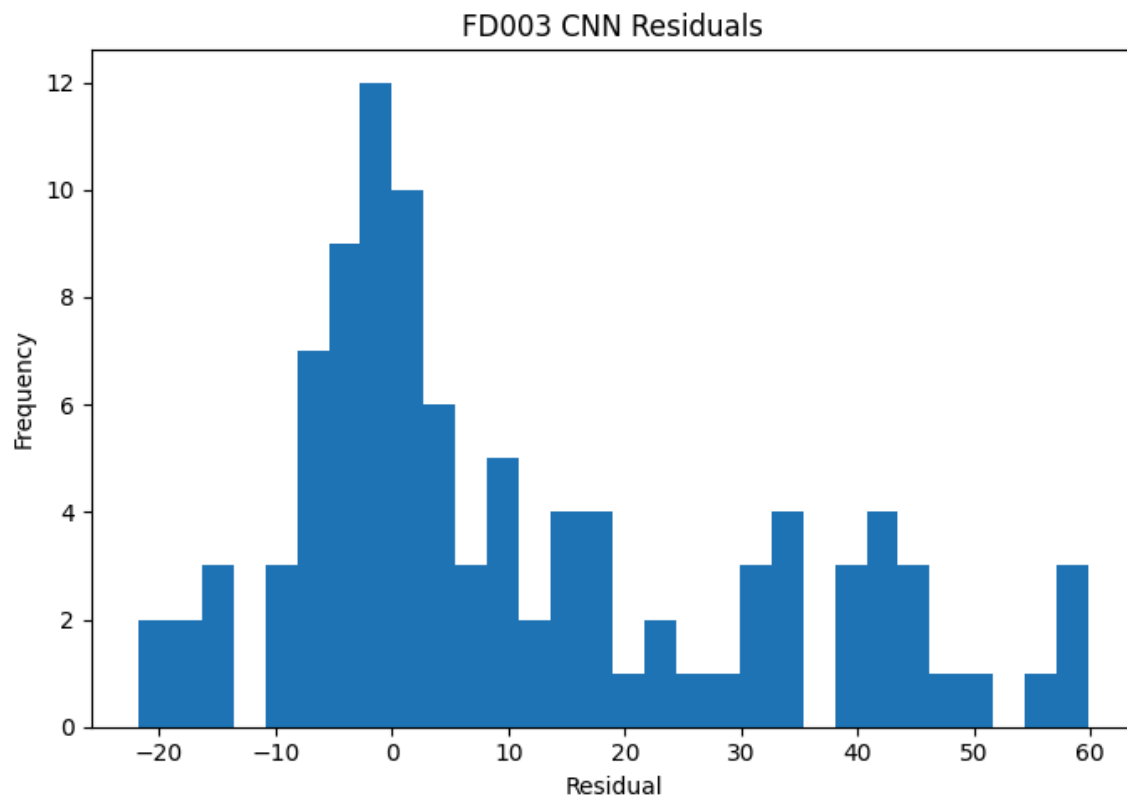
```
In [142]: ▶ plt.figure(figsize=(8,5))
plt.plot(train_losses_3, label="Train Loss")
plt.plot(val_losses_3, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("FD003 CNN Training vs Validation Loss")
plt.legend()
plt.show()
```



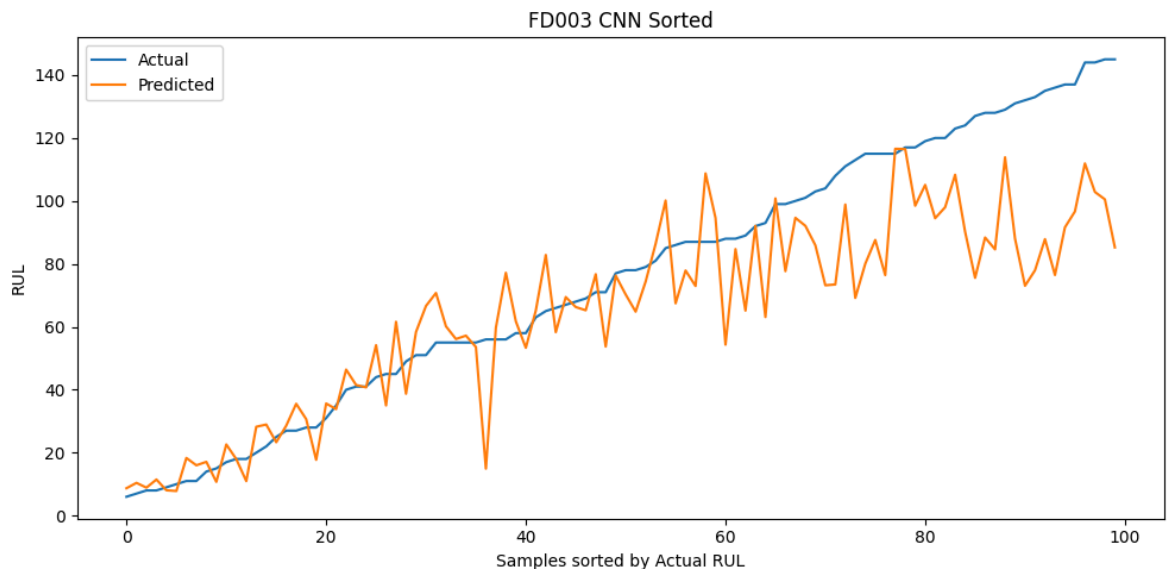
```
In [143]: plot_actual_vs_predicted(y_test_seq_3, y_pred_cnn_3, title="FD003 CNN - Actual vs Predicted")
```



```
In [144]: plot_residual_histogram(y_test_seq_3, y_pred_cnn_3, title="FD003 CNN Residuals")
```



```
In [145]: plot_sorted_actual_vs_predicted(y_test_seq_3, y_pred_cnn_3, title="FD003 CNN Sorted")
```



FD003 CNN Model – Observations

- The CNN achieved a strong **validation R^2 score of approximately 0.89**, indicating that it was able to learn meaningful temporal degradation patterns from the FD003 sequence data.
- However, on the official test set, the performance dropped to an **R^2 score of approximately 0.69**, showing a noticeable gap between validation and test results.
- This suggests that the CNN model is able to fit the validation data well, but its **generalization to unseen FD003 test engines is weaker** than expected.
- Compared to the best traditional model on FD003 (**SVR, $R^2 \approx 0.76$**), the CNN does **not outperform** the tabular approach on the test set.

Interpretation

- FD003 contains **one operating condition** but **two fault modes**, making it more complex than FD001.
- Although the single operating condition makes the data cleaner than FD002, the presence of multiple fault patterns likely increases variability in degradation behavior.
- As a result, the CNN captures temporal structure effectively during validation, but this advantage does not transfer as strongly to the official test set.

Comparison with Previous Datasets

- **FD001**: CNN clearly outperformed traditional models and generalized well.
- **FD002**: CNN provided only a small improvement over traditional models.
- **FD003**: CNN achieved strong validation performance, but **SVR remained the more reliable model on test data**.

Key Insight

The FD003 results show that **deep learning is not always the best-performing approach**, even when temporal sequence information is available.

For this dataset, the CNN learned useful patterns, but a simpler traditional model (**SVR**) generalized better on the official test set.

FD004 - Data Loading and Initial Inspection

```
In [160]: train_df_4, test_df_4, rul_df_4 = load_cmapss_data(
    "data/CMAPS/train_FD004.txt",
    "data/CMAPS/test_FD004.txt",
    "data/CMAPS/RUL_FD004.txt"
)

train_df_4 = add_train_rul(train_df_4)
test_df_4 = add_test_rul(test_df_4, rul_df_4)
```

C:\Users\karam\ML_PROJECT\rul_functions.py:701: FutureWarning: SeriesGroupBy.fillna is deprecated and will be removed in a future version. Use obj.ffill() or obj.bfill() for forward or backward filling instead. If you want to fill with a single value, use Series.fillna instead

C:\Users\karam\ML_PROJECT\rul_functions.py:701: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.

C:\Users\karam\ML_PROJECT\rul_functions.py:701: FutureWarning: SeriesGroupBy.fillna is deprecated and will be removed in a future version. Use obj.ffill() or obj.bfill() for forward or backward filling instead. If you want to fill with a single value, use Series.fillna instead

C:\Users\karam\ML_PROJECT\rul_functions.py:701: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.

C:\Users\karam\ML_PROJECT\rul_functions.py:701: FutureWarning: SeriesGroupBy.fillna is deprecated and will be removed in a future version. Use obj.ffill() or obj.bfill() for forward or backward filling instead. If you want to fill with a single value, use Series.fillna instead

C:\Users\karam\ML_PROJECT\rul_functions.py:701: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.

C:\Users\karam\ML_PROJECT\rul_functions.py:701: FutureWarning: SeriesGroupBy.fillna is deprecated and will be removed in a future version. Use obj.ffill() or obj.bfill() for forward or backward filling instead. If you want to fill with a single value, use Series.fillna instead

C:\Users\karam\ML_PROJECT\rul_functions.py:701: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.

```
In [147]: basic_dataset_summary(train_df_4, "FD004 Training Data")
```

```
missing_summary_4 = missing_values_summary(train_df_4)
unique_summary_4 = unique_value_summary(train_df_4)
```

FD004 Training Data Shape: (61249, 27)

	unit_nr	cycle	op_setting_1	op_setting_2	op_setting_3	sensor_1	sensor_2	sensor_3	sensor_4	sensor_5	..
0	1	1	42.0049	0.8400	100.0	445.00	549.68	1343.43	1112.93	3.91	..
1	1	2	20.0020	0.7002	100.0	491.19	606.07	1477.61	1237.50	9.35	..
2	1	3	42.0038	0.8409	100.0	445.00	548.95	1343.12	1117.05	3.91	..
3	1	4	42.0000	0.8400	100.0	445.00	548.70	1341.24	1118.03	3.91	..
4	1	5	25.0063	0.6207	60.0	462.54	536.10	1255.23	1033.59	7.05	..

5 rows × 27 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 61249 entries, 0 to 61248
Data columns (total 27 columns):
#   Column                Non-Null Count  Dtype
---  -
0   unit_nr                61249 non-null  int64
1   cycle                  61249 non-null  int64
2   op_setting_1           61249 non-null  float64
3   op_setting_2           61249 non-null  float64
4   op_setting_3           61249 non-null  float64
5   sensor_1               61249 non-null  float64
6   sensor_2               61249 non-null  float64
7   sensor_3               61249 non-null  float64
8   sensor_4               61249 non-null  float64
9   sensor_5               61249 non-null  float64
10  sensor_6               61249 non-null  float64
11  sensor_7               61249 non-null  float64
12  sensor_8               61249 non-null  float64
13  sensor_9               61249 non-null  float64
14  sensor_10              61249 non-null  float64
15  sensor_11              61249 non-null  float64
16  sensor_12              61249 non-null  float64
17  sensor_13              61249 non-null  float64
18  sensor_14              61249 non-null  float64
19  sensor_15              61249 non-null  float64
20  sensor_16              61249 non-null  float64
21  sensor_17              61249 non-null  int64
22  sensor_18              61249 non-null  int64
23  sensor_19              61249 non-null  float64
24  sensor_20              61249 non-null  float64
25  sensor_21              61249 non-null  float64
26  RUL                    61249 non-null  int64
dtypes: float64(22), int64(5)
memory usage: 12.6 MB
None
```

Missing Count	Missing %
---------------	-----------

No missing values found.

	Column	Unique Values
4	op_setting_3	2
23	sensor_19	2
20	sensor_16	2
5	sensor_1	6
9	sensor_5	6
22	sensor_18	6
14	sensor_10	21
10	sensor_6	46
21	sensor_17	54
3	op_setting_2	105
0	unit_nr	249
17	sensor_13	483
2	op_setting_1	536
1	cycle	543
26	RUL	543
24	sensor_20	652
15	sensor_11	737
12	sensor_8	1038
6	sensor_2	1704
16	sensor_12	5627
11	sensor_7	5926
19	sensor_15	11915
7	sensor_3	13558
18	sensor_14	15938
8	sensor_4	17353
25	sensor_21	21574
13	sensor_9	25297

```
In [148]: ▶ low_variance_df_4 = low_variance_summary(train_df_4)
zero_std_cols_4 = zero_std_columns(train_df_4)
```

Columns with variance <= 1e-08:

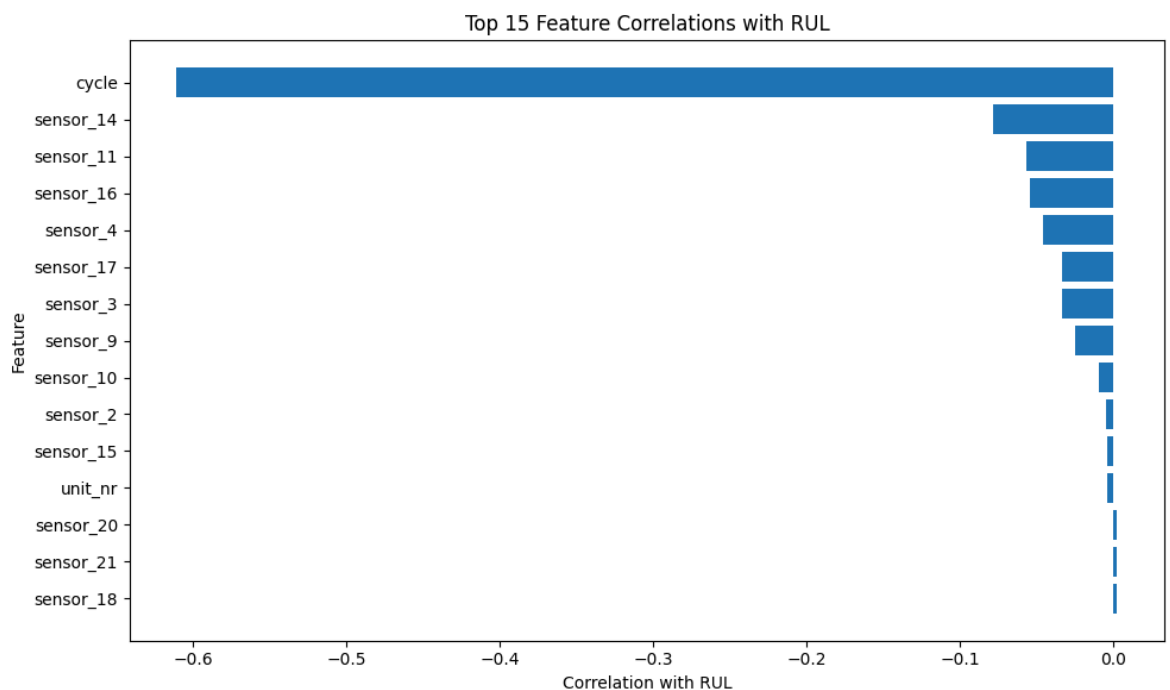
Column	Variance
--------	----------

Zero standard deviation columns:

[]

```
In [149]: plot_top_correlations_with_rul(train_df_4, top_n=15)
```

	Feature	Correlation_with_RUL	Abs_Correlation
1	cycle	-0.610620	0.610620
18	sensor_14	-0.078126	0.078126
15	sensor_11	-0.056639	0.056639
20	sensor_16	-0.053804	0.053804
8	sensor_4	-0.045881	0.045881
21	sensor_17	-0.032939	0.032939
7	sensor_3	-0.032924	0.032924
13	sensor_9	-0.024727	0.024727
14	sensor_10	-0.008924	0.008924
6	sensor_2	-0.004443	0.004443
19	sensor_15	-0.003957	0.003957
0	unit_nr	-0.003656	0.003656
24	sensor_20	0.002812	0.002812
25	sensor_21	0.002791	0.002791
22	sensor_18	0.002765	0.002765
2	op_setting_1	-0.002380	0.002380
4	op_setting_3	0.002303	0.002303
23	sensor_19	0.002303	0.002303
3	op_setting_2	-0.002280	0.002280
12	sensor_8	0.002086	0.002086
5	sensor_1	0.001889	0.001889
9	sensor_5	0.001679	0.001679
16	sensor_12	-0.001639	0.001639
17	sensor_13	0.001501	0.001501
11	sensor_7	-0.001429	0.001429
10	sensor_6	0.001349	0.001349



```

In [150]: drop_cols_4 = [
            'sensor_1',
            'sensor_5',
            'sensor_18'
          ]

In [161]: train_df_4 = drop_columns(train_df_4, drop_cols_4)
          test_df_4 = drop_columns(test_df_4, drop_cols_4)

In [162]: train_df_4 = filter_late_cycles(train_df_4, threshold=125)

In [163]: train_split_4, val_split_4 = engine_level_train_val_split(train_df_4)

In [164]: feature_cols_4 = get_feature_columns(train_split_4)
          test_last_4 = get_last_cycle_per_engine(test_df_4)

In [165]: X_train_4, X_val_4, X_test_4, y_train_4, y_val_4, y_test_4, scaler_4 = scale_tabular_feature
          train_split_4,
          val_split_4,
          test_last_4,
          feature_cols_4
        )

In [166]: models_4 = train_regression_models(X_train_4, y_train_4)

Training Linear Regression...
Training KNN...
Training Decision Tree...
Training Random Forest...
Training SVR...
Training XGBoost...

In [167]: val_results_4 = evaluate_all_models(models_4, X_val_4, y_val_4)
          test_results_4 = evaluate_all_models(models_4, X_test_4, y_test_4)

```

	Model	MAE	RMSE	R2
0	XGBoost	15.703031	20.948773	0.668270
1	Random Forest	15.885086	21.095723	0.663600
2	Decision Tree	17.500705	23.109178	0.596321
3	SVR	17.975563	23.217748	0.592519
4	Linear Regression	18.716328	23.555963	0.580561
5	KNN	18.745111	24.042596	0.563052

	Model	MAE	RMSE	R2
0	XGBoost	25.775919	35.958941	0.565048
1	Random Forest	26.128669	36.702881	0.546864
2	Decision Tree	28.240189	38.948445	0.489720
3	Linear Regression	29.785915	39.061966	0.486742
4	SVR	28.764902	39.260147	0.481520
5	KNN	29.897581	39.972480	0.462535

FD004 Model Observations

- The best-performing model on FD004 was **XGBoost**, achieving a test R^2 score of approximately **0.57**, followed closely by Random Forest (~0.55).
- Overall performance is **significantly lower than FD001 and FD003**, and slightly lower than FD002.

Interpretation

- FD004 is the most complex dataset, containing:
 - **Multiple operating conditions (6)**
 - **Multiple fault modes (2)**
- This combination introduces high variability in the data, making it difficult for models to learn consistent relationships between sensor values and RUL.
- Correlation analysis showed that most features have **very weak relationships with RUL**, except for the cycle variable.
- This indicates that **individual sensor readings are less informative**, and the mapping between inputs and target is less stable.

Model Behavior

- Tree-based models (**XGBoost and Random Forest**) performed best, likely because they:
 - capture nonlinear relationships
 - handle noisy and heterogeneous data better
- Linear models and SVR performed worse, as the underlying relationships are no longer simple or smooth.

Comparison with Previous Datasets

- **FD001**: Highest performance (~0.85–0.90 R²), simple structure
- **FD002**: Lower performance (~0.60–0.70 R²), multiple conditions
- **FD003**: Moderate performance (~0.70–0.80 R²), multiple fault modes
- **FD004**: Lowest performance (~0.55–0.60 R²), both conditions and fault modes vary

Key Insight

FD004 demonstrates that when both **operating conditions and fault modes vary simultaneously**, the prediction task becomes significantly more difficult.

Final Model Comparison Across Datasets

Dataset	Best Model	MAE	RMSE	R ²
FD001	CNN (Optuna)	~9.7	~12.9	~0.90
FD002	CNN	~21.1	~30.0	~0.69
FD003	SVR	~15.4	~20.4	~0.76
FD004	XGBoost	~25.8	~36.0	~0.57

Final Conclusion

This project investigated the prediction of Remaining Useful Life (RUL) using both traditional machine learning models and deep learning approaches across four C-MAPSS datasets of increasing complexity.

Key Findings

- **FD001 (Simplest Dataset):**
 - CNN achieved the best performance (~0.90 R²)
 - Strong temporal patterns allowed deep learning to outperform traditional models significantly
- **FD002 (Multiple Operating Conditions):**
 - Performance dropped (~0.69 R²)
 - CNN provided only marginal improvement, even after hyperparameter tuning, indicating that performance is constrained by dataset complexity rather than model configuration
 - Variability in operating conditions reduced model effectiveness
- **FD003 (Multiple Fault Modes):**
 - Moderate performance (~0.75 R²)
 - CNN achieved strong validation results but did not outperform SVR on test data

- Indicates challenges in generalizing across different fault patterns
- **FD004 (Most Complex Dataset):**
 - Lowest performance ($\sim 0.57 R^2$)
 - Weak feature–target relationships
 - Feature engineering and model tuning provided minimal improvements
 - Tree-based models (XGBoost, Random Forest) performed best

Overall Insights

- Model performance is heavily influenced by dataset complexity
- Deep learning is highly effective when strong temporal patterns exist (FD001), but less reliable in highly variable datasets
- Traditional models, particularly SVR and XGBoost, provide more stable performance across complex datasets
- Feature engineering and hyperparameter tuning provide only limited improvements when the underlying data lacks strong predictive signals, highlighting the importance of dataset characteristics over model complexity.

Final Takeaway

There is no single model that performs best across all scenarios.

The choice of model should depend on the structure and complexity of the dataset.

This project demonstrates that understanding the data is more important than increasing model complexity, as model performance is ultimately constrained by the structure and quality of the dataset.

Key Takeaways

- FD001 → clean patterns → CNN shines
 - FD002 → multiple conditions → harder
 - FD003 → multiple faults → mixed results
 - FD004 → hardest → weak signal Deep learning is NOT always better
- Data complexity determines performance
- Best models:
- CNN (FD001, FD002)
 - SVR (FD003)
 - XGBoost (FD004)

Limitations and Future Work

- Performance is limited by weak feature–target relationships in complex datasets
- Additional feature engineering (e.g., advanced temporal features) may provide small improvements
- More advanced architectures (e.g., LSTM, Transformers) could be explored
- Domain-specific knowledge could improve feature representation

Future work could focus on improving generalization across varying operating conditions and fault modes.

In []: ▶